

系统设计与工程保证工作手册

六篇完整草案 · 合订工作版

ESD-HANDBOOK

vpreview.2

DRAFT / COMBINED WORKING VIEW

权威内容来源为 Markdown。本件仅供阅读与评审，未获正式发布批准。

阅读与版本说明

PREVIEW / DRAFT · 非权威阅读视图

本件保留源文档的标题、条款编号、版本与草案状态，不构成新的规范权威。封面、导航、索引与排版标签为生成内容，不能代替正文中的适用范围、条件或批准记录。

制品标识 ESD-HANDBOOK

版本 preview.2

状态 DRAFT / COMBINED WORKING VIEW

阅读密度 balanced · Visual Profile Candidate，尚未冻结

基准 commit 783c7d3b9b6a1e4f715a960c18e687604143f98b

准备标识 353090fade94323347038eee

使用方式 目录与书签可跳转；合订版中的核心文档链接跳转到本件对应篇章。独立版在源字节匹配时提供固定 commit 链接，否则指向随附的 source 快照。完整输入以准备记录为准，可能包含未提交变更；正文原有编号不重新连续编号。

ESD-SUITE-000 v2.0.0-draft.1 DRAFT / SUITE CONTROL

ESD-METHOD-001 v2.0.0-draft.1 DRAFT / NORMATIVE FOUNDATION

ESD-REFERENCE-001 v2.0.0-draft.1 DRAFT / OPERATIONAL REFERENCE

ESD-GOV-001 v2.0.0-draft.1 DRAFT / NORMATIVE CONTROL

ESD-ASSURANCE-001 v2.0.0-draft.1 DRAFT / NORMATIVE DOMAIN

ESD-OPS-001 v2.0.0-draft.1 DRAFT / NORMATIVE DOMAIN

按条款 ID 查找 末尾索引含可点击标题和纸面页码。完整章节与条款通过分组书签定位。

目录

阅读与版本说明	2
优秀系统设计与工程保证标准：文档体系与规范关系	9
1. 套系目标与非目标	10
2. 文档地图	10
3. 规范权威与冲突裁决	12
4. 规范性级别	13
5. 套系版本与文档版本	14
6. 文档生命周期	15
7. 风险等级与使用剖面	16
8. 角色与阅读路径	17
9. 标准、模板、案例与自动化的边界	18
10. 文件、链接与单一事实来源	19
11. 套系治理	19
12. 符合性声明的最小形式	20
13. 当前修订与发布状态	21
附录 A：套系检查清单	22
附录 B：结论	22
优秀系统设计与工程保证方法论	23
1. 文档定位	24
2. 严格定义：什么是优秀的系统	25
3. 认识论边界：工程保证能够证明什么	25
4. 系统设计十条总纲	27
5. 完整的架构推导模型	29
6. 从问题、约束与风险开始	30
7. 关键性质：从愿望到带作用域的契约	32
8. 状态、身份、边界与所有权	34

9. 失败、一致性、提交与恢复	36
10. 有界资源、背压与动态稳定性	38
11. 容量、单位经济性与复杂度预算	40
12. 契约、安全、演进与可逆性	42
13. 工程保证：从设计主张到持续证据	43
14. 组织、成熟度与渐进式采用	46
15. 最终判定标准	47
附录 A：核心术语边界	48
附录 B：参考资料	48
附录 C：历史 v1.1.0 关键校准	49
优秀系统设计：从约束、不变量到证据	50
1. 操作模型	51
2. 风险分级与裁剪	52
3. Stage 0: Problem Brief 与风险定级	54
4. Stage 1: Workload、SLO、约束与假设	55
5. Stage 2: Critical Properties 与优先级	56
6. Stage 3: State、Identity、Ownership 与 Boundary Contract	57
7. Stage 4: Failure、Consistency、Commit 与 Recovery	58
8. Stage 5: Architecture、Planes、Pipeline 与 Modes	60
9. Stage 6: Resource Bounds、Capacity、Economics 与 Control Loops	63
10. Stage 7: Contracts、Security、Evolution 与 Operations	65
11. Stage 8: Verification、Evidence 与 Change Envelope	66
12. Stage 9: Production Readiness 与持续保证	68
13. 可复用工程模板	69
14. Architecture Review 会议操作	79
15. 常见反模式与校正	80
16. 完整案例：对象存储 CAN 分块文件处理流水线	82

17. Production Readiness Checklist	100
18. 最小 19 问设计清单	102
19. 团队成熟度与采用路线	103
20. 文档与标准维护	103
系统设计与工程保证治理及符合性规范	105
1. 适用范围	106
2. 核心治理原则	107
3. 受管对象与变更分类	108
4. 风险分级	110
5. 角色与责任	111
6. 职责分离	113
7. 最低工程制品	114
8. Gate 治理	115
9. 例外、豁免与风险接受	117
10. 符合性模型	118
11. 符合性评估流程	127
12. 设计评审治理	129
13. 生产放行治理	130
14. 紧急变更治理	131
15. 标准偏离与领域 Profile	131
16. 审计与记录保留	132
17. 治理指标与反 Goodhart	133
18. 渐进式采用	134
19. 符合性声明模板	135
20. Gate 决策模板	135
21. Exception 模板	136
22. 最终符合性检查清单	137
23. 结论	138

工程保证论证、证据与裁决规范	139
1. 适用范围与非目标	140
2. 核心术语	141
3. 工程保证基本原则	141
4. Assurance Object 与身份	143
5. Claim Contract	144
6. Context、Assumption 与 Guarantee	146
7. Argument 结构	148
8. Defeater 与 Counterevidence	150
9. Evidence 分类	151
10. Evidence Quality Model	152
11. Evidence Envelope	153
12. 结果状态的严格区分	155
13. Evidence 生命周期与失效	156
14. Verification Plan	157
15. 工具与验证器可信度	159
16. 覆盖、代表性与采样	160
17. 组合保证	161
18. Assurance Case 充分性	162
19. Evidence Conclusion 与 Claim Status	163
20. Gate 与发布裁决	164
21. Release Evidence Bundle	164
22. Change Envelope	166
23. 运行时持续保证	166
24. Incident → Assurance Delta	167
25. Assurance Plane 元不变量	169
26. 角色与独立性	170

27. 常见反模式	170
28. Assurance Case 模板	171
29. Verification & Evidence Matrix 模板	173
30. Evidence Review Checklist	173
31. Release Evidence Bundle Manifest 示例	174
32. 最终判定标准	175
生产就绪与运行韧性标准	176
1. 适用范围与非目标	177
2. 核心生产原则	178
3. 生产服务定义	179
4. 生产就绪状态模型	181
5. SLI、SLO 与错误预算	182
6. 运行模式	184
7. 故障域与依赖韧性	186
8. 资源边界、容量与恢复余量	188
9. 准入、背压与过载	191
10. 发布策略与 Change Envelope	193
11. 回滚、前向修复与不可逆变更	195
12. 数据完整性、备份与恢复	196
13. RPO、RTO 与灾难恢复	198
14. 可观测性与诊断	200
15. Runbook 与操作安全	203
16. On-call 与运营所有权	204
17. 安全与运行控制	205
18. 第三方与供应商韧性	206
19. 事故响应	207

20. 韧性验证与演练	209
21. Production Readiness Review	210
22. 上线后观察与扩大	211
23. 持续运行评审	212
24. 按风险等级的最低运行要求	213
25. Production Readiness Decision 模板	214
26. Operating Mode Contract 模板	215
27. Dependency Resilience Contract 模板	215
28. Recovery Drill Record 模板	216
29. 最终 Production Readiness Checklist	217
30. 结论	218
条款定位索引（生成导航）	219

优秀系统设计与工程保证标准：文档体系与规范关系

规范地图、权威边界、版本治理与使用路径

文档属性	内容
文档编号	ESD-SUITE-000
文档版本	2.0.0-draft.1 (基于 1.0.0)
套系版本	2.0.0-draft.1
状态	DRAFT / SUITE CONTROL；未获发布批准
文档角色	定义整套标准的组成、规范权威、冲突裁决、版本与生命周期
适用对象	标准 Owner、架构负责人、技术负责人、评审者、SRE、平台、安全与审计人员
权威格式	本 Markdown 文件；PDF、网页与其他导出仅为发布视图

控制声明

本文不重新定义系统设计方法，也不替代项目级设计制品。它控制“哪些文档构成标准、每篇文档对什么主题拥有权威、发生冲突时如何裁决，以及团队应沿哪条路径使用这些材料”。

摘要

“优秀系统设计与工程保证标准”不是若干相似文章的集合，而是一套分层、可裁剪、可审计的工程规范体系。它将系统设计拆分为五类相互配合但职责不同的制品：

方法论基础

- > 治理与符合性
- > 论证、证据与裁决
- > 生产就绪与运行韧性
- > 操作手册、模板、案例与自动化

任何单篇文档都不能独立完成系统治理：方法论给出推导逻辑，治理规范定义谁必须做什么，证据规范定义“凭什么相信”，运行标准定义生产系统如何持续保持可接受行为，操作手册则把这些原则转化为阶段、门禁与项目制品。

本套系坚持三个控制原则：

1. **Markdown 是唯一权威源**：排版格式不得成为平行事实来源；
2. **规范权威按主题划分**：不以文件编号或篇幅替代语义边界；
3. **所有裁剪必须显式**：允许风险匹配的轻量化，不允许静默省略关键状态、责任、风险或证据。

1. 套系目标与非目标

1.1 目标

本套系用于建立以下组织能力：

- 从业务问题、约束、风险和工作负载推导架构，而不是从组件反推理由；
- 将关键正确性、安全性、活性、持久性、资源和运行目标表达为具有类型匹配验证义务的性质；
- 明确权威状态、稳定身份、变更权限、技术 Owner 和运营 Owner；
- 对重复、乱序、部分成功、过载、依赖退化、恢复和迁移建立可执行语义；
- 将设计主张映射为机制、验证、证据、门禁和运行反馈；
- 允许不同风险等级采用不同厚度的流程，同时保持认识论边界不被削弱；
- 让标准、项目文档、自动化规则和发布证据能够互相追溯。

1.2 非目标

本套系不用于：

- 规定所有项目必须采用相同架构、语言或基础设施；
- 以文档数量、模板完成率或 Gate 数量替代风险降低；
- 通过中央审批替代明确所有权和自助式工程能力；
- 对未编码、未验证或超出覆盖范围的性质作绝对正确性承诺；
- 将探索项目强行套入核心生产系统的全部流程；
- 把操作手册、案例或自动化实现提升为不可挑战的最高原则。

2. 文档地图

2.1 核心文档

表 2 · 字段记录视图（列名与原单元格逐项对应）

表 2 · 记录 1 · 00	
顺序	00
文档编号	ESD-SUITE-000
文档名称	《优秀系统设计与工程保证标准：文档体系与规范关系》
文档版本	2.0.0-draft.1
规范角色	套系控制、规范地图与冲突裁决

表 2 · 记录 2 · 01	
顺序	01
文档编号	ESD-METHOD-001
文档名称	《优秀系统设计与工程保证方法论》
文档版本	2.0.0-draft.1
规范角色	术语、原则、认识边界与架构推导基础

表 2 · 记录 3 · 02	
顺序	02
文档编号	ESD-REFERENCE-001
文档名称	《优秀系统设计：从约束、不变量到证据》
文档版本	2.0.0-draft.1
规范角色	Stage/Gate、模板、评审流程与工程案例

表 2 · 记录 4 · 03	
顺序	03
文档编号	ESD-GOV-001
文档名称	《系统设计与工程保证治理及符合性规范》
文档版本	2.0.0-draft.1
规范角色	适用范围、风险分级、角色、例外与符合性

表 2 · 记录 5 · 04	
顺序	04
文档编号	ESD-ASSURANCE-001
文档名称	《工程保证论证、证据与裁决规范》
文档版本	2.0.0-draft.1
规范角色	Assurance Case、证据质量、失效与裁决

表 2 · 记录 6 · 05	
顺序	05
文档编号	ESD-OPS-001
文档名称	《生产就绪与运行韧性标准》
文档版本	2.0.0-draft.1
规范角色	SLO、发布、恢复、过载、值班与持续韧性

2.2 后续扩展类别

核心规范之外的材料必须归入以下类别，不得与核心规范混杂：

templates/	可实例化、机器可解析的项目模板
examples/	非规范性完整参考案例与反例
profiles/	数据、控制面、AI、实时系统等领域收紧规则
automation/	Schema、Lint、CI Gate 与 Evidence Bundle 规则

这些扩展可以增加领域要求，但不得静默改变核心术语、UNKNOWN 语义、状态权威、证据绑定或例外治理规则。

3. 规范权威与冲突裁决

3.1 按主题确定权威

规范权威不采用简单的“文件编号越小越高”模型，而按主题划分：

主题	最终权威文档
核心术语、系统定义、认识论边界、关键设计原则	ESD-METHOD-001
适用范围、风险分级、角色、职责分离、Gate 状态与权限、例外、符合性声明	ESD-GOV-001
Claim、Argument、Assumption 状态、Evidence、Evidence Envelope、证据时效与裁决充分性	ESD-ASSURANCE-001
SLO、运行模式、发布、回滚、恢复、容量、值班、事故与生产韧性	ESD-OPS-001
Stage/Gate 默认流程、可复用模板、检查清单和参考案例	ESD-REFERENCE-001
文档组成、状态、文件命名、版本关系和套系冲突规则	ESD-SUITE-000

3.2 冲突处理顺序

发生冲突时必须按以下顺序处理：

1. 判断冲突是否来自术语误用、适用范围不同或版本不兼容；
2. 根据上表确定该主题权威文档；
3. 若权威文档仍存在歧义，创建标准解释记录，而不是由项目自行发明永久语义；
4. 在解释完成前，高风险项目对相关结论保持 UNKNOWN；
5. 若项目确需偏离规范，依 GOV 的授权程序办理 Exception 或偏离评审；ADR 只能记录决策及引用授权，不能自行豁免要求，也不能修改事实描述来伪造符合。

3.3 方法论与操作手册的关系

ESD-METHOD-001 与 ESD-REFERENCE-001 的关系为：

METHOD

定义：为什么、是什么、能够证明到什么范围

REFERENCE

实例化：按什么顺序工作、提交什么、如何评审与复用

操作手册可以提供默认流程，但不能：

- 把所有目标都泛化为 **invariant**；
- 把无证据状态解释为通过；
- 通过模板省略状态身份、Owner、失败语义或资源上界；
- 用项目便利改变方法论中的权威边界。

4. 规范性级别

4.1 文档级别

级别	含义
normative-foundation	定义术语、原则和不可被下位材料静默改变的语义
normative-control	定义体系组成、角色、生命周期、符合性与裁决控制
normative-domain	对特定主题提供强制工程要求，如证据或生产运行
operational-reference	提供默认流程、模板、示例和推荐实践
informative	解释、案例、教程和背景，不单独形成强制要求

4.2 需求用语

核心规范统一采用以下词义：

- **必须 (MUST)**：不满足会破坏关键正确性、安全性、恢复能力、生产可控性或标准自身可信度；
- **不得 (MUST NOT)**：会形成不可接受风险或破坏认识边界；
- **应 (SHOULD)**：默认要求，偏离必须记录理由、风险、Owner 和复审触发；
- **可以 (MAY)**：由风险、成本和局部约束决定；
- **未知 (UNKNOWN)**：当前证据不足，不能合理判断为符合或不符合；
- **不适用 (NOT_APPLICABLE)**：经论证后与当前对象无关，并有 Reviewer 接受该论证。

4.3 Requirement ID

从本套系新增的规范性要求应拥有稳定 ID：

GOV-REQ-###
ASSUR-REQ-###
OPS-REQ-###

要求 ID 一经发布不得重用于无相关要求。对同一义务的修订必须绑定文档版本并记录语义差异；不兼容修订不得作为 Patch 发布。删除的要求保留为 RETIRED，避免历史裁决失去引用对象。

4.4 要求覆盖与材料性质

编号不是强制级别：正文中的 MUST、MUST NOT、SHOULD 与 MAY 才决定义务；例如 ASSUR-REQ-020 和 OPS-REQ-047 包含 SHOULD。未编号的规范性正文、被明确要求的模板字段、阻断条件同样必须纳入适用性审查，不能只扫描 Requirement ID。

控制索引以 GOV §10.4 为准，将编号条款与未编号段落、字段及 Gate 条件映射到同一组评估记录。未完成覆盖及语义审查前，不得将索引直接当作生产 CI 放行规则。

本套系区分四类来源：外部协议/理论事实需标注来源与版本；工程实践建议需说明适用前提；R0-R3、Gate 0-9、成熟度分级等是本套系的组织治理选择，不宣称为外部标准的原文规定；REFERENCE 案例及模板填充值是说明性材料，不构成已执行证据或项目批准。

5. 套系版本与文档版本

5.1 双版本模型

每篇文档同时包含：

```
suite_version: "<adopted-suite-version>"  
version: "<document-version>"
```

- suite_version 表示该文档被哪一版整体标准采用；
- version 表示该文档自身的语义版本；
- 单篇文档可以独立升级 Patch 或 Minor，不要求所有文档同步变更；
- 当规范关系、核心术语或套系组成出现不兼容变化时，提升 suite Major。

5.2 版本规则

变化	文档版本	套系版本
排版、链接、错字，不改变语义	Patch	通常不变
新增兼容要求、模板或解释	Minor	可能提升 Minor
删除或改变既有强制语义	Major	必须评估 Major
新增核心规范但不破坏既有语义	新文档 1.0.0	提升或保持 Minor，由 Suite Owner 决定
改变权威边界或冲突规则	Major 或 Minor	至少提升 Minor

5.3 兼容性声明

每次发布必须声明：

- 与哪些文档版本组合经过校准；
- 是否存在暂时未更新的 companion document；
- 哪些自动化规则仍停留在旧语义；
- 是否存在迁移期或双轨解释；
- 哪些历史裁决需要重新评审。

6. 文档生命周期

6.1 状态

状态	含义
DRAFT	内容仍可发生显著变化，不应用作强制生产裁决
CANDIDATE	语义基本稳定，正在试点和收集反例
BASELINE	已批准，可作为规范或正式参考
DEPRECATED	仍可读取，但不得用于新项目；有明确替代物
SUPERSEDED	已被新版本完全取代
WITHDRAWN	因错误或风险被撤回，不应继续引用

6.2 状态转换

DRAFT -> CANDIDATE -> BASELINE -> DEPRECATED -> SUPERSEDED
 \-> WITHDRAWN

状态转换必须记录：

- 决策日期；
- 决策主体；
- 变更摘要；
- 影响分析；
- 迁移要求；
- 替代版本；
- 对既有项目和证据的影响。

6.3 复审触发

以下任一事件触发相关文档复审：

- 重大事故或近失事件暴露规范缺口；
- 关键术语在多个项目中产生不一致解释；
- 大量 Exception 指向同一不可执行要求；
- 新法律、合规、安全或业务硬约束；
- 工作负载或基础设施范式显著改变；
- 自动化规则无法忠实表达当前规范；
- 至少每 12 个月一次基线复审。

7. 风险等级与使用剖面

以下是 GOV §4 的阅读摘要，风险定义与定级权限以该处为准。套系统一采用 R0-R3：

等级	典型场景	默认使用方式
R0 – Exploration	可删除实验、无真实高敏数据、无生产依赖	阅读方法论摘要；使用最小 Brief 与退出条件
R1 – Bounded Production	内部或有限用户、影响可隔离	应用治理规范的最小制品与基础证据
R2 – Core Production	核心数据、收入、跨团队依赖、持续运行	完整 Stage/Gate、Assurance Case、Production Readiness
R3 – High Assurance	资金、安全、合规、人身风险或大爆炸半径	独立评审、职责分离、持续证据、严格例外和漂移检测

风险等级决定流程厚度，不改变以下底线：

- 无权威状态不得进入生产；
- 不可逆副作用必须有身份、提交和恢复语义；
- 所有可增长资源必须有界；
- 高后果 UNKNOWN 不得静默放行；
- 例外必须有 Owner、期限、补偿控制和退出路径。

8. 角色与阅读路径

8.1 按角色阅读

角色	必读路径
架构师 / 技术负责人	METHOD -> GOV -> ASSURANCE -> REFERENCE -> OPS
Design Author	REFERENCE -> METHOD -> GOV -> ASSURANCE -> OPS
SRE / Operational Owner	OPS -> REFERENCE -> GOV -> ASSURANCE
Assurance Reviewer	ASSURANCE -> METHOD -> GOV -> REFERENCE
Security / Compliance Reviewer	GOV -> METHOD -> ASSURANCE -> OPS
Release Authority	GOV -> ASSURANCE -> OPS -> 项目 Production Readiness Decision
标准 Owner	全部核心文档，并维护本索引

8.2 按任务阅读

任务	最短路径
新系统立项	REFERENCE Stage 0-3 + METHOD 6-8 + GOV 风险分级
重大架构变更	GOV 变更分类 + REFERENCE Stage 4-9 + ASSURANCE
Production Readiness	OPS + ASSURANCE + REFERENCE Gate 9
事故后改进	OPS 事故治理 + ASSURANCE Assurance Delta
例外或 break-glass	GOV 例外治理 + ASSURANCE 决策记录
建立 CI 门禁	ASSURANCE Evidence Envelope + 后续 ESD-AUTO-001

9. 标准、模板、案例与自动化的边界

9.1 标准

标准定义：

- 必须满足的性质；
- 角色与责任；
- 裁决状态；
- 证据质量；
- 生命周期和例外规则。

9.2 模板

模板是标准的实例化容器，不是标准本身。模板字段可以增加，但不得：

- 删除规范要求的语义；
- 将 UNKNOWN 隐藏为未填写；
- 用单一勾选框替代必要论证；
- 把项目特例提升为全局规则。

9.3 案例

案例用于说明一种符合性实现，始终是非规范性的。案例中的技术选型、数值和架构不得被解释为默认要求。

9.4 自动化

自动化只能确定性裁决已编码属性。任何自动化输出必须说明：

- 检查了什么；
- 没有检查什么；
- 绑定了哪个制品与配置；
- 工具失败时的状态；
- 证据时效与失效条件。

未执行、工具异常或覆盖不足必须返回 UNKNOWN，不得默认成功。

10. 文件、链接与单一事实来源

10.1 文件命名

核心文件采用：

NN-中文标题-vMAJOR.MINOR.PATCH.md

文档 ID 存放在 **Front Matter**，不使用文件名替代稳定身份。草案可保留既有路径，但必须在元数据与 §13 声明版本和迁移安排；正式发布不能以旧版本文件名混淆新内容。

10.2 权威源

- Markdown 文件是唯一权威源；
- PDF 是不可变发布视图；
- 网页是导航与阅读视图；
- 表单和门户是模板实例化入口；
- CI Schema 是已编码子集；
- 任何派生物都必须标明源文件版本和 digest。

10.3 交叉引用

项目和标准引用必须优先使用：

Document ID + Document Version + Requirement ID

示例：

ESD-ASSURANCE-001 v2.0.0-draft.1 / ASSUR-REQ-012（草案引用，不是发布批准）

不得只写“按设计规范执行”，因为这无法审计具体要求和版本。

11. 套系治理

11.1 Suite Owner

Suite Owner 负责：

- 维护文档地图和权威边界；
- 协调跨文档术语与版本；
- 接受或拒绝标准变更提案；
- 组织周期复审；
- 防止重复规范和互相矛盾的要求；
- 确保 UNKNOWN、证据绑定、状态权威等核心语义不被弱化。

Suite Owner 不应独自批准涉及自身高风险变更的全部证据和例外。

11.2 变更提案最低内容

```
proposal ID
affected documents / requirements
problem and evidence
proposed semantic delta
compatibility impact
migration plan
automation impact
open unknowns
reviewers and decision authority
```

11.3 标准解释记录

当规范存在真实歧义但尚不需要修改正文时，可以发布 Interpretation Record。记录必须：

- 引用具体文档与要求；
- 描述歧义；
- 给出临时解释和适用范围；
- 有到期或并入正文的计划；
- 不得长期替代正式版本更新。

12. 符合性声明的最小形式

任何声称“符合本标准”的系统或变更，至少必须说明：

```
system_or_change: "...
risk_tier: "R0|R1|R2|R3"
suite_version: "<adopted-suite-version>"
applicable_documents:
  - id: "ESD-GOV-001"
    version: "<adopted-document-version>"
  - id: "ESD-ASSURANCE-001"
    version: "<adopted-document-version>"
  - id: "ESD-OPS-001"
    version: "<adopted-document-version>"
profile: "...
conformance_result: "CONFORMANT|CONDITIONALLY_CONFORMANT|NONCONFORMANT|UNKNOWN"
open_exceptions: []
blocking_unknowns: []
evidence_bundle: "...
decision_authority: "...
decided_at: "...
review_trigger: "...
```

“使用了部分模板”或“通过了 CI”不能单独构成符合性声明。

13. 当前修订与发布状态

当前六篇源文档为 2.0.0-draft.1，基于套系 1.1.0 修订。Claim 支持语义、符合性分层和受控状态字段涉及不兼容变化，按 Major 草案评审，不作为排版 Patch。文件名暂保留旧版本后缀以稳定现有链接；文件名仅为草案编辑位置，引用须使用文档 ID、正文版本和条款定位。正式发布时再按 §10.1 迁移文件名与导出，不在本轮变更中执行。

事项	当前状态	解释
本轮文档状态	DRAFT	未获 Suite Owner 发布批准
作者/内容批准记录	UNKNOWN	不从旧 BASELINE 标签推定批准记录存在
跨文档语义校准批准	PENDING_REVIEW	文本修正与静态检查不替代独立语义复审
项目试运行	NOT_RUN	尚无本轮 R1/R2 试运行证据
实践有效性	UNKNOWN	不能由文字完整或基线标签推出
PDF	保留既有 v1.1.0	未重新生成；不代表当前草案内容

BASELINE 只表示按 §6 批准的文档基线，不等于经过实践验证。建议正式批准前分别在一个 R1 有界项目和一个 R2 数据处理项目试运行，记录成本、歧义、漏检及门禁效果；这是待执行计划，不是完成声明。

本草案保持六篇文档的角色与主章节结构，其核心关系为：

ESD-SUITE-000	controls the document system
ESD-METHOD-001	defines semantics and epistemic boundaries
ESD-GOV-001	defines applicability, roles, exceptions and conformance
ESD-ASSURANCE-001	defines assurance argument, evidence and decision sufficiency
ESD-OPS-001	defines production readiness and operational resilience
ESD-REFERENCE-001	turns the above into stages, gates, templates and examples

后续模板库、CAN 完整参考设计、自动化规范和领域 Profile 应在该关系下扩展，不得重新复制一套核心方法论。

附录 A：套系检查清单

- ☐ 每篇核心文档拥有唯一 Document ID；
- ☐ 文档版本与 suite version 已区分；
- ☐ 规范主题的最终权威明确；
- ☐ 交叉引用使用稳定 ID；
- ☐ 导出物能够追溯 Markdown digest；
- ☐ 不存在两篇同时定义相反语义的核心规范；
- ☐ 新增要求拥有 Requirement ID；
- ☐ UNKNOWN、NOT_APPLICABLE 和 Exception 未被混用；
- ☐ Deprecated/Superseded 文档有明确替代物；
- ☐ 每次发布有兼容性和迁移说明。

附录 B：结论

这套标准的质量不取决于文档数量，而取决于边界是否清楚、要求是否可执行、证据是否可信，以及项目能否从规范一路追溯到运行事实。文档体系本身也必须遵循系统工程原则：单一权威、显式状态、可逆变更、版本化契约、失效检测和可审计裁决。

优秀系统设计与工程保证方法论

从约束、不变量与认识边界，推导可运营、可演进、可验证的工程系统

文档属性	内容
文档编号	ESD-METHOD-001
版本	2.0.0-draft.1 (基于 1.1.0)
状态	DRAFT / NORMATIVE FOUNDATION；未获发布批准
文档角色	定义系统设计与工程保证的原则、术语、推导顺序和认识边界
配套文档	ESD-REFERENCE-001 《优秀系统设计：从约束、不变量到证据》
适用对象	架构师、Staff/Principal Engineer、技术负责人、平台与基础设施工程师
适用系统	分布式服务、数据平台、流处理、存储系统、控制面、高性能系统与关键业务系统
权威格式	本 Markdown 文件；任何 DOCX、PDF 或网页均只是非权威视图

文档关系与冲突规则

本文对通用设计原则、核心概念与认识边界拥有主题权威；不是覆盖所有领域的最高优先级文档。主题划分以 [ESD-SUITE-000 §3.1](#) 为准：治理、风险与 Gate 状态由 GOV 定义；Claim、Assumption 与 Evidence 状态由 ASSURANCE 定义；运行与恢复契约由 OPS 定义；REFERENCE 提供执行流程与示例。ADR 记录决策及其依据，不自行授予豁免权；偏离要求必须经过 GOV 规定的授权程序。

摘要

优秀的系统不是技术组件的集合，也不是一张在正常路径上看起来完整的架构图。它应当是在明确的业务、技术、组织、经济、安全与时间约束下，以尽可能低的必要复杂度持续维护关键性质；在失败、过载、升级、依赖退化和需求变化中保持可接受行为；并通过与具体制品、配置、环境和工件负载绑定的证据，说明这些性质在既定覆盖范围内确实成立。

这一定义包含三个彼此不可替代的层次：

1. **架构推导**：由问题、约束、工作负载、风险和不变量推导状态、边界、语义与机制；
2. **工程控制**：通过所有权、有界资源、准入、背压、恢复、版本化和变更控制维持系统稳定；
3. **工程保证**：把设计主张转换为已编码属性、验证机制、证据、门禁和运行反馈，同时诚实保留无法证明的 UNKNOWN。

本文反对两种对称的错误：一端是由时髦技术名词驱动的组件堆砌；另一端是以敏捷为名拒绝处理高代价、难逆转决策。正确方法不是“先设计一切”，而是：

对高后果、高不确定性和难逆转的决策提前建模；对低后果、可逆的实现细节延迟决定，并用原型、基准和运行证据收敛。

1. 文档定位

1.1 目标

本文建立一套可长期复用的系统工程认知基线，用于回答以下问题：

- 系统为什么存在，哪些结果才具有业务价值；
- 哪些性质在任何允许场景下都不能被破坏；
- 哪些状态是权威事实，由谁拥有、修改、恢复和运营；
- 网络、进程、依赖、配置和资源失败时系统如何收敛；
- 一致性、交付、幂等、恢复和降级语义如何从不变量推导；
- 吞吐、时延、容量和单位成本如何计算；
- 发布、回滚、重放、迁移、应急和事故学习如何进入架构；
- 关键主张如何被验证，证据能证明什么、不能证明什么；
- 当假设失效、环境漂移或证据过期时，系统如何降低确定性而不是伪造 PASS。

1.2 非目标

本文不是：

- 某种语言、数据库、消息系统、云平台或 Kubernetes 的选型目录；
- 对微服务、事件驱动、CQRS、Service Mesh 或 Event Sourcing 的默认推荐；
- 要求所有项目在编码前完成同等厚度的“大设计”；
- 以文档数量、门禁数量或测试覆盖率替代真实风险降低；
- 对未编码属性、未知环境或开放世界行为提供绝对正确性承诺。

1.3 规范性用语

- **必须 (MUST)**：缺失会直接威胁关键正确性、安全性、可恢复性或生产可控性；
- **应 (SHOULD)**：默认要求，偏离时必须记录理由、风险、Owner 和复审条件；
- **可以 (MAY)**：可根据风险、成本与团队能力选择；
- **不得 (MUST NOT)**：会形成不可接受风险或破坏本框架的认识边界；
- **未知 (UNKNOWN)**：当前证据不足，不能合理判定为通过或失败；
- **不适用 (NOT_APPLICABLE)**：经论证后该要求与当前系统或变更无关。

2. 严格定义：什么是优秀的系统

优秀的系统，是一个受约束的社会技术控制系统：它以尽可能低的必要复杂度，在明确的状态、身份、责任与资源边界内持续维护关键性质；能够在正常、降级、恢复和紧急模式之间安全转换；并通过有作用域、有来源、有时效的证据说明这些性质在当前制品、配置、环境、工作负载和故障模型下成立。

这个定义刻意包含以下限定：

- **受约束**：架构不是技术自由发挥，而是业务效用、风险、成本和工程能力共同约束下的选择；
- **社会技术系统**：代码、基础设施、数据、配置、权限、团队、流程和应急主体共同构成系统；
- **关键性质**：正确性必须表达为可检查的安全、活性、持久性、安全隔离和恢复要求；
- **运行模式**：系统不只有“正常/失败”，还存在受控降级、隔离、恢复、维护和紧急模式；
- **必要复杂度**：复杂度必须为明确价值或风险控制承担举证责任；
- **证据作用域**：任何验证结论都只对其覆盖的制品、配置、环境、输入分布和故障场景有效。

架构因此不是：

Kafka + Kubernetes + Redis + Rust + Service Mesh

也不是：

一张包含所有服务和箭头的拓扑图

架构更接近：

一组关于状态、身份、边界、失败、资源、控制、演进和证据的版本化决策，以及用于持续维护这些决策所声明性质的机制。

3. 认识论边界：工程保证能够证明什么

3.1 验证工具不是绝对仲裁者

编译器、类型系统、静态分析、模型检查、测试套件、策略引擎、CI 和运行时监控，只能裁决它们已经表达、编码和覆盖的属性。它们不能自动发现所有隐含业务假设，也不能从有限输入空间推出开放世界中的绝对正确性。

因此，高质量工程语言应避免：

测试通过，所以系统正确。

结论必须与验证方法匹配：

方法	可以支持的结论	不能自动推出的结论
有限测试、故障注入	在已执行的输入与轨迹中，断言未被违反	所有输入与调度均正确
模型检查	在明确模型、假设与已探索状态空间内性质成立；未完成探索必须声明边界	实现与开放环境自动符合模型
定理证明	在形式化定义、前提及证明系统内命题成立	前提在生产恒真或代码自动满足规格
运行监测	在信号覆盖、采样与检测能力内得到运行事实	未报警即不存在违反

Safety 通常可由有限坏前缀反驳；有界活性可由截止时间违反反驳；无界活性不能仅凭有限时间“尚未完成”否定，需公平性、依赖和资源假设以及进展论证；SLO 需统计窗口、样本与测量不确定性。可裁决不等于所有性质都能被有限运行观测直接证伪。具体义务见 [ASSURANCE §5.2](#)。

这不是削弱工程保证，而是避免贬低形式证明，也避免把模型内证明夸大为实现或开放世界的保证。

3.2 四层主张模型

每个重要设计结论应区分四个层次：

层次	含义	典型错误
Desired Claim	业务或架构希望成立的性质	把愿望直接写成事实
Encoded Property	已被契约、类型、策略、断言或测试形式化的性质	未编码却声称已覆盖
Verified Result	验证器在给定范围内得到的结果	忽略输入、环境与工具边界
Evidence-backed Decision	与当前制品和环境绑定、足以支持门禁的证据	证据过期、错绑或不可复现

只有完成从主张到证据的映射，系统才具备工程保证意义上的可裁决性。

3.3 UNKNOWN 是一等状态

以下情况不得被解释为 PASS：

- 没有运行验证；
- 验证工具不覆盖该性质；
- 证据无法关联当前制品或配置；
- 环境、依赖或工作负载已经超出验证范围；
- 证据过期、损坏或不可复现；
- 关键假设没有被验证或监测。

此时应返回：

UNKNOWN / INSUFFICIENT_EVIDENCE

成熟系统不是永远绿色，而是能够准确暴露当前不知道什么。

3.4 证据不是永久有效

任何下列变化都可能使既有证据失效：

- 二进制、依赖、编译器或构建参数变化；
- 配置、策略、Schema、DBC、模型或参考数据变化；
- 节点规格、内核、网络、存储后端或区域变化；
- 输入分布、租户偏斜、对象大小或突发模式变化；
- 外部依赖语义或 SLO 变化；
- 新事故暴露了未建模故障路径。

因此，证据必须具有主体身份、环境身份、覆盖范围、产生时间和失效规则。

4. 系统设计十条总纲

总纲一：约束先于架构

系统设计必须先明确业务、技术、组织、经济、安全、合规和时间约束，再讨论组件与部署形态。团队技能、既有运维能力和迁移窗口本身就是硬约束，不得在选型后才补写理由。

最低实践：建立版本化的 Constraint / Assumption Register。

总纲二：关键性质先于机制

需求描述“希望发生什么”，关键性质描述“无论发生什么都不能被破坏什么”。事务、消息队列、锁、租约、幂等键和副本只在维护明确性质时才有正当性。

最低实践：为每项高后果风险建立带作用域的 Critical Property Contract。

总纲三：状态权威与责任必须显式

每份持久状态必须有唯一权威来源；每个关键组件同时必须有技术 Owner、运营 Owner 和必要时的安全 Owner。共享可读不等于共享可写，复制不等于多权威。

最低实践：维护 State Ownership Register，明确写权限、恢复来源和冲突裁决。

总纲四：失败、过载与恢复是一等设计输入

网络超时、进程崩溃、重复、乱序、部分成功、磁盘写满、依赖变慢和配置错误不是异常边角，而是生产系统的正常状态空间。系统必须事先定义失败后的可观察行为、恢复权威、最大影响和人工介入点。

最低实践：维护 Failure / Recovery Matrix 与显式运行模式。

总纲五：所有可增长资源必须有界

线程、任务、队列、连接、缓存、inflight bytes、重试次数、重试时间、临时文件和待补偿工作都必须有上界。无界异步不是吞吐设计，而是把压力隐藏到 OOM 或级联故障发生。

最低实践：每个队列和缓冲区必须拥有容量单位、满载行为、背压方向、Owner 与观测指标。

总纲六：一致性应选择最弱但足够的语义

系统既不应默认强一致，也不应把最终一致当作分布式系统信仰。应从业务不变量反推需要的原子性、顺序、隔离、可见性和冲突语义。at-least-once + idempotent commit 只有在稳定操作身份和原子条件写明确时，才形成 exactly-once business effect。

最低实践：对每个外部副作用明确提交点、ACK 顺序、幂等身份和冲突结果。

总纲七：控制面、数据面、管理面与保证面分离

高吞吐数据路径不应被频繁变化的策略污染；策略不应与执行机制混为一体；管理操作不应绕过正常控制；验证规则和门禁本身也必须受保护和审计。

最低实践：明确四个平面的状态、权限、接口和失效模式。

总纲八：容量、成本和动态稳定性必须可计算

平均吞吐不足以证明系统稳定。设计必须覆盖峰值、恢复积压、依赖降容、重试放大、排队时延、内存上界和单位经济性，并评审自动扩缩容、重试、熔断和准入等控制回路之间的相互作用。

最低实践：建立 Capacity & Economic Model 和 Control Loop Register。

总纲九：不可逆决策提前设计，可逆决策延迟决定

数据身份、权威边界、外部契约、租户隔离、不可逆写入和安全边界属于高代价决策；内部模块、初始队列大小和局部库通常可以通过实验调整。MVP 应减少不可逆承诺，而不是取消架构纪律。

最低实践：ADR 必须记录假设、替代方案、迁移路径、回滚或 forward-recovery 路径及失效条件。

总纲十：没有证据的关键主张只能是未知

关键性质必须形成：

Claim -> Property -> Mechanism -> Verification -> Evidence -> Gate -> Runtime Feedback

门禁不得把缺失、过期或错绑证据解释为通过；运行漂移应自动降低相关结论的可信度；事故必须推动性质、机制、验证或控制发生变化。

最低实践：维护 Verification & Evidence Matrix，并保护 Assurance Plane 自身。

5. 完整的架构推导模型

推荐的推导顺序是：

```
Business Goal
-> Problem / Goals / Non-goals
-> Constraints / Assumptions / Risk
-> Workload / SLO / Recovery Objectives
-> Scoped Critical Properties / Precedence
-> State / Identity / Ownership
-> Boundary Contracts / Failure Model
-> Consistency / Delivery / Recovery Semantics
-> Architecture / Planes / Control Loops
-> Resource Bounds / Capacity / Economics
-> Contracts / Security / Evolution
-> Verification / Evidence / Gate
-> Runtime Drift / Incident Learning
-> Revised Assumptions / Properties / Architecture
```

这个顺序不是形式主义，而是依赖关系：

- 没有工作负载，就无法合理确定并发模型和容量；
- 没有不变量，就无法决定一致性和事务边界；
- 没有状态权威，就无法定义恢复、去重和冲突裁决；
- 没有失败模型，就无法证明重试和 ACK 顺序安全；
- 没有资源上界，就无法判断系统在持续过载下是否稳定；
- 没有证据绑定，就无法知道门禁结论是否适用于当前部署对象。

架构可以被抽象为受约束优化：

```
maximize:
  business utility

subject to:
  critical properties
  SLO and recovery objectives
  security and compliance boundaries
  cost ceiling
  engineering and operational capacity
  delivery window

minimize where possible:
  accidental complexity
  coordination points
  irreversible commitments
  recovery ambiguity
  epistemic debt
```

这里的“最优”不是单指标极致。吞吐更高但无法恢复、延迟更低但成本不可持续、功能更多但状态语义模糊，都不能称为优秀设计。

6. 从问题、约束与风险开始

6.1 问题定义

高质量问题定义至少包含：

```
Context
Business outcome
Actors and authority
Input / output
Current failure or opportunity
Success criteria
Non-goals
Consequences of failure
Decision deadline
```

“建设一个高性能数据平台”不是工程问题，因为它没有说明价值、负载和失败后果。更好的表述是：

```
对象存储中持续产生不超过 40 MiB 的 CAN 文件；
每个文件约含 3000 个独立 gzip member，元数据由 Kafka 投递；
系统需要按车型与不可变 DBC 版本进行解码，在有界资源内水平扩展；
重复投递、进程崩溃和依赖退化不得产生重复正式结果或错误 DBC 解码；
当前阶段优化目标是持续吞吐和可恢复性，而不是单文件最低尾延迟。
```

6.2 约束与假设的区别

- **约束**：设计必须遵守的现实边界，例如法规、预算、硬件、交付窗口、既有协议；
- **假设**：当前被认为成立、但可能失效的事实，例如对象大小分布、依赖延迟或团队支持能力。

关键假设不得停留在文档备注中。验证结论、生命周期、监测状态和风险标记是独立维度，不构成单一状态链。字段与迁移规则以 [ASSURANCE §6.2](#) 为唯一受控定义：

```
validation_status: UNVALIDATED | VALIDATED | VIOLATED | UNKNOWN
lifecycle_status: PROPOSED | ACTIVE | EXPIRED | RETIRED
monitoring_status: MONITORED | UNMONITORED | UNAVAILABLE | NOT_APPLICABLE
risk_flag: NONE | AT_RISK
```

例如已启用监测的假设仍可能尚未验证；证据过期不等于事实已被违反。替换假设使用新 ID 与 `supersedes` 关系，保留旧记录。

每项关键假设应包含：

字段	含义
Statement	精确、可证伪的陈述
Dependents	哪些性质、ADR、容量或安全结论依赖它
Evidence	当前支持证据与置信理由
Owner	负责监测、复审和失效响应的主体
Signal	运行时观测信号
Expiry	复审日期或失效条件
Response	假设失效后的限流、隔离、复审或迁移动作

无 Owner、无监测、无到期时间的关键假设形成**认识债务 (Epistemic Debt)**。

6.3 工作负载模型

工作负载必须同时描述：

- 到达率、字节率、对象率和业务事件率；
- 平均值、p95、p99、p99.9、硬上限和突发持续时间；
- 读写比例、数据偏斜、租户偏斜和热点；
- CPU、I/O、网络、内存、压缩、序列化和外部调用成本；
- 有序性、重复、乱序、迟到、回放和补算特征；
- 正常、峰值、恢复积压和依赖降容场景；
- 数据增长与未来容量窗口。

只写“峰值 QPS”通常不够。一个系统可能 QPS 不高，但单请求产生大量 fan-out、内存驻留或外部副作用。

6.4 SLO、恢复目标与正确性不得混写

必须区分：

类型	示例
Safety	不得使用未知 DBC 生成正式结果
Liveness	已接纳任务最终完成或进入显式终态
Durability	已提交结果在规定故障模型下不丢失
Availability	每月成功服务时间比例
Latency SLO	99% 文件在 10 分钟内完成
RPO	灾难后最多允许丢失多少已确认状态
RTO	在多长时间恢复到定义能力
Policy	当前优先处理某类车型

SLO 可以被预算和降级；硬安全不变量通常不能被“可用性目标”覆盖。

6.5 风险决定设计深度

设计深度应由三个维度共同决定：

$$\text{Design Depth} \sim \text{Consequence} \times \text{Uncertainty} \times \text{Irreversibility}$$

- 后果低、可逆、可隔离的实验可以快速迭代；
- 涉及权威数据、资金、安全边界或外部长期契约的决策必须更早、更深地建模；
- 不确定性高但可逆的部分优先用原型和观测缩小未知；
- 不确定性高且不可逆的部分应优先降低暴露半径或延迟承诺。

7. 关键性质：从愿望到带作用域的契约

7.1 Safety、Liveness 与其他性质

一个成熟的性质模型至少区分：

- **Safety**：坏事永远不会发生；
- **Liveness**：在明确前提下，好事最终会发生；
- **Durability**：已确认事实在规定故障模型下保持；
- **Isolation / Security**：未授权主体不能观察或修改受保护状态；
- **Recoverability**：系统可从权威状态收敛到一致状态；
- **SLO**：在统计窗口内满足的服务目标；
- **Assumption**：性质成立依赖的外部事实；
- **Policy**：可调整的业务或运营选择。

把所有内容都叫“不变量”会模糊语义。例如“任务最终完成”是活性要求，不是安全不变量；“p99 小于 100ms”是统计 SLO，不应伪装为永真命题。

7.2 属性契约

关键性质至少应表达为：

```
Property = <
  identity,
  scope,
  temporal semantics,
  assumptions,
  guarantee,
  allowed degradation,
  authority boundary,
  verification coverage,
  evidence freshness,
  owner
>
```

示例：

```
Identity:
  source_object_version + pipeline_semantic_version

Scope:
  Result Store 的正式 completion manifest

Temporal semantics:
  在任意可观察时刻最多存在一个 committed manifest

Assumptions:
  source object version 不变; Result Store conditional create 具有线性化语义

Guarantee:
  重复投递、重试或 worker crash 不会形成第二个正式结果

Allowed degradation:
  冲突执行可进入 reconciliation, 但不得覆盖既有正式结果

Verification coverage:
  duplicate delivery、commit/ACK crash、consumer rebalance、worker restart

Evidence freshness:
  仅对当前制品、配置、存储实现和测试故障档案有效
```

这能防止把局部性质夸大成端到端性质，也能暴露性质赖以成立的边界假设。

7.3 性质优先级与冲突

真实系统中的性质可能冲突：完整性要求暂停未知输入，可用性要求继续输出；安全要求 fail closed，应急恢复可能要求 break-glass。系统必须预先定义优先级和裁决机制。

一个常见但非普适的顺序是：

人身安全 / 法律硬约束

- > 权威数据完整性与不可逆业务正确性
- > 身份、授权与租户隔离
- > 可恢复性与证据完整性
- > 核心活性与可用性
- > 延迟和吞吐 SLO
- > 单位成本与局部性能

每个系统都应根据自身风险建立 Property Precedence Matrix。任何事故期间允许牺牲的性质必须提前说明，不能由现场人员临时猜测。

7.4 不变量必须映射到机制

“不重复”“高可用”“数据安全”都不是可执行设计。高质量表达应回答：

什么对象？
在什么作用域？
从什么时候到什么时候？
在什么假设下？
由哪个权威边界维护？
冲突时谁裁决？
如何验证？
违反后如何观测和恢复？

8. 状态、身份、边界与所有权

8.1 先问状态，再问服务

系统边界应从状态和责任推导，而不是先拆服务再分配数据库。对每份重要状态，必须回答：

- 它表示什么业务事实；
- 它的稳定身份是什么；
- 权威副本在哪里；
- 谁能创建、修改、删除和读取；
- 是否不可变、版本化或派生；
- 冲突如何裁决；
- 进程崩溃后从哪里恢复；
- 哪个团队承担 SLO、升级和事故责任。

状态可分为：

类型	说明	设计重点
Authoritative	决定业务事实的权威状态	唯一写入语义、审计、恢复
Durable Derived	可重建但长期保存的派生状态	来源版本、重建成本、漂移
Ephemeral	进程或任务生命周期内状态	丢失后的安全恢复
Evidence	用于证明和门禁的状态	不可静默修改、主体绑定、时效
Configuration / Policy	影响行为或语义的控制状态	版本、权限、发布与回滚

8.2 所有权是多维的

至少区分：

- **Semantic / Technical Owner**：定义状态语义、契约和兼容性；
- **Operational Owner**：承担 SLO、容量、发布、恢复和事故响应；
- **Security Owner**：定义访问、审计和高权限变更边界；
- **Mutation Authority**：真正有权改变权威状态的主体或流程。

“由某服务拥有”不够，因为服务名不能替代组织责任。

8.3 Assume–Guarantee 边界契约

组件局部正确不能自动推出端到端正确。关键边界应表达为：

```
Component C assumes A from its environment.  
If A holds, C guarantees G.  
If A is violated, C produces explicit bounded failure F.
```

例如：

```
Framer assumes:  
  object bytes 与声明 version 一致，且文件不超过支持上限。  
  
Framer guarantees:  
  每个 frame 对应完整 gzip member；损坏边界不会进入正式 decode。  
  
If assumptions fail:  
  返回 typed corruption / unsupported-size，隔离该对象，不猜测继续。
```

端到端主张只有在上游保证真实满足下游假设、所有外部假设均被治理、违反路径有界且可观测时才成立。

8.4 平面划分

成熟系统通常需要区分：

平面	责任
Data Plane	高频数据传输、计算和业务执行
Control Plane	策略、配置、调度、路由和状态收敛
Management Plane	部署、升级、诊断、恢复和人工操作
Assurance Plane	验证、证据、策略门禁、例外和运行漂移判定

分离不是为了增加服务，而是为了防止不同变化速度、权限和失败语义相互污染。

9. 失败、一致性、提交与恢复

9.1 失败是部分进展，而不只是错误码

分布式失败最危险之处在于参与者对“发生了什么”拥有不同视图：

输出已持久化，但 ACK 未提交；
 请求超时，但远端操作可能已经成功；
 配置发布到一半，部分实例已经切换；
 主库提交成功，派生索引尚未更新；
 worker 崩溃，本地状态丢失但外部副作用仍存在。

因此，失败模型必须描述：

- 失败发生在哪个边界和阶段；
- 哪些副作用可能已发生；
- 参与者能观察到什么；
- 哪份状态拥有最终裁决权；
- 重试是否安全；
- 是否需要 reconciliation、补偿或人工裁决；
- 最大积压、恢复时间和数据暴露是什么。

9.2 一致性是推导结果

推荐推导链：

Business Property
 -> Required visibility / ordering / atomicity / isolation
 -> Coordination mechanism
 -> Availability and latency cost
 -> Operational complexity

强一致可能是正确选择，也可能是昂贵的过度协调；最终一致可能足够，也可能直接破坏库存、余额、唯一性或授权边界。应选择**满足关键性质的最弱充分语义**。

9.3 Delivery 与 Effect 分离

必须区分：

- at-most-once delivery;
- at-least-once delivery;
- exactly-once delivery;
- exactly-once business effect。

工程上常用的可靠模式是：

```
at-least-once execution
+ stable operation identity
+ atomic conditional commit / deduplication
+ explicit conflict semantics
= exactly-once business effect within a defined authority boundary
```

这里最后一句的作用域不能省略。它通常只在某个权威存储或业务提交边界成立，而不是证明整个世界“只执行一次”。

9.4 ACK 顺序必须与业务完成语义一致

若消息 ACK 早于权威结果提交，崩溃可能造成任务丢失；若结果提交早于 ACK，重复投递必须由幂等提交吸收。设计必须明确：

```
accepted
started
side effect attempted
authoritative commit succeeded
acknowledged
visible to downstream
```

哪些状态是内部进展，哪个状态才构成业务完成。

9.5 恢复必须从权威状态开始

恢复不得依赖“某个进程应该还记得”。正确恢复通常以持久权威状态为基础，结合：

- create-only 或 conditional update;
- generation、version、fence 或 lease;
- immutable source + replay;
- reconciliation loop;
- compensation 或 forward repair;
- 明确的人工裁决入口。

9.6 显式运行模式

系统应把以下模式建模为一等状态：

```
NORMAL
DEGRADED
RECOVERY
QUARANTINE
MAINTENANCE
EMERGENCY
```

每种模式必须定义：进入条件、授权主体、硬性质、可降级目标、禁止行为、资源策略、最大持续时间、退出条件与证据。降级不是“所有质量都降低”，而是按预先定义的优先级牺牲可牺牲性质，同时保护硬不变量。

10. 有界资源、背压与动态稳定性

10.1 所有增长都必须有上界

必须显式限制：

```
max inflight requests / files / bytes
max queue depth or queued bytes
max worker tasks / threads
max decompression and decode jobs
max open connections / file descriptors
max retry attempts and retry time
max temporary disk
max reconciliation backlog
max cardinality retained in memory
```

上界应优先使用真实稀缺资源单位。例如对象大小差异很大时，inflight bytes 往往比“文件数量”更可靠。

10.2 Queue Contract

每个队列都必须回答：

问题	要求
为什么存在	解耦哪两个速率或调度域
容量单位	items、bytes、CPU cost 或其他
满载行为	阻塞、暂停、拒绝、降级、丢弃或溢写
背压方向	压力如何传播到真正可控的上游
公平性	是否按租户、优先级或成本隔离
终止语义	drain、shutdown、crash 后如何处理
观测	depth、age、wait time、drop、saturation
Owner	谁维护容量和故障策略

Queue 是容量，不是垃圾桶。无界队列只是把失败推迟成更难恢复的失败。

10.3 Deadline、timeout 与 retry budget

局部超时必须服从端到端 deadline。若每层独立重试，最坏请求量可能呈乘法放大。应共享：

```
request deadline
retry budget
attempt identity
backoff and jitter policy
maximum amplification ratio
```

只有瞬态且重试安全的失败才应自动重试；永久错误、权限错误、结构损坏和未知语义通常应快速失败或隔离。

10.4 控制回路稳定性

准入、重试、熔断、自动扩缩容、负载均衡、缓存和人工处置本质上是反馈控制回路。多个局部合理回路叠加后可能形成振荡：

```
依赖变慢
-> timeout 增加
-> retry 放大
-> queue 增长
-> autoscaler 延迟扩容
-> 冷启动争夺依赖
-> 依赖进一步变慢
```

评审至少应检查：

- 传感信号与观测延迟；
- 控制器目标、阈值和时间尺度；
- 执行器能力、动作上界和安全默认；
- hysteresis、cooldown、rate limit；
- 多个控制器是否操纵同一变量；

- 控制器不可用或误判时的失效方式；
- 是否通过仿真、故障注入或历史回放验证稳定性。

10.5 Little's Law 的最低限度应用

稳定系统中：

$$L = \lambda \times W$$

其中 L 是系统内平均在途数量， λ 是完成率， W 是平均停留时间。它提醒设计者：延迟上升会直接增加在途量和内存压力；队列变长不是免费吸收峰值，而是在积累未来必须处理的工作。

11. 容量、单位经济性与复杂度预算

11.1 容量模型必须能够粗算

最小容量模型至少包括：

```
arrival rate
item / byte size distribution
work per item
service time per stage
parallelism and utilization target
memory residency
network and storage bandwidth
recovery backlog and catch-up window
headroom
```

例如：

```
100 files/s x 3000 blocks/file = 300,000 blocks/s
300,000 x 50 microseconds CPU/block = 15 CPU-seconds/second
```

这只给出理论 CPU 下界；实际还必须加入解压、路由、序列化、缓存未命中、调度、抖动和恢复余量。

11.2 Pipeline 吞吐由瓶颈决定

以下速率全部已折算为源输入字节等价吞吐，不是各阶段自身输入或输出字节率：

```
fetch:      10 GB/s
decompress:  4 GB/s
decode:      1 GB/s
output:      8 GB/s
```

理论瓶颈上限为 1 GB/s 源输入等价吞吐；只有阶段独立、负载稳定且调度开销可忽略时，端到端吞吐才可能接近该值。一般按 $\lambda_{\max} \leq \min_i(C_i / d_i)$ 计算： C_i 是资源容量， d_i 是每单位源输入消耗的同单位资源；共享资源、解压膨胀、输出放大与竞争必须另计。不能直接比较压缩字节、解压字节和输出字节的原始 GB/s。继续优化非瓶颈下载不能突破当前上限。性能工程必须区分：

- service time 与 queue time;
- throughput 与 utilization;
- steady state 与 recovery state;
- average 与 tail;
- 单阶段微基准与端到端约束。

11.3 Headroom 是恢复能力的一部分

容量只够平均负载意味着系统无法应对突发、节点故障、升级降容和积压恢复。Headroom 应根据故障域、扩容延迟、恢复窗口和成本目标明确，而不是习惯性保留固定百分比。

11.4 单位经济性

成熟模型应计算：

```
$/request
$/GB processed
$/1M events
$/tenant
$/vehicle-hour
cost of recovery
cost of retained evidence
```

当精确成本数据不可得时，不应放弃，而应标记置信等级：

等级	含义
C0	无成本模型，只能标记 UNKNOWN
C1	依据公开价格或资源配额的粗估
C2	依据账单、标签或节点分摊的工作负载估算
C3	依据实际利用率、共享成本和业务单位的校准模型

平台工程应逐步把 C1/C2 升级为可操作的成本遥测，但不能因为尚未达到 C3 就伪造精确数字。

11.5 复杂度预算

系统复杂度的风险来自可达状态空间、协调点和认知负担，而不只是代码行数。建议跟踪：

- authoritative stores 数量；
- 跨权威边界写入与双写路径；
- lock、lease、fence 和一致性协调点；
- 异步 channel、retry loop 和补偿路径；
- 同时在线的 Schema、协议与配置版本；
- 特权主体、break-glass 和人工恢复步骤；
- 运行模式、quarantine 类型和长期例外；

- 无 Owner、无删除路径或无失效条件的组件。

新增持久状态和分布式边界必须说明必要价值、失败方式、运营责任和未来删除条件。优秀架构不仅会增加机制，也会持续回收复杂度。

12. 契约、安全、演进与可逆性

12.1 契约优先于实现

组件边界必须显式定义：

- identity 与 idempotency key；
- schema、version 与兼容规则；
- ordering、duplication、timeout 和 retry 语义；
- unknown、malformed、not-found 和 conflict 行为；
- authority、ownership 和 mutation rights；
- observability、audit 与 retention。

契约中的错误和未知路径与成功路径同等重要。

12.2 影响语义的内容必须版本化

包括但不限于：

```
schema
protocol
configuration
policy
algorithm
DBC / model / reference data
feature flags with semantic impact
build and dependency graph
```

业务结果应能够追溯到源数据身份、处理语义版本和关键配置版本。

12.3 Replayability

理想的数据处理系统具有：

```
immutable raw input
+ versioned processing logic
+ versioned configuration / reference data
+ deterministic or explainable commit semantics
-> reproducible output
```

重放能力支持审计、修复、迁移、版本比较和事故分析，但其成本、访问权限和结果替换语义必须明确。

12.4 安全从信任边界开始

必须识别：

- 谁能创建任务、修改策略、读取数据和提交结果；
- 哪些输入不可信；
- 哪些控制面操作拥有高爆炸半径；
- 哪个身份代表人、服务、流水线或恢复主体；
- 哪些权限只能通过时限、范围受限的 **break-glass** 获得；
- 谁能够修改验证规则、**required checks** 和豁免逻辑。

安全机制应遵循最小权限、显式身份、默认拒绝、不可静默修改和可审计变更。

12.5 决策可逆性

ADR 除了 Decision 与 Rationale，还必须记录：

```
assumptions
alternatives
consequences
risks
reversibility
migration / rollback / forward-recovery path
invalidation conditions
review triggers
```

对于数据结构和不可逆写入，回滚往往不是切回旧二进制。应优先使用 **expand-migrate-contract**、双读、影子验证、**versioned write**、**reconciliation** 和逐步收缩。

13. 工程保证：从设计主张到持续证据

13.1 保证链

```
Business Requirement
-> Critical Property
  -> Assumption / Boundary Contract
    -> Enforcement Mechanism
      -> Verification
        -> Evidence
          -> Artifact / Config / Environment Identity
            -> Gate Decision
              -> Runtime Observation
                -> Revised Model
```

每个箭头都必须可追踪。断裂的链条意味着只能保持 **UNKNOWN**。

13.2 验证层级

层级	主要能力	不能替代什么
Type / compile checks	表示、接口和局部静态约束	业务语义与运行环境
Static analysis / policy	已编码规则和禁止模式	未编码假设
Unit / property tests	局部行为和广泛输入性质	真实边界组合
Contract / integration tests	边界协议与依赖交互	生产规模和动态稳定性
Load / benchmark	容量、尾延迟、资源上界	故障恢复与业务正确性
Fault injection / chaos	失败路径和控制回路	未注入的未知故障
Formal methods	对精确模型的强推理	模型与现实的一致性
Runtime monitoring	真实运行分布与漂移	未观测、未编码属性
Review / audit	隐含假设、社会技术风险	可重复机器验证

高风险性质通常需要多层互补证据，而不是单一工具。

13.3 Evidence Envelope

关键证据至少应绑定：

```
Evidence {
  evidence_id
  claim_id
  subject_digest
  configuration_digest
  environment_identity
  workload_corpus_digest
  fault_profile
  verifier_identity
  produced_at
  validity / invalidation rule
  coverage
  limitations
  result
  provenance
}
```

以下仅为 ASSURANCE §12.3 的阅读摘要，证据生命周期的受控定义以该处为准：

```
CURRENT
STALE
INVALIDATED
SUPERSEDED
NOT_REPRODUCIBLE
```

环境漂移、关键依赖升级或负载越界应触发结论重评；支持不足时为 UNKNOWN，已有适用反证时保留 CONTRADICTED。不得通过失效处理清除不利事实。

13.4 组合保证

服务 A 和 B 各自测试通过，不能自动推出 $A \rightarrow B$ 端到端性质成立。必须检查：

- Guarantee(A) 是否满足 Assumption(B)；
- 超时、乱序、重复和错误是否也在契约内；
- 容量假设在故障降容时是否仍成立；
- 版本滚动期间的混合组合是否被覆盖；
- 局部证据是否与同一制品、配置和环境组合相匹配。

13.5 Assurance Plane 元不变量

保证面自身也可能失效或被绕过。高风险系统至少应维护：

- MA-01 缺失、过期或错绑证据不得解释为 PASS。
- MA-02 验证规则、门禁策略和豁免逻辑的变更必须经过独立门禁。
- MA-03 高风险变更的 proposer、evaluator 与 final approver 不得由同一非受控主体兼任。
- MA-04 例外和 break-glass 必须有范围、到期时间、审计和补偿复审。
- MA-05 已用于裁决的证据不得被静默修改。
- MA-06 制品、配置、部署目标与运行证据必须通过不可混淆身份关联。
- MA-07 保证面不可用时，高风险变更不得默认放行。
- MA-08 运行漂移超出已验证范围时，相关结论必须自动降级。

13.6 变更是受控实验

重要变更应定义 Change Envelope：

```
semantic delta
assumptions
maximum exposure
blast-radius boundary
observation window
success criteria
abort conditions
rollback / forward-recovery path
owner and authority
evidence to retain
```

Canary 不是“先放少量流量看看”，而是预先定义成功、停止和扩大条件的实验。

13.7 事故必须推动保证体系增长

每次重大事故或近失事件至少应改变以下一项：

Critical Property
Assumption / Boundary Contract
Failure Model
Mechanism
Verification Scenario
Runtime Evidence / Alert
Runbook / Authority
Capacity / Economic Model
Architecture Decision

如果复盘只增加提醒和文字，而没有改变系统属性、控制、验证或权限结构，系统对同类事故的抵抗能力通常没有实质增长。

13.8 防止 Goodhart 化

测试通过率、覆盖率和门禁数量一旦成为目标，团队可能优化指标而非风险。应：

- 从关键性质和故障场景出发选择验证；
- 记录不能证明的内容和覆盖空白；
- 使用负向测试、mutation testing 和未知输入挑战；
- 用真实事故、生产分布和近失事件校准测试集；
- 删除与明确风险无关的低价值门禁；
- 对关键性质保留独立挑战和人工审查。

14. 组织、成熟度与渐进式采用

14.1 Conway 定律不是借口，而是约束

技术边界若没有相应责任边界，最终会在事故和演进中失效。系统设计应同时优化：

state authority
bounded context
team ownership
on-call and recovery path
change coordination cost

平台团队不应通过中央审批拥有所有变更，而应提供有约束的自助能力、默认安全路径和可审计例外。

14.2 渐进式采用

不同风险等级需要不同厚度：

类型	最低要求
探索型原型	Non-goals、数据安全边界、不可逆决策、退出条件
有限业务系统	关键性质、状态权威、失败路径、资源上界、基础证据
核心生产系统	完整门禁、容量与恢复模型、运行模式、证据绑定
高风险/合规系统	独立审批、组合保证、保证面元不变量、例外治理与持续漂移检测

“渐进”意味着保证深度与风险匹配，不意味着关键数据身份和安全边界可以缺失。

14.3 成熟度模型

等级	特征
L0 — Component-driven	以技术栈和拓扑为中心
L1 — Functional	正常路径可用，但失败、状态和责任模糊
L2 — Bounded	资源有界，具备基本失败与恢复语义
L3 — Operable	SLO、Owner、发布、回滚、容量和可诊断性明确
L4 — Evidence-driven	关键主张映射到验证、证据和门禁
L5 — Adaptive Assurance	假设、证据和环境可关联；漂移自动失效结论；事故驱动保证增量

成熟度不能通过采购更多基础设施跳级。缺少状态语义和所有权时，引入更多组件通常只会扩大状态空间。

15. 最终判定标准

一个系统设计只有在能够回答以下问题时，才具备进入生产评审的基础：

1. 系统解决什么问题，明确不解决什么？
2. 真实工作负载、风险、约束和关键假设是什么？
3. 哪些安全性质、活性要求和恢复目标必须成立？
4. 每份权威状态和稳定身份由谁拥有、修改与恢复？
5. 边界双方的假设、保证和违反行为是什么？
6. 重复、超时、乱序、部分成功和资源耗尽时如何收敛？
7. 一致性、ACK、幂等和冲突语义如何维护业务性质？
8. 所有可增长资源是否有界，压力能否端到端传播？
9. 峰值、故障降容、积压恢复和单位成本是否经过计算？
10. 正常、降级、恢复和紧急模式分别允许什么？
11. 系统如何升级、迁移、回滚、重放、排空和灾难恢复？
12. 每项关键主张的机制、验证、证据、覆盖和失效条件是什么？
13. 保证面如何防止被绕过、弱化或错误地默认放行？

14. 假设或环境失效后，谁按 ASSURANCE 重评结论、保留反证与未知并触发复审？
15. 系统新增的复杂度何时、由谁、在什么条件下回收？

结论

高级系统工程并不是熟悉更多组件，而是能够维持一条完整、诚实且可复审的推理链：为什么存在，什么不能错，什么可能错，谁拥有事实，需要什么语义，资源和成本是多少，如何恢复与演进，以及我们凭什么相信这些结论仍然适用于当前系统。

附录 A：核心术语边界

术语	严格含义
Critical Property	对业务、正确性、安全、活性、资源或运营具有关键影响，并具有与性质类型匹配的验证义务的上位概念
Invariant	在规定身份、作用域和允许状态转换内始终成立的 Safety 性质，不应泛指所有目标
Safety	“坏事永远不发生”的性质，例如不重复正式提交、不越权写入
Liveness	在明确假设下，“好事最终发生或进入明确终态”的性质
Durability	在指定故障集合内，已确认事实保持可恢复的性质
SLO	在统计窗口和作用域内对服务结果的目标，不等同于正确性硬约束
Assumption	某项性质、容量或组件保证赖以成立、可被证伪并可能到期的外部条件
Guarantee	在假设成立时，组件、边界或系统承诺提供的行为
Authority Boundary	对某项业务事实拥有最终裁决权的状态或组件边界
Engineering Assurance	从主张、已编码性质、机制、验证、证据、门禁到运行反馈的持续控制过程
Evidence Envelope	证据所绑定的制品、配置、环境、工作负载、故障模型、覆盖、限制与有效期
Epistemic Debt	关键假设未验证、无 Owner、无监测或无失效响应所形成的认识风险
Change Envelope	对变更语义、最大暴露、观察窗口、成功/停止条件和恢复路径的预先约束
Assurance Delta	事故或运行事实对性质、假设、机制、验证、证据或治理产生的可审计增量

附录 B：参考资料

以下资料提供分布式系统、可靠性、软件复杂度、安全工程、控制与验证方面的理论背景；它们不替代具体项目的约束分析和实证证据：

1. Martin Kleppmann, *Designing Data-Intensive Applications*.
2. Michael T. Nygard, *Release It!*.
3. Betsy Beyer et al., *Site Reliability Engineering*.
4. Eric Evans, *Domain-Driven Design*.
5. John Ousterhout, *A Philosophy of Software Design*.

6. Nancy Leveson, *Engineering a Safer World*.
7. Leslie Lamport, “Time, Clocks, and the Ordering of Events in a Distributed System”.
8. Leslie Lamport, *Specifying Systems*, 尤其第 8 章 Liveness and Fairness、第 9 章 Real Time 与第 14 章模型检查。
9. Pat Helland, “Life Beyond Distributed Transactions: An Apostate's Opinion”.
10. Gerard J. Holzmann, *The SPIN Model Checker*.
11. John D. C. Little, “A Proof for the Queuing Formula: $L = \lambda W$ ”.
12. RFC 2119 and RFC 8174, requirements language for normative specifications.

附录 C：历史 v1.1.0 关键校准

以下记录保留 v1.1.0 相对早期 v1.0 稿件的历史变化，不代表当前草案已获发布批准：

1. 用 Critical Property 区分并统摄 Safety、Liveness、Durability、Resource Bound 与 SLO，不再把所有目标泛称为 invariant；
2. 把关键性质升级为带 identity、scope、temporal semantics、assumptions、allowed degradation 和 evidence freshness 的契约；
3. 增加 Property Precedence，提前裁决完整性、可用性、安全和恢复之间的冲突；
4. 增加 Assume–Guarantee 边界契约，防止局部通过被夸大为端到端保证；
5. 把假设、证据时效、运行漂移和 UNKNOWN 纳入正式治理；
6. 把显式运行模式、控制回路稳定性、单位经济性和复杂度预算纳入架构；
7. 为 Assurance Plane 增加元不变量、职责分离和信任链；
8. 把 canary、重大发布和迁移定义为受控实验；
9. 要求重大事故产生可验证的 Assurance Delta；
10. 将方法论与操作手册拆分，避免认知基线和项目模板形成两份互相漂移的重复文档。

优秀系统设计：从约束、不变量到证据

Architecture Review、工程落地与生产准入操作手册

文档属性	内容
文档编号	ESD-REFERENCE-001
版本	2.0.0-draft.1（基于 1.1.0）
状态	DRAFT / OPERATIONAL REFERENCE；未获发布批准
规范来源	METHOD、GOV、ASSURANCE、OPS；主题权威由 SUITE §3.1 控制
核心用途	系统设计、Architecture Review、ADR、容量规划、生产准入、事故后保证增量
适用对象	设计作者、Reviewer、技术负责人、平台/安全/SRE、发布裁决者
权威格式	本 Markdown 文件；其他导出不是独立事实来源

规范来源：ESD-METHOD-001 《优秀系统设计与工程保证方法论》

使用规则

本文将 METHOD、GOV、ASSURANCE、OPS 转换为工程阶段、产出物、门禁、模板与案例，主题冲突按 SUITE §3.1 裁决，不建立全局文档优先级。默认流程的裁剪按 GOV 授权并留痕；ADR 只记录决策，不授予豁免权。案例与填充示例不构成规范新增要求或已执行证据。

快速开始

对一个新系统或重大变更，最小工作流是：

0. 定义问题、目标、非目标和风险等级
1. 建立工作负载、SLO、约束与假设
2. 登记关键性质及其优先级
3. 明确状态、身份、所有权和边界契约
4. 建模失败、一致性、提交与恢复
5. 推导架构、平面、流水线和运行模式
6. 计算资源上界、容量、成本与控制回路
7. 设计契约、安全、演进、迁移与操作
8. 建立验证、证据和变更暴露计划
9. 通过生产准入，并持续监测假设与证据漂移

任何阶段出现以下情况，都不得以“后续补充”为由自动通过：

- 关键状态没有权威 Owner；
- 高后果性质没有精确作用域；
- 重复、部分成功或 ACK 顺序不明确；
- 关键资源无上界；
- 生产完成语义和恢复权威不明确；

- 关键证据无法绑定当前制品、配置或环境；
- 关键假设无 Owner、无监测或已失效；
- 高风险例外没有范围和到期时间。

1. 操作模型

1.1 角色

角色	责任
Design Author	维护问题、假设、性质、方案、风险与证据链
Semantic Owner	定义业务语义、状态身份、契约与兼容规则
Operational Owner	负责 SLO、容量、发布、恢复、on-call 与事故响应
Security / Compliance Reviewer	审查信任边界、权限、审计和硬约束
Platform / SRE Reviewer	审查资源、故障、可观测性、发布和恢复
Assurance Reviewer	检查主张、验证、证据覆盖和认识边界
Decision Authority	根据风险、证据和例外做最终门禁裁决

高风险变更中，提出方案、评价证据和最终批准不得由同一非受控主体兼任。

1.2 门禁状态

下表为 GOV §8.3 的阅读摘要；受控状态及裁决权限以该处为准，不维护独立 Gate 状态机。

状态	含义
GO	强制条件满足，证据与当前对象绑定，剩余风险在阈值内
CONDITIONAL_GO	可在明确范围、期限、Owner、监测和撤销条件下放行
NO_GO	已知违反关键要求，或风险不可接受
UNKNOWN	证据不足、已过期、无法复现、作用域不清或关键假设未验证
NOT_APPLICABLE	经论证与记录后确实不适用

UNKNOWN 不是较弱的 GO。对高风险性质，UNKNOWN 默认阻断生产放行。

1.3 评审原则

评审顺序必须从语义到机制：

Problem -> Property -> State -> Failure -> Resource -> Architecture -> Technology

不应从“为什么不用某组件”开始，也不应让架构图的视觉完整性掩盖状态和失败语义缺失。

2. 风险分级与裁剪

2.1 风险等级

下表为 **GOV §4** 的使用摘要。R0–R3、Gate 0–9 和 §19 成熟度模型是本套系的治理选择，不是外部标准的统一分级。

等级	典型场景	后果	默认保证深度
R0 — Exploration	一次性实验、无真实敏感数据、可随时删除	局部、可逆	最小 Problem Brief、数据边界、退出条件
R1 — Bounded Production	有限用户或内部系统，影响可隔离	中等、可恢复	关键性质、Owner、失败语义、资源上界、基础证据
R2 — Core Production	核心数据、收入、跨团队依赖、持续运行	高、恢复昂贵	完整阶段与门禁、容量/恢复模型、运行模式、证据绑定
R3 — High Assurance	资金、安全、合规、人身风险或大爆炸半径	极高或不可逆	独立审批、组合保证、元不变量、例外治理、持续漂移检测

风险等级必须根据**后果、暴露半径、不确定性和不可逆性**确定，不能只根据项目规模或 QPS。

2.2 裁剪规则

允许裁剪执行材料，但不得裁剪认识边界。任何裁剪必须记录：

```
omitted requirement
reason
risk level
compensating control
owner
expiry / review trigger
```

以下内容即使在 MVP 中也通常不可省略：

- 核心数据身份与权威来源；
- 不可逆外部副作用；
- 认证、授权和租户隔离；
- 外部长期契约；
- 资源硬上限与删除/退出路径；
- 无法回滚的数据或 Schema 变化。

2.3 采用剖面

Exploration Profile

- Problem / Non-goals
- Data and trust boundary
- Irreversible decisions
- Hard resource cap
- Exit / deletion criteria

Production-Lite Profile

在 Exploration 基础上增加：

- Critical Property Register
- State Ownership
- Failure / Recovery Matrix
- Basic capacity estimate
- Operational Owner
- Smoke / contract / fault evidence

Core Production Profile

增加：

- Boundary Contracts
- Operating Modes
- Capacity and recovery model
- Change Envelope
- Evidence binding and freshness
- Production Readiness Gate

High-Assurance Profile

增加：

- Property precedence
- Independent reviewer / approver
- Assurance meta-invariants
- Exception and break-glass governance
- Evidence drift automation
- Assurance delta after incidents

3. Stage 0: Problem Brief 与风险定级

3.1 输入

- 业务背景、当前问题或机会；
- 主要参与者和决策主体；
- 当前系统事实与已知事故；
- 交付窗口和初始预算。

3.2 必须产出

Problem statement
Goals
Non-goals
Success metrics
Failure consequences
Risk tier
Decision deadline
Initial unknowns

3.3 关键问题

1. 解决的是业务问题、运营问题还是技术便利？
2. 不做什么，防止哪些范围膨胀？
3. 失败会造成延迟、数据错误、资金损失、安全问题还是不可恢复损坏？
4. 什么结果可以客观证明项目成功？
5. 这是系统新建、迁移、扩容、可靠性修复还是成本优化？

3.4 Gate 0: Problem Validity

GO 条件：

- 问题、价值和非目标清楚；
- 成功标准可检验；
- 风险等级有依据；
- 不以预选技术作为问题定义。

典型 NO_GO：

- “因为要上 Kubernetes/微服务/AI”；
- 问题无法与任何业务或运营后果关联；
- 不知道失败影响却直接承诺架构。

4. Stage 1：Workload、SLO、约束与假设

4.1 工作负载最小字段

维度	内容
Arrival	平均、峰值、突发时长、增长率
Size	item/byte 分布、p99、硬上限
Work	CPU、I/O、网络、压缩、fan-out、外部调用
Shape	热点、租户偏斜、读写比、顺序、重复、迟到
Modes	正常、峰值、恢复、重放、依赖降容
Retention	保留期、重建时间、存储增长

4.2 SLO 与恢复目标

必须分别记录：

availability
latency / completion time
freshness / lag
correctness or error budget where meaningful
RPO
RTO
recovery catch-up objective
capacity headroom target

正确性硬约束不得藏在 error budget 中。

4.3 约束与假设登记

约束至少分为：业务、技术、组织、经济、安全/合规和时间。假设必须可证伪，并关联 Owner、证据、监测信号、失效阈值和响应。

4.4 Gate 1：Inputs Are Decision-Ready

GO 条件：

- 已覆盖真实峰值和恢复负载；
- 工作负载单位明确；
- SLO 与硬正确性分离；
- 团队技能、预算和运维约束已进入模型；
- 关键假设可监测且有复审条件。

UNKNOWN 条件：

- 只有平均值，没有尾部和硬上限；
- 无法估计恢复积压；
- 成本数据缺失且连粗估范围都没有；
- 关键外部依赖语义未确认。

5. Stage 2: Critical Properties 与优先级

5.1 产出物

- Critical Property Register；
- Property Precedence Matrix；
- Safety、Liveness、Durability、SLO、Assumption 和 Policy 分类；
- 每项性质的违反后果和验证义务。

5.2 编写规则

弱表述：

系统应可靠、幂等、高可用。

强表述：

对相同 `source URI + immutable source version/checksum` 与 `pipeline_semantic_version`, `Result Store` 在任意可观察时刻最多存在一个 `committed completion manifest`；
冲突执行只能返回 `ALREADY_COMMITTED` 或进入 `reconciliation`，不能覆盖正式结果。

每项关键性质必须包含：身份、作用域、时间语义、假设、保证、允许降级、权威边界、Owner、验证覆盖和证据失效规则。Safety、有界/无界活性与 SLO 的验证义务以 ASSURANCE §5.2 为准，不要求无界活性由有限运行前缀直接证伪。

5.3 Gate 2: Semantic Integrity

GO 条件：

- 高后果风险均有对应关键性质；
- 性质可判定，不是愿望；
- 性质冲突已有优先级；
- UNKNOWN 和 allowed degradation 被明确；
- 每项性质拥有 Owner 和验证义务。

典型 NO_GO：

- “exactly once” 没有定义作用域；
- 可用性在事故现场可以任意覆盖数据完整性；

- 对未知输入默认继续执行。

6. Stage 3: State、Identity、Ownership 与 Boundary Contract

6.1 State Inventory

对每份状态记录：

```
state name
business meaning
stable identity
authoritative / derived / ephemeral / evidence / configuration
authoritative store
mutation authority
technical owner
operational owner
security owner
retention
recovery source
conflict semantics
```

6.2 Boundary Contract

对关键边界记录：

```
Assumptions
Guarantees
Violation behavior
Identity and version
Timeout / retry / ordering / duplicate semantics
Capacity assumptions
Verification
Owner
```

6.3 Gate 3: Authority and Composition

GO 条件：

- 每份权威状态唯一；
- 复制、缓存和派生状态不会伪装成多权威；
- 技术与运营 Owner 明确；
- 上游 Guarantee 能满足下游 Assumption；
- 违反边界契约时行为显式、有界、可观测；
- 跨团队边界与 Bounded Context 基本对齐。

典型 NO_GO：

- 多个服务直接修改同一业务表但无人裁决；
- 依赖本地内存恢复关键事实；
- 服务 API 只定义成功响应，不定义重复、冲突和超时。

7. Stage 4: Failure、Consistency、Commit 与 Recovery

7.1 Failure Matrix

至少覆盖：

- 请求或消息重复；
- 乱序、迟到和部分可见；
- 网络 timeout 与请求结果未知；
- worker crash、进程重启和 consumer rebalance；
- 写入成功但 ACK 失败；
- ACK 成功但业务提交失败；
- 依赖变慢、不可用或返回陈旧数据；
- 磁盘写满、内存耗尽、连接池耗尽；
- 配置错误、版本不兼容和权限变更；
- 区域或控制面故障；
- 恢复期间新流量与积压竞争。

每项失败记录：检测、可能已发生的副作用、权威裁决、自动动作、人工动作、重试安全性、最大影响、恢复目标和证据。

7.2 Consistency Decision

对每个关键状态明确：

```
visibility requirement
ordering requirement
atomicity boundary
isolation requirement
conflict behavior
staleness tolerance
coordination mechanism
availability / latency trade-off
```

7.3 Commit Protocol

必须区分业务状态、执行尝试和交付位置：

```

business:
  DISCOVERED -> ACCEPTED -> RUNNING -> OUTPUT_STAGED
    -> AUTHORITATIVELY_COMMITTED
  alternative dispositions:
    QUARANTINED / PERMANENTLY_REJECTED

uncertain operation:
  OUTPUT_STAGED -> RECONCILIATION_REQUIRED
    -> authority resolution -> committed / safe final disposition / still unresolved

delivery:
  durable disposition -> delivery-specific acknowledgement eligibility
    -> confirmed delivery position update

```

RECONCILIATION_REQUIRED 是未决责任，不是业务终态；一次 `attempt` 返回不等于任务完成。QUARANTINED/PERMANENTLY_REJECTED 只有在业务合同允许、处置已持久化且无悬而未决副作用时，才构成该处理操作的终止处置，不等于成功。隔离后的修复/重放仍须有具名责任。

条件创建或其他外部副作用至少区分四种调用结论：

结论	含义与后续责任
CONFIRMED_SUCCESS	权威系统确认请求已生效，继续按业务协议推进
CONFIRMED_NO_EFFECT	有合同级证据确认本次请求没有且不会迟到生效，可按同一意图安全重试或终止
CONFIRMED_CONFLICT	权威条件检查拒绝创建；读取并核对既有事实，不能仅凭冲突码判定等价
UNKNOWN	超时、连接中断或不可解释返回；由 <code>reconciliation</code> 持续承担责任

超时可能发生在远端成功之后；即使权威读取暂时 `not-found`，原请求仍可能在途并稍后成功。不得由一次空读推出“确定未发生”，也不得更换幂等键重新提交同一意图。必须写明后端条件写入、读取一致性、幂等键作用域/保留期与迟到请求语义；具体后端不满足前提时，本协议不能直接采用。

责任移交至少记录 `OperationIdentity`、原请求/候选摘要、未知窗口、暂存数据位置、具名接收 Owner、持久化接收确认、重试预算、截止时间、告警升级与重启恢复规则。在接收确认之前原 Owner 不得释放责任；截止时间到达只触发升级，不把未知变成失败。移交可结束当前 `attempt`，但是否允许推进消息位置须由交付合同另行定义。CAN 案例采用 §16.11 的保守策略：未决状态不推进对应 Kafka 位置。

7.4 Gate 4: Failure Semantics Are Closed

GO 条件：

- 部分成功和未知结果有明确处理；
- ACK 顺序与业务完成语义一致；
- 重试有稳定身份和预算；
- 幂等由原子机制构造，不依赖“通常不会重复”；
- 恢复从权威状态出发；

- 降级、隔离和恢复模式已定义。

典型 NO_GO:

- 依赖消息中间件标签宣称端到端 **exactly-once**;
- 超时后一律重试不可逆操作;
- 回滚只描述切换旧镜像, 忽略数据已经变化。

8. Stage 5: Architecture、Planes、Pipeline 与 Modes

8.1 架构图最低要求

架构图必须能够表达:

- 权威状态与派生状态;
- 数据流、控制流、管理流和证据流;
- **trust boundary** 与 **mutation authority**;
- **sync / async** 边界;
- **commit**、**ACK** 和可见性边界;
- 背压方向;
- **failure domain** 与 **isolation boundary**;
- 正常、降级和恢复路径。

只画服务方框和箭头不构成完整架构。

8.2 平面划分

Data Plane:

高频传输、计算、解码、查询或业务执行

Control Plane:

策略、调度、路由、配置和收敛

Management Plane:

发布、升级、诊断、恢复、人工操作

Assurance Plane:

验证、证据、门禁、例外、漂移判定

平面可以部署在同一进程中, 但责任、权限和失败语义必须区分。

8.3 Fast Path / Slow Path

Fast Path 应只处理已验证、常见、可缓存的正常输入。损坏、未知版本、冲突、人工裁决和长期补偿应进入显式 Slow Path 或 Quarantine，而不是把热路径变成无限分支。

8.4 Operating Mode Matrix

至少定义：

表 7 · 字段记录视图（列名与原单元格逐项对应）

表 7 · 记录 1 · NORMAL	
Mode	NORMAL
Entry	关键依赖健康
Hard Properties	全部硬性质
Allowed Degradation	无
Forbidden	猜测、旁路提交
Exit	—

表 7 · 记录 2 · DEGRADED	
Mode	DEGRADED
Entry	依赖或容量下降
Hard Properties	完整性、安全、资源上界
Allowed Degradation	延迟、完成率
Forbidden	将未知结果标正式
Exit	依赖恢复并校验积压

表 7 · 记录 3 · RECOVERY	
Mode	RECOVERY
Entry	故障后重新收敛
Hard Properties	权威状态、可追踪重放
Allowed Degradation	暂停新接纳
Forbidden	用临时状态强行完成
Exit	reconciliation 完成

表 7 · 记录 4 · QUARANTINE	
Mode	QUARANTINE
Entry	输入或语义未知
Hard Properties	隔离、审计、不可污染正式结果
Allowed Degradation	活性
Forbidden	自动猜测修复
Exit	人工/规则裁决

表 7 · 记录 5 · MAINTENANCE	
Mode	MAINTENANCE
Entry	计划变更
Hard Properties	数据和权限边界
Allowed Degradation	可用性
Forbidden	未声明数据迁移
Exit	迁移验证完成

表 7 · 记录 6 · EMERGENCY	
Mode	EMERGENCY
Entry	正常控制面阻断关键恢复
Hard Properties	审计、时限、最小权限
Allowed Degradation	正常审批路径
Forbidden	无期限 break-glass
Exit	补偿审查完成

8.5 Gate 5：Architecture Preserves Semantics

GO 条件：

- 架构能够逐项解释关键性质如何被维护；
- 平面和权限边界合理；
- 快慢路径与故障隔离明确；
- 运行模式可判定、可观测、有退出条件；
- 技术组件是推导结果，而不是前置约束的伪装。

9. Stage 6: Resource Bounds, Capacity, Economics 与 Control Loops

9.1 Resource Budget

必须为以下资源定义上界：

```
inflight work
inflight bytes
queue depth / queued bytes
threads / tasks / permits
connections / file descriptors
cache and temporary disk
retry attempts / retry time
reconciliation backlog
per-tenant share
```

9.2 Queue Contract

每个队列记录：目的、容量单位、容量值、生产/消费速率、满载行为、背压方向、公平性、数据持久性、drain/crash 语义、指标和 Owner。

9.3 Capacity Worksheet

最小计算：

```
input rate
x work per input
= CPU / I/O / network demand

inflight bytes
= arrival byte rate x residence time

required service rate
>= peak arrival rate + recovery catch-up rate
```

同时评估：

- 单节点与集群上限；
- N-1 或目标故障域降容；
- 扩容和冷启动延迟；
- 恢复积压的清空窗口；
- 外部依赖限额；
- 热点与租户公平；
- 尾延迟和排队时间。

9.4 Economic Model

至少提供 C1 粗估：

```
compute
storage read/write
network
message / request charges
retained evidence
replay / recovery
operational overhead
```

并给出单位成本和置信等级。无法计算时必须标为 UNKNOWN，不能写“成本可接受”。

9.5 Control Loop Register

字段	内容
Controlled Variable	queue age、lag、utilization、error rate 等
Sensor / Delay	信号、窗口、采样和延迟
Controller	admission、autoscaling、retry、routing 等
Actuator	permits、replicas、traffic weight、feature flag
Bounds	动作上下限和速率
Interaction	与其他控制回路的耦合
Safe Default	控制器失败时的默认行为
Verification	仿真、故障注入或历史回放

9.6 Gate 6: Stable Operating Region

GO 条件：

- 所有可增长资源有界；
- 背压能够传播到真正可控的上游；
- 峰值、故障降容和恢复容量满足目标；
- retry amplification 有上界；
- 控制回路不存在明显正反馈放大；
- 单位成本有模型和监测路径；
- 新增复杂度有 Owner 和回收条件。

典型 NO_GO：

- async everywhere + unbounded queue；
- 只算稳态，不算恢复；
- autoscaling 被当作无限容量；

- 每层各自重试且没有共享 deadline。

10. Stage 7: Contracts、Security、Evolution 与 Operations

10.1 Contract Checklist

- 稳定 identity 与 idempotency key;
- schema 和 semantic version;
- unknown field / unknown enum / malformed input 行为;
- ordering、duplicate、timeout、retry 和 conflict;
- compatibility window;
- retention 与删除语义;
- audit 与 observability;
- deprecation 和 migration path。

10.2 Security Checklist

- trust boundary 和不可信输入;
- human/service/pipeline/recovery identity;
- least privilege 与 mutation authority;
- secret、key 和证书生命周期;
- data classification、encryption 和 retention;
- control plane / assurance plane 变更权限;
- break-glass 范围、时限、审计与复审;
- dependency / artifact provenance。

10.3 Evolution Checklist

- N/N-1 或声明的兼容矩阵;
- rolling upgrade 混合版本行为;
- expand-migrate-contract;
- dual-read / shadow / reconciliation;
- rollback 与 forward-recovery;
- replay 对历史版本的处理;
- ADR 失效条件和复审触发。

10.4 Operations Checklist

- deploy、upgrade、drain、shutdown；
- rollback、forward repair、reconcile；
- backup、restore、RPO、RTO；
- dependency outage 和区域故障 runbook；
- capacity expansion 和 emergency shedding；
- ownership、escalation 与 incident command；
- quarantine 清理和长期积压处理。

10.5 Gate 7：Day-2 Ready

GO 条件：

- 外部契约和版本策略明确；
- 权限与变更权威符合最小化原则；
- 数据变化具有安全迁移路径；
- 回滚不依赖不成立的数据兼容假设；
- 部署、排空、恢复、重放和灾难恢复可执行；
- 运营 Owner 已接受责任。

11. Stage 8：Verification、Evidence 与 Change Envelope

11.1 Assurance Matrix

每项关键性质建立：

空表模板 · 11.1 Assurance Matrix	
Claim	
Mechanism	
Verification	
Evidence	
Coverage	
Limitations	
Freshness	
Owner	

验证应组合：类型/静态检查、单元与属性测试、契约/集成测试、负载、故障注入、恢复演练、策略检查、运行监控和人工审查。

11.2 Evidence Binding

关键证据必须绑定：

```
subject_digest
configuration_digest
environment_identity
workload_corpus_digest
fault_profile
verifier_identity
produced_at
validity / invalidation rule
coverage and limitations
```

11.3 Assurance Meta-invariants

生产流水线必须检查：

- 缺失或过期证据不转为 PASS；
- required checks、验证器和豁免规则的修改本身受门禁；
- 高风险审批职责适当分离；
- 证据不可静默修改；
- 保证面不可用时不默认放行；
- 运行漂移会使相关结论失效；
- 例外和 break-glass 有范围、到期与补偿复审。

11.4 Change Envelope

重要变更必须定义：

```
semantic delta
assumptions
maximum exposure
blast-radius boundary
representative traffic / corpus
observation window
success criteria
abort conditions
rollback / forward-recovery
owner and decision authority
evidence to retain
```

11.5 Gate 8: Evidence Is Decision-Grade

GO 条件：

- 每项高风险性质有适当验证；

- 证据与当前制品、配置、环境绑定；
- 覆盖和限制被明确记录；
- 性能证据使用代表性负载与统计方法；
- 故障与恢复证据覆盖关键边界；
- Change Envelope 有自动或明确的停止条件。

UNKNOWN 条件：

- 只展示测试数量或覆盖率，没有对应性质；
- 性能数据来自不同硬件或不同制品；
- 没有验证重复、崩溃或依赖退化；
- canary 没有预定义成功和退出条件。

12. Stage 9: Production Readiness 与持续保证

12.1 Gate 9: Production Admission

生产放行前必须确认：

```
all mandatory gates resolved
all blocking UNKNOWN resolved or explicitly rejected
residual risks accepted by authority
operational owner ready
runbooks and dashboards available
rollback / forward-recovery viable
change envelope configured
evidence retained and traceable
```

12.2 放行后的持续保证

运行时必须持续观察：

- 工作负载是否越过假设范围；
- 关键依赖、Schema、策略和配置是否漂移；
- 证据是否到期或失效；
- 关键性质对应的语义指标是否异常；
- retry amplification、queue age 和 recovery debt；
- 单位成本和容量 headroom；
- 例外是否到期；
- Assurance Plane 是否可用且未被绕过。

12.3 Incident → Assurance Delta

重大事故、近失事件或长时间 UNKNOWN 必须产生至少一项保证增量：

```
new or revised property
new assumption / boundary contract
new failure scenario
new mechanism
new verification
new runtime evidence or alert
new authority / runbook
revised capacity or economic model
revised ADR
```

13. 可复用工程模板

13.1 Decision Brief 与 Design Dossier

13.1.1 一页决策摘要

用于会议入口，控制为一页；详细推导通过链接引用，不把全部设计章节压进一页。

```
# Decision Brief — System / Change

- Decision needed / deadline:
- Context, business goal and non-goals:
- Proposed decision and main alternative:
- Critical properties and scope:
- Principal risks, unknowns and exceptions:
- Evidence / Design Dossier links:
- Decision authority, owner and next action:
```

13.1.2 完整设计档案

以下 18 项保留完整设计语义，按项目风险裁剪材料厚度；它是 Design Dossier，不是一页摘要。

```
# System / Change Name

## 1. Context and Business Outcome

## 2. Problem

## 3. Goals

## 4. Non-goals

## 5. Risk Tier and Failure Consequences
```

字面文本 · continued
6. Workload and Growth
7. SLO / RPO / RTO
8. Constraints and Assumptions
9. Critical Properties
10. State, Identity and Ownership
11. Failure / Consistency / Commit
12. Architecture and Operating Modes
13. Resource Bounds and Capacity
14. Security and Trust Boundaries
15. Evolution / Migration / Recovery
16. Verification and Evidence
17. Alternatives and Decision
18. Unknowns, Exceptions and Review Triggers

13.2 Constraint / Assumption Register

表 10 · 字段记录视图（列名与原单元格逐项对应）

排版说明：横线表示原表空字段，不表示已有值或已完成填写。

表 10 · 记录 1 · A-001	
ID	A-001
Type	Assumption
Statement	
Dependents	
Evidence	
Signal	
Owner	
Expiry / Invalidation	
Response	

假设还须附带以下独立状态字段；约束不机械套用假设状态。唯一受控定义和旧字段迁移规则见 [ASSURANCE §6.2](#)。

```
validation_status: "UNVALIDATED|VALIDATED|VIOLATED|UNKNOWN"
lifecycle_status: "PROPOSED|ACTIVE|EXPIRED|RETIRED"
monitoring_status: "MONITORED|UNMONITORED|UNAVAILABLE|NOT_APPLICABLE"
risk_flag: "NONE|AT_RISK"
last_validated_at: "..."/>

```

13.3 Critical Property Contract

```
### CP-XXX — Property Name

- Category: Safety / Liveness / Durability / Isolation / Recovery / SLO
- Identity:
- Scope:
- Temporal semantics:
- Assumptions:
- Guarantee:
- Allowed degradation:
- Authority boundary:
- Violation consequence:
- Mechanism:
- Verification coverage:
- Evidence freshness:
- Owner:
```

13.4 Property Precedence Matrix

空表模板 · 13.4 Property Precedence Matrix	
Property A	
Property B	
Conflict Scenario	
Priority	
Allowed Sacrifice	
Decision Mechanism	
Authority	

13.5 State Ownership Register

空表模板 · 13.5 State Ownership Register

State	
Meaning	
Identity	
Class	
Authority	
Mutation Authority	
Technical Owner	
Operational Owner	
Recovery Source	
Conflict	

13.6 Boundary Contract Register

空表模板 · 13.6 Boundary Contract Register

Boundary	
Assumptions	
Guarantees	
Violation Behavior	
Version	
Capacity / Time Assumption	
Verification	
Owner	

13.7 Failure / Recovery Matrix

空表模板 · 13.7 Failure / Recovery Matrix

Failure	
Detection	
Possible Partial Effect	
Authority	
Automatic Action	
Retry Safety	
Manual Action	
Bound	
Evidence	

13.8 Operating Mode Matrix

空表模板 · 13.8 Operating Mode Matrix

Mode	
Entry / Authority	
Hard Properties	
Allowed Degradation	
Forbidden	
Resource Policy	
Exit	
Max Duration	
Evidence	

13.9 Queue / Resource Contract

Resource / Queue Name

- Purpose:
- Capacity unit:
- Hard bound:
- Producer rate:
- Consumer rate:
- Full behavior:
- Backpressure direction:
- Fairness / isolation:
- Persistence:
- Drain / crash semantics:
- Metrics and alerts:
- Owner:

13.10 Capacity & Economic Worksheet

Workload

- item rate:
- byte rate:
- size distribution:
- peak and burst:
- growth:

Per-item Work

- CPU:
- memory residency:
- network:
- storage I/O:
- external calls:

Stage Capacity

Stage	Service Time	Parallelism	Capacity	Utilization Target	Queue Bound
1	10	1	10	0.9	10
2	10	1	10	0.9	10
3	10	1	10	0.9	10
4	10	1	10	0.9	10
5	10	1	10	0.9	10
6	10	1	10	0.9	10
7	10	1	10	0.9	10
8	10	1	10	0.9	10
9	10	1	10	0.9	10
10	10	1	10	0.9	10
11	10	1	10	0.9	10
12	10	1	10	0.9	10
13	10	1	10	0.9	10
14	10	1	10	0.9	10
15	10	1	10	0.9	10
16	10	1	10	0.9	10
17	10	1	10	0.9	10
18	10	1	10	0.9	10
19	10	1	10	0.9	10
20	10	1	10	0.9	10
21	10	1	10	0.9	10
22	10	1	10	0.9	10
23	10	1	10	0.9	10
24	10	1	10	0.9	10
25	10	1	10	0.9	10
26	10	1	10	0.9	10
27	10	1	10	0.9	10
28	10	1	10	0.9	10
29	10	1	10	0.9	10
30	10	1	10	0.9	10
31	10	1	10	0.9	10
32	10	1	10	0.9	10
33	10	1	10	0.9	10
34	10	1	10	0.9	10
35	10	1	10	0.9	10
36	10	1	10	0.9	10
37	10	1	10	0.9	10
38	10	1	10	0.9	10
39	10	1	10	0.9	10
40	10	1	10	0.9	10
41	10	1	10	0.9	10
42	10	1	10	0.9	10
43	10	1	10	0.9	10
44	10	1	10	0.9	10
45	10	1	10	0.9	10
46	10	1	10	0.9	10
47	10	1	10	0.9	10
48	10	1	10	0.9	10
49	10	1	10	0.9	10
50	10	1	10	0.9	10
51	10	1	10	0.9	10
52	10	1	10	0.9	10
53	10	1	10	0.9	10
54	10	1	10	0.9	10
55	10	1	10	0.9	10
56	10	1	10	0.9	10
57	10	1	10	0.9	10
58	10	1	10	0.9	10
59	10	1	10	0.9	10
60	10	1	10	0.9	10
61	10	1	10	0.9	10
62	10	1	10	0.9	10
63	10	1	10	0.9	10
64	10	1	10	0.9	10
65	10	1	10	0.9	10
66	10	1	10	0.9	10
67	10	1	10	0.9	10
68	10	1	10	0.9	10
69	10	1	10	0.9	10
70	10	1	10	0.9	10
71	10	1	10	0.9	10
72	10	1	10	0.9	10
73	10	1	10	0.9	10
74	10				

Failure / Recovery

- N-1 capacity:
- backlog after outage:
- catch-up target:
- scaling delay:

Economics

- compute:
- storage:

字面文本 · continued

- network:

- managed-service charges:

- replay / recovery:

- evidence retention:

- unit cost:

- confidence level: C0 / C1 / C2 / C3

13.11 Control Loop Register

空表模板 · 13.11 Control Loop Register

Loop	
Variable	
Sensor / Delay	
Controller	
Actuator	
Bounds	
Interaction	
Safe Default	
Verification	
Owner	

13.12 Verification & Evidence Matrix

空表模板 · 13.12 Verification & Evidence Matrix

Property	
Mechanism	
Verification	
Evidence ID	
Subject / Config / Env	
Coverage	
Limitations	
Status	
Freshness	
Owner	

证据生命周期摘要：CURRENT / STALE / INVALIDATED / SUPERSEDED / NOT_REPRODUCIBLE；唯一受控定义见 ASSURANCE §12.3，不与执行结果或 Claim 状态混用。

13.13 Change Envelope

- # Change Envelope
- Change / semantic delta:
- Assumptions:
- Risk tier:
- Maximum exposure:
- Blast-radius boundary:
- Representative traffic / corpus:
- Observation window:
- Success criteria:
- Abort conditions:
- Rollback / forward-recovery:
- Owner:
- Decision authority:
- Evidence to retain:

13.14 Exception / Risk Acceptance

本模板是 [GOV §9](#) 的摘要视图；授权、期限和状态以该处为准，不以填写模板替代批准。

- # Exception EX-XXX
- Requirement being bypassed:
- Scope:
- Reason:
- Risk:
- Compensating control:
- Owner:
- Approver:
- Start:
- Expiry:
- Revocation condition:
- Required follow-up:
- Evidence / audit link:

无到期时间的例外不得进入正式基线。

13.15 ADR Template

```
# ADR-NNNN: Decision Title

- Status:
- Date:
- Owner:
- Risk tier:

## Context

## Problem and Constraints

## Decision

## Assumptions

## Alternatives Considered

## Consequences

## Risks and Unknowns

## Security and Operational Impact

## Capacity and Cost Impact

## Verification Evidence

## Reversibility

## Migration / Rollback / Forward-Recovery

## Invalidation Conditions

## Review Triggers

## Follow-up Actions
```

13.16 Production Readiness Decision

```
# Production Readiness Decision

- System / change:
- Version / subject digest:
- Environment:
- Risk tier:
- Decision: GO / CONDITIONAL_GO / NO_GO / UNKNOWN
- Decision authority:
- Date:

## Gate Summary
| Gate | State | Evidence / Finding | Owner | Expiry | Follow-up |
|---|---|---|---|---|
## Blocking Unknowns
-

## Accepted Residual Risks
- scope:
- risk owner:
- compensating control:
- expiry / revocation:

## Change Envelope
- maximum exposure:
- observation window:
- abort conditions:
- rollback / forward-recovery:

## Evidence Package
- property evidence:
- capacity evidence:
- fault / recovery evidence:
- security evidence:
- cost evidence:

## Final Sign-off
- Semantic Owner:
- Operational Owner:
- Security / Compliance:
- Assurance Reviewer:
- Decision Authority:
```

13.17 Incident → Assurance Delta

```
# Incident / Near-miss Assurance Delta

- Incident:
- Original blind spot:
- Affected property / assumption / contract:
- New or revised property:
- Mechanism change:
- Verification change:
- Runtime evidence / alert change:
- Runbook / authority change:
- Capacity / cost change:
- ADR / architecture change:
- Owner:
- Completion evidence:
```

事故复盘只有在改变性质、假设、机制、验证、证据、权限或架构中的至少一项后，才形成了可验证的系统性改进。

14. Architecture Review 会议操作

14.1 会前材料

Reviewer 应在会前获得：

- 一页 Decision Brief (§13.1.1) 及链接的 Design Dossier (§13.1.2)；
- 风险等级；
- Critical Property Register；
- State / Ownership 与 Failure Matrix；
- 至少一个备选方案；
- 容量粗算；
- 关键未知和需要会议裁决的问题。

没有这些材料时，会议很容易退化为现场看图和技术偏好辩论。

14.2 会议顺序

推荐顺序：

1. 作者用五分钟说明问题、非目标和风险；
2. 先审关键性质与状态权威；
3. 再审失败、提交、资源和恢复；
4. 然后审架构是否维护这些语义；
5. 最后才讨论技术选型、实现与优化；

6. 明确 GO / CONDITIONAL_GO / NO_GO / UNKNOWN;
7. 每个未决项必须有 Owner、期限和阻断级别。

14.3 Reviewer 应避免

- 用个人熟悉度替代约束分析;
- 用“业界都这样”替代性质证明;
- 在未理解权威状态前建议拆服务;
- 只审正常路径;
- 用更多监控掩盖语义不清;
- 把所有问题都推成“实现细节”;
- 追求文档完整度而忽视真实高风险缺口。

14.4 评审记录

每次评审至少记录:

```
decision state
blocking findings
accepted residual risks
unknowns
exceptions
actions / owners / deadlines
evidence required
review trigger
```

15. 常见反模式与校正

15.1 Resume-driven Architecture

症状: 先选 Kafka、Kubernetes、微服务、Rust、Service Mesh，再寻找需求。

风险: 复杂度先于价值，状态和失败语义被组件标签掩盖。

校正: 要求每个组件映射到具体性质、负载或运营问题；无法映射则默认删除。

15.2 Architecture Diagram = Architecture

症状: 架构图精美，但没有状态权威、ACK、失败和恢复。

校正: 图中必须标记权威状态、提交边界、背压、信任边界与故障域；并配合 Register。

15.3 Microservice-first

症状: 按名词或团队人数拆服务，每个服务共享数据库或同步级联。

风险: 形成分布式单体，增加协调和恢复状态。

校正：优先 modular monolith 或清晰 bounded context；只有独立状态、变化率、扩展或故障隔离需求才引入分布式边界。

15.4 Unbounded Async

症状：异步下载、解压、处理和写出之间全部使用无界 channel。

风险：下游变慢时内存增长、超时、重试和级联故障。

校正：按 bytes 或真实成本设 permit，建立端到端背压和满载策略。

15.5 Exactly-once Magic

症状：依赖消息系统或流框架标签声称端到端 exactly-once。

风险：外部副作用、重放和跨系统提交仍会重复。

校正：定义业务 effect 的作用域、稳定身份、原子提交和冲突语义。

15.6 Retry Everything

症状：所有 timeout 和错误都自动重试。

风险：永久错误被放大，依赖故障形成 retry storm。

校正：错误分类、共享 deadline、retry budget、jitter、最大放大比和准入联动。

15.7 Shared Mutable Truth

症状：多个服务或团队直接修改同一权威表。

风险：规则分散、冲突无裁决、迁移不可控。

校正：单一 mutation authority，其他主体通过契约提议变更或消费投影。

15.8 Monitoring After Launch

症状：上线后再补 CPU、内存和错误率。

风险：无法判断系统为什么慢、是否重复、是否使用错误版本。

校正：从性质、队列、状态机和恢复模型推导语义指标。

15.9 Rollback Fantasy

症状：发布计划只写“失败则回滚”。

风险：新数据、Schema 或外部副作用已使旧代码不兼容。

校正：区分 binary rollback、data compatibility、reconciliation 和 forward repair。

15.10 Evidence Illusion

症状：测试很多、覆盖率高、CI 全绿，因此宣称系统正确。

风险：证据没有映射性质，或与当前制品/环境无关。

校正：从 Claim 开始建立 Evidence Matrix，并记录覆盖与限制。

15.11 Unknown Becomes Default Continue

症状：未知版本、未知身份或证据缺失时使用默认值继续。

风险：把认识不足转换为数据污染或越权行为。

校正：根据风险 fail closed、quarantine 或返回显式 UNKNOWN。

15.12 Gate Bureaucracy

症状：门禁数量增加，但与风险无关；团队只优化绿色状态。

风险：Goodhart 化、交付停滞、真实风险反而被隐藏。

校正：门禁必须关联关键性质或事故；定期删除低价值检查；按风险裁剪。

16. 完整案例：对象存储 CAN 分块文件处理流水线

本案例是协议推导与评审练习，不是已经完成实现验证的参考系统。数值、Gate 结论与反例轨迹均为说明性材料；正式设计必须用真实采样、基准、故障测试和授权记录替换。

16.1 场景、风险等级与 Gate 映射

- 文件发现服务将对象元数据写入 Kafka；
- 每条消息包含对象 URI、对象版本、大小、校验信息和采集元数据；
- 单个对象不超过 40 MiB；
- 文件由大量“业务头 + gzip member”连续组成，典型约 3000 个 member；
- 每个 member 属于一个车型，需要使用确定版本的 DBC 解码；
- gzip 解压后执行 CPU-heavy CAN 解析、信号解码和聚合；
- URL 数量极大，系统更关注持续吞吐、成本与可恢复性，而非单文件最低尾延迟。

风险等级

该系统默认按 R2 — Core Production 设计：它处理持续产生的生产数据，结果会被下游分析、诊断或业务流程消费，错误 DBC、重复正式结果或不可追溯输出可能形成长期数据污染。若结果进一步参与车辆安全控制、资金结算或法规判断，应提升至 R3 — High Assurance，增加独立审批、职责分离和持续证据漂移检测。

Gate 映射

评审门禁	本案例对应内容
Gate 0	问题、Goals、Non-goals 与风险等级
Gate 1	Workload、SLO、约束与关键假设
Gate 2	Critical Properties 与优先级
Gate 3	Source/Operation/Result Identity、状态权威与 DBC 绑定
Gate 4	失败、提交点、ACK 顺序与恢复
Gate 5	数据面、控制面、调度、快慢路径与运行模式
Gate 6	有界资源、容量、恢复能力和单位经济性
Gate 7	契约、安全、演进、运维与语言决策
Gate 8	Verification & Evidence Matrix
Gate 9	Change Envelope、停止条件与生产裁决

16.2 Goals / Non-goals

Goals

1. 高吞吐、可水平扩展；
2. 内存、任务、连接、输出和重试全部有界；
3. Kafka 重复投递和 worker crash 不产生重复正式结果；
4. DBC 身份和版本可追踪，未知或不匹配时不猜测；
5. partial output 不作为 completed result 暴露；
6. 可重放、可审计、可恢复；
7. CPU pipeline 可测量并按瓶颈优化。

Non-goals

1. 不保证全局文件顺序；
2. 不追求每个 member 的独立超低延迟；
3. 不在第一阶段实现跨区域 active-active；
4. 不通过无限并发隐藏对象存储或 DBC Registry 变慢；
5. 不把“最新 DBC”作为隐式处理语义。

16.3 Workload、SLO、约束与假设

工作负载模型

正式设计必须从生产采样建立分布，而不是只记录均值。至少测量：

```
files / second: average, p95, peak, burst duration
compressed bytes / file: p50, p95, p99, hard maximum
members / file: p50, p95, p99, hard maximum
compressed bytes / member and inflate ratio
vehicle-model / DBC skew
CPU seconds / member and output amplification
object-store, registry and result-store latency distribution
corrupt / unsupported / unknown-model ratio
replay and recovery workload
```

设计容量必须同时覆盖持续新流量、故障降容和积压恢复；仅验证稳态平均负载不足以支持生产结论。

初始 SLO 示例

维度	示例目标	说明
端到端完成 延迟	99% 的受支持对象在目标窗口内正式提交	从 Kafka 可见到 completion manifest
数据完整性	重复正式结果 0；未标记 partial completion 0	硬性质，不以延迟换取
DBC 正确性	正式结果的 DBC identity/digest 覆盖率 100%	未知必须隔离
可恢复性	N-1 条件下在约定时间清空积压	必须包含持续新流量
资源安全	RSS、临时磁盘、连接和队列始终低于硬上限	达到上限时停止接纳
可追溯性	任意正式结果可追溯 source、pipeline、config、binding 与 DBC digest	作为发布与审计条件

数值必须绑定具体 workload corpus、节点规格、依赖版本和成本边界。没有这些限定的“吞吐”或“p99”不构成可复用证据。

关键约束

- 源对象必须可通过不可变版本或内容摘要稳定寻址；
- 单文件当前合同上限为 40 MiB，但 admission 仍按 bytes 而非文件数控制；
- Kafka 提供至少一次投递语义，系统必须吸收重复；
- 对象、member、车型和 DBC 分布可能偏斜；
- CPU-heavy 解码不得占用异步 I/O executor；
- 生产结果只在权威 completion manifest 提交后可见；
- DBC Registry 和 Result Store 可能超时、降级或短时不可用；
- 技术选型受既有库、团队生产经验、诊断能力和 on-call 责任约束。

关键假设登记示例

表 20 · 字段记录视图（列名与原单元格逐项对应）

表 20 · 记录 1 · A-01	
ID	A-01
假设	单对象不超过生产合同上限
运行信号	object_size
失效条件	任意超限
响应	admission reject；复审 producer contract

表 20 · 记录 2 · A-02	
ID	A-02
假设	member 数和 inflate ratio 处于建模范围
运行信号	members/object、inflate ratio
失效条件	超阈值或分布漂移
响应	quarantine；容量与格式复审

表 20 · 记录 3 · A-03	
ID	A-03
假设	对象版本读取稳定
运行信号	version/checksum mismatch
失效条件	任意不一致
响应	停止处理；重新读取权威版本

表 20 · 记录 4 · A-04	
ID	A-04
假设	同版本 DBC artifact 内容不可变
运行信号	digest mismatch
失效条件	任意不一致
响应	fail closed；触发 Registry 事故

表 20 · 记录 5 · A-05	
ID	A-05
假设	Result Store 的 conditional create 语义可靠
运行信号	contract/fault test
失效条件	语义或实现变化
响应	阻断发布；重新设计提交协议

表 20 · 记录 6 · A-06	
ID	A-06
假设	当前“整对象下载”在成本与内存上优于细粒度 range
运行信号	benchmark、request amplification
失效条件	对象上限/延迟目标/存储计费变化
响应	重新比较 streaming/range 方案

假设必须有 Owner、复审触发和证据状态。失效假设不得继续作为已批准架构的隐含前提。

16.4 关键性质

CP-01 — 单一正式完成结果

Identity:
OperationIdentity = canonical source URI + immutable source version/checksum + PipelineSemanticVersion (完整定义见 §16.5)
Guarantee:
Result Store 最多存在一个正式 completion manifest。
Mechanism:
staged output + create-only/conditional completion manifest。
Allowed degradation:
确认同一 OperationIdentity 的正式结果可复用；并发候选只有满足 §16.11 的严格结果等价关系才返回 ALREADY_COMMITTED, 否则进入 RESULT_CONFLICT / reconciliation。

CP-02 — DBC 可追踪且不猜测

每个正式输出必须包含 `vehicle_model`、`dbc_id`、`dbc_version`、`dbc_content_digest` 与 `pipeline_semantic_version`。

`UNKNOWN_MODEL` / `DBC_NOT_FOUND` / `VERSION_MISMATCH`

必须进入显式隔离状态，不得回退到 `latest` 或相似车型。

CP-03 — Partial output 不可见为正式完成

成员级输出可以暂存，但只有 `completion manifest` 提交后，下游才将该对象视为完整正式结果。

CP-04 — 资源有界

`inflight compressed bytes`、`active files`、`CPU jobs`、`output bytes`、`open connections`、`retry budget` 和 `reconciliation backlog` 均有硬上限。

CP-05 — 已接纳任务有持续责任与明确处置

`business dispositions`: `COMMITTED` / `QUARANTINED` / `PERMANENTLY_REJECTED`

`nonterminal responsibility`: `RECONCILIATION_REQUIRED`

已接纳操作不得静默丢失。终止处置必须持久化并可恢复；隔离/拒绝不能掩盖未决提交。`reconciliation` 的 `attempt` 可以结束，但责任必须继续或持久化移交，见 §7.3。

“最终进入处置状态”属于有条件的无界活性主张，依赖公平调度、持续资源、依赖恢复及可完成的权威裁决；有限观察不能证明最终完成。另定义队列年龄、处理 `deadline` 和升级时间等有界运营目标，未达标按其自身 `Claim` 记录违反，不伪造业务终态。

16.5 状态与所有权

表 21 · 字段记录视图（列名与原单元格逐项对应）

表 21 · 记录 1 · Discovery offset	
State	Discovery offset
Authority	Kafka consumer group
Identity	topic/partition/offset
Owner	Processing Platform
Recovery	Kafka

表 21 · 记录 2 · Source bytes	
State	Source bytes
Authority	Object Storage
Identity	object URI + immutable version
Owner	Data Ingestion
Recovery	re-read exact version

表 21 · 记录 3 · DBC artifact	
State	DBC artifact
Authority	DBC Registry
Identity	dbc_id + version + digest
Owner	Vehicle Data Platform
Recovery	immutable artifact store

表 21 · 记录 4 · Model→DBC binding	
State	Model→DBC binding
Authority	Versioned Binding Registry
Identity	binding version + model
Owner	Vehicle Data Platform
Recovery	versioned snapshot

表 21 · 记录 5 · Staged outputs

State	Staged outputs
Authority	Result Store staging namespace
Identity	run_id + member identity
Owner	Processing Platform
Recovery	对账后仅回收确定不再被引用、且无迟到提交可能的对象

表 21 · 记录 6 · Completion manifest

State	Completion manifest
Authority	Result Store
Identity	OperationIdentity
Owner	Processing Platform
Recovery	authoritative read / conditional create

表 21 · 记录 7 · Disposition / reconciliation record

State	Disposition / reconciliation record
Authority	Durable responsibility ledger
Identity	OperationIdentity + attempt / handoff ID
Owner	Processing Platform；具名接收 Owner
Recovery	重启扫描、恢复对账与升级；不替代 completion authority

表 21 · 记录 8 · Runtime queues

State	Runtime queues
Authority	Worker process
Identity	ephemeral
Owner	Runtime Owner
Recovery	rebuild from durable state

表 21 · 记录 9 · Verification evidence

State	Verification evidence
Authority	Evidence Store
Identity	evidence_id + subject digest
Owner	Assurance Owner
Recovery	immutable retained record

稳定身份

```
SourceIdentity
  = canonical object URI
  + immutable object version or source checksum

PipelineSemanticVersion
  = processor version
  + output contract version
  + semantic configuration digest
  + DBC binding policy / snapshot version

OperationIdentity
  = SourceIdentity + PipelineSemanticVersion

ResultIdentity
  = OperationIdentity + output partition or range identity
```

Kafka offset、Worker 实例、临时文件路径和本次启动时间都是执行身份，不能替代业务幂等身份。重新处理同一源对象但采用不同语义版本时，应形成新的 `OperationIdentity`，而不是覆盖旧结果。

16.6 DBC 绑定语义

不得在 worker 执行时隐式查询“当前最新 DBC”。推荐在接纳或调度阶段解析出不可变绑定，并将实际 binding snapshot 纳入 `OperationIdentity`；重试、移交与重放同一操作不得再次解析“当前”绑定：

```
DecodeBinding {
  vehicle_model
  dbc_id
  dbc_version
  dbc_content_digest
  binding_policy_version
  effective_time_basis
}
```

`effective_time_basis` 必须明确是采集时间、文件产生时间、任务接纳时间还是显式版本。否则重放相同源数据可能产生不同结果。

数据面只消费不可变 binding 和 DBC artifact；Binding Registry 不可用时，任务可以暂停或隔离，但不能猜测。

16.7 推荐逻辑架构

Kafka

- > Admission / Dispatcher
 - > Object Fetchers (async I/O, bounded byte permits)
 - > File Buffer / Bounded Spool
 - > Framer
 - > CPU Work Scheduler
 - > Decompress + Parse + DBC Decode
 - > Bounded Aggregation / Output
 - > Staging Store
 - > Conditional Completion Manifest
 - > Kafka ACK

Control Plane:

- pipeline semantic version
- DBC binding snapshot
- resource limits
- retry / quarantine policy

Management Plane:

- deploy / drain / replay / reconcile / emergency actions

Assurance Plane:

- contract tests / load tests / fault injection / evidence / gates

16.8 为什么默认避免“两遍偏移预扫描”

方案二“先下载并记录所有 gzip 偏移，再交给下游线程”是否有价值，取决于外层格式是否提供可信的压缩长度：

1. 若业务头包含经过校验的 **compressed_length**, Framer 可以低成本单遍定位 member 边界；此时记录 offset table 有利于并行调度、重试和局部位。
2. 若没有可信长度, gzip/deflate 本身没有可直接跳过到末尾的固定长度字段；发现 member 结束通常需要实际解析压缩流。此时“预扫描”可能等价于先做一遍解压/解析，再由下游重复工作，得不偿失。

因此推荐：

有可靠外层长度：

single download -> cheap framing -> parallel member work

无可靠外层长度：

single download -> one-pass decompress/framing -> decode or fused handoff

不得把无法证明正确的 magic-byte 搜索作为 gzip 边界算法。

16.9 下载与缓冲策略

由于单文件硬上限为 40 MiB，默认优先考虑：

```
完整对象单次下载到 pooled buffer
+ 全局 inflight-byte permits
+ 单遍 framing
+ 不重复读取对象存储
```

这通常比为 3000 个小 member 发起大量 Range 请求更简单、更经济，也能避免对象存储请求放大。其安全前提是全局按 bytes 限制在途文件，而不是只限制文件数。

若实际吞吐使内存预算不可接受，可选择 bounded spool / mmap 临时文件，但必须增加临时磁盘上限、清理、崩溃恢复和 I/O 模型；不能把 spill 当作免费内存。

16.10 CPU 调度与局部性

推荐分离：

- 异步 I/O executor：对象下载和外部存储请求；
- 固定 CPU worker pool：解压、解析、DBC decode；
- 有界输出 executor：序列化与结果写入。

通常应优先融合 decompress + parse + DBC decode，避免把大块未压缩中间数据跨队列传递。若基准表明解压与 decode 的瓶颈、向量化或库约束明显不同，再拆分独立 CPU stage。

调度可以采用：

```
bounded global ready queue
+ per-worker local deque / work stealing
+ DBC/model locality hint
+ fairness and starvation bound
```

不要为每个车型永久创建无界队列。局部性优化必须服从资源上界与租户公平。

批大小不应拍脑袋固定。应通过基准在以下目标间权衡：

- 调度开销；
- DBC cache locality；
- cancellation 粒度；
- 输出聚合成本；
- 公平性和尾延迟。

16.11 幂等与提交协议

16.11.1 权威提交与严格结果等价

本案例采用**严格等价**，不是仅按“首个合法结果胜出”。条件创建只保证同一 `OperationIdentity` 至多一个正式 `manifest`，不能证明不同执行得到相同业务结果。

`ResultEquivalence` 必须同时核对 `OperationIdentity`、规范化业务结果摘要与必要 `provenance`：

- 按 `source member index`、`frame/event identity` 和明确排序规则生成稳定记录序列；
- 固定字段编码、数值/浮点归一化、空值、单位、`Schema` 与 `output contract version`；
- 对 `member` 覆盖、错误/丢弃处置、计数、每个输出范围及总结果计算内容摘要；
- `provenance` 包含 `source digest`、`pipeline semantic version`、`semantic config digest`、`binding snapshot` 与实际 `DBC digests`；
- 排除 `run_id`、`worker ID`、临时路径、执行时间等非业务字段；若时间或顺序本身有业务含义，则不得排除。

规范化算法和摘要算法必须版本化并验证确定性；非确定性业务输出需另行设计等价合同，不得沿用此案例自动放宽比较。相同语义版本只是必要身份条件，不是结果等价证据。

对象级提交流程：

1. 接收 `Kafka record`；正式接纳前，持久化源身份、固定绑定、`OperationIdentity` 及责任记录，登记 `partition`、`offset` 与 `assignment epoch`；禁用自动提交。接纳记录不可写时停止接纳，不能只依赖 `worker` 内存保存已接纳责任。
2. 先读取持久化责任/处置记录，再权威读取 `manifest`。存在未闭合 `UNKNOWN` 或 `RESULT_CONFLICT` 时必须恢复对账并比较已保存候选，不得走“已有结果”快路径将其抹除。无未决记录且身份、必要 `provenance`、输出完整性与摘要可验证时，才可复用已有正式结果；这不宜称另一未执行候选必然等价。已持久化的合法隔离/拒绝应按合同复用处置，后续重放须显式授权，不静默覆盖原记录。
3. 无已确认可复用结果或终止处置时，将当前执行输出写入独立 `run_id staging`，校验成员覆盖、完整性和 `error policy`，生成规范化摘要与不可变 `manifest` 候选。在发起提交前持久化 `intent`（含 `OperationIdentity`、`attempt ID`、候选 `digest`、`staging refs` 与 `Owner`），确保进程在发送后、处理返回前崩溃也能恢复不确定责任。
4. 以完整 `OperationIdentity` 的无歧义编码/摘要作为条件创建键，重试保持同一键和相同请求意图。`manifest` 引用的输出必须先持久化；禁止覆盖正式 `manifest`。
5. 按 §7.3 区分调用结论：确认成功后持久化完成记录；确认无效果时按预算重试；确认冲突时权威读取既有 `manifest` 并比较；未知时进入持久化 `reconciliation`。
6. 当前候选与既有结果严格等价才记 `ALREADY_COMMITTED`。同身份、同版本但结果摘要或必要 `provenance` 不同，记 `RESULT_CONFLICT`，保留两个候选和差异证据，停止该操作的成功推进并升级裁决；既有 `manifest` 仍是权威事实，不能覆盖，也不能把冲突执行记成正常成功。
7. 只有确认成功、可验证复用或合法持久化终止处置，才标记 `record` 为可推进；`Kafka` 恢复位置仍由 §16.11.3 的分区协调器决定，不能由单个 `worker` 直接提交。

16.11.2 未知结果与责任恢复

超时后 reconciliation 使用同一 OperationIdentity 读取权威事实；not-found 仍可能与在途迟到成功并存。仅在后端合同提供“不会再生效”的确认，或安全的同键同意图重试已得到确定结论时，才能结束不确定窗口。不得把超时当作 CONFIRMED_NO_EFFECT，也不得因重试次数耗尽而丢弃责任。

持久化 ledger 记录候选 manifest digest、staging refs、原请求身份、已知结果、接收 Owner、重试/升级期限和交接确认。重启或接管后扫描未闭合记录；依赖不可用时保留未知并背压，超过期限升级给运营 Owner。本案例不允许仅凭“已写入 reconciliation 队列”推进该 record 的 Kafka 位置。

GC 不得仅凭 staging 年龄删除未决输出。回收必须证明没有正式 manifest 引用、没有仍可能迟到生效的写入，并已完成安全对账或确认无效果；还须通过受控状态转换阻止新执行引用待回收对象，消除检查后又被引用的竞态。否则保持受保护引用。达到存储或责任积压上限时停止接纳，不通过删除未决数据制造空间。

QUARANTINED/PERMANENTLY_REJECTED 作为交付终止处置，须持久化源身份、原因、处理策略版本、修复/重放 Owner 与可恢复记录，并通过协调/fencing 确认其他执行不能继续为该已终止操作发起提交、既有在途请求不会再生效。它们结束本次处理责任的方式必须可审计，不能等同于成功输出。

上述超时与幂等设计是按后端合同推导的工程选择；参考 [AWS Builders' Library](#)：幂等 API 与安全重试，实际存储实现仍需单独验证。

16.11.3 Kafka 分区连续完成前缀

这里限定为传统 consumer group 的 KafkaConsumer 分区 offset 语义，不适用于其他逐条确认协议。提交的是每个 partition 的下次恢复位置，不是单条 record 的独立 ACK。基础语义参考 [Apache Kafka 4.1 KafkaConsumer API](#)。

本案例选择“分区内并发处理、按实际投递顺序的连续已处置前缀推进”：

1. 单一 consumer/coordinator 线程拥有 poll、pause/resume 与 commit；worker 只回报完成事件，不能共享调用非线程安全 consumer。
2. 每个 partition 在当前 assignment epoch 内，按 poll 实际交付顺序登记 record 与可恢复 next-position（保留相应 leader epoch 元数据）；完成表包含成功、合法隔离/拒绝或未决。只有前缀中的每条 record 都已持久化处置，才允许推进。
3. 若 offset 100 未决、101 已完成，不能提交 102；101 完成只能登记在窗口中等待。这里的“连续”指交付序列前缀，不要求数值 offset 相邻；事务控制记录、日志压实（compaction）或隔离级别过滤等可能造成间隙；应用自行跳过业务 record 则必须另有合法处置依据。
4. 前缀后仍有已交付未处置 record 时，next offset 不得越过它，使用该 record 的 offset；已处理完整批次时，使用该批次捕获的 ConsumerRecords.nextOffsets() 等经所用客户端合同确认的后继位置。不得把最新 poll position 用于较早的未完成批次，也不得靠逐个 offset 加一构造不存在的 record。
5. 完成事件绑定 partition、offset 与 assignment epoch。撤销时停止分发，仅在仍持有所有权且协议允许时提交安全前缀；分区丢失或新 assignment 后，旧回调不得更新新窗口或提交位置。Kafka group/generation 校验与应用 epoch 检查配合使用。

6. 提交由协调器串行管理，禁止乱序异步重试旧 `offset` 使已确认位置倒退；提交失败或结果未知时保留未确认状态，按客户端协议恢复，不假定 `broker` 已接受。重新投递通过权威 `manifest/` 处置记录吸收重复。
7. 同时限制在途条数、字节数、完成窗口大小与最老未决年龄。前缀阻塞触及预算时 `pause` 相应分区并向上游背压，`consumer` 继续满足 `poll/`心跳契约；超时升级或交接责任，不跳过空洞强制提交。

Kafka 保留窗口与可接受停顿时间必须进入恢复合同；位置因保留/截断不可恢复时，应报警并从持久化接纳记录及源目录核对缺口，不得静默 `reset` 到 `latest` 并宣称无丢失。尚未接纳的发现消息如何重建，由发现服务与不可变源目录合同负责。

`assignment fencing` 保护的是消费协调状态，不自动撤销已经发往 `Result Store` 的副作用；迟到 `worker` 写入仍必须受同一 `OperationIdentity` 的条件创建与 `reconciliation` 协议约束。

若业务允许 `member` 级独立可见，可把该模式下沉到 `member identity`，但会增加状态、提交放大与 `reconciliation` 成本，需重新评审等价合同和交付边界。

16.12 有界资源示例

```
max_active_files           = derived from byte budget
max_inflight_source_bytes  = B_source
max_cpu_jobs               = near measured CPU saturation point
max_queued_compressed_bytes= B_work
max_output_buffer_bytes    = B_output
max_object_connections     = object-store quota aware
max_retry_attempts         = classified by error
max_retry_time             = within end-to-end deadline
max_reconciliation_backlog = explicit operational bound
```

准入应同时受 `source bytes`、`CPU permits` 和 `output pressure` 约束。只按 `Kafka message` 数量暂停不足以控制大小差异和 `CPU` 成本差异。

16.13 容量模型

定义：

```
lambda_f = files / second
S_c       = average compressed bytes / file
N_m       = members / file
t_d       = CPU seconds / member for decompress + decode
R_o       = output bytes / input byte
```

则：

```
network input demand = lambda_f x S_c
member rate           = lambda_f x N_m
CPU core lower bound = lambda_f x N_m x t_d
output bandwidth      = lambda_f x S_c x R_o
```

实际 CPU 预算还要加入 framing、聚合、序列化、缓存 miss、调度开销和 headroom。

示例：

```
lambda_f = 25 files/s
S_c      = 20 MiB/file
N_m      = 3000

t_d      = 80 microseconds/member

input bandwidth ~= 500 MiB/s
member rate     = 75,000/s
CPU lower bound = 6 CPU-seconds/s ~= 6 fully utilized cores
```

若目标利用率为 60%–70%，并考虑恢复和抖动，实际配置显著高于理论 6 核。最终数值必须由代表性语料端到端基准确定。

故障降容与积压恢复

仅证明正常状态下 $\text{service_rate} > \text{arrival_rate}$ 不足以支持高可用结论。还必须验证：

```
recovery_service_rate > live_arrival_rate
catch_up_time ~= backlog / (recovery_service_rate - live_arrival_rate)
```

N-1、依赖恢复、重放和 rolling upgrade 都可能与实时流量竞争相同资源。恢复速率、在线流量保护比例和最大 backlog age 必须显式建模。

单位经济性

至少跟踪：

```
cost / GiB source data
cost / million gzip members
cost / vehicle-hour
cost of replay
cost of quarantine and reconciliation
cost of retained evidence
```

性能优化只有在满足关键性质、SLO 和成本边界后才有意义。更高峰值吞吐并不自动优于更稳定、成本更低且恢复能力更强的实现。

16.14 失败与恢复矩阵节选

| Failure | Partial Effect | Decision / Recovery |
|--|----------------------|---|
| Kafka duplicate | 任务可能再次执行 | completion manifest 吸收重复 |
| Download timeout | 对象可能只下载一部分 | 丢弃不完整 buffer；按分类重试 |
| Object version changed | 输入身份不再稳定 | 拒绝；要求 immutable version |
| Corrupt member | 部分成员已处理 | 按合同 quarantine whole object 或显式 partial policy；不得静默完成 |
| Unknown model | 无合法 DBC | quarantine；等待 binding 修复后重放 |
| DBC Registry unavailable | binding 无法验证 | pause / degraded；不回退猜测 |
| Worker crash before manifest | staging 存在，创建请求可能仍在途 | 从 ledger/权威状态对账；未确认无效果前保护 staging，不直接按年龄回收 |
| Manifest committed, offset commit failed | 正式结果存在，恢复位置未确认 | 重投后验证复用；仅按分区连续完成前缀推进 |
| Result Store conflict | 另一执行已提交 | 比较规范化结果摘要与必要 provenance；不等价记 RESULT_CONFLICT，保留证据并裁决 |
| Conditional create timeout | 成功、未生效或在途均可能 | UNKNOWN；同键同意图对账，not-found 不结束责任 |
| Offset 101 completed before 100 | 后续结果已有，前缀有空洞 | 禁止提交 102；有界完成窗口、pause 与升级 |
| Rebalance during work | 旧 worker 仍可能完成 | epoch fencing 拒绝旧回调推进新 assignment；副作用由权威提交协议吸收 |
| Output slowdown | output queue 增长 | 背压到 CPU，再到 fetch/Kafka pause |
| Memory pressure | byte permits 接近上限 | 停止接纳；不得继续扩大队列 |

16.15 关键可观测性

```
files_discovered / accepted / committed / quarantined
source_bytes_inflight
active_files
members_processed_rate
fetch / framing / decode / commit service time
per-stage queue depth and queue age
CPU worker utilization and steal rate
dbc_cache_hit / miss / unknown / version_mismatch
retry amplification ratio
staging_age and orphan_staging_count
completion_conflict_rate
end_to_end_lag
recovery_backlog_age
cost_per_GiB / cost_per_vehicle_hour
```

指标应使用稳定业务身份关联 source、run、pipeline version、DBC version 和 completion manifest；避免把高基数原始标签无控制地写入所有指标系统。

16.16 验证计划

| Property | Verification |
|-------------|--|
| 单一正式结果 | 重复投递、commit/ACK crash、rebalance、并发执行测试 |
| Partial 不可见 | 在每个阶段注入 crash，验证无 completion manifest 时下游不可见 |
| DBC 不猜测 | unknown model、registry timeout、version mismatch 负向测试 |
| 资源有界 | 2× 峰值、持续下游变慢、超大合法文件和混合大小压力测试 |
| 背压闭合 | 人为降低 output/DBC/object store，验证压力传播和内存稳定 |
| 恢复能力 | 积压回放、节点 N-1、进程反复崩溃和 orphan staging 回收 |
| 容量模型 | 代表性语料端到端基准，比较预测与实际误差 |
| 版本追踪 | 对任意结果追溯 source、pipeline、binding、DBC 和 config digest |

16.16.1 反例轨迹与预期 Oracle

以下为待实现的验证场景，不是已执行结果；项目需保存实际轨迹与 Evidence ID。

| 场景 | 最小轨迹 | 预期 Oracle |
|--------------|--------------------------|----------------------------|
| 越过未决 offset | 收到 100、101；101 完成，100 未决 | 提交位置不超过 100；不得提交 102 |
| offset 数值有空洞 | 实际投递 100、104；100 完成 | 可推进到 104；不要求虚构的 101–103 完成 |

| 场景 | 最小轨迹 | 预期 Oracle |
|-----------------|--|---|
| 迟到写成功 | create 超时；read not-found；原 create 随后成功 | 保持未知直至权威确认；不换键、不提前 GC |
| 发送后立即崩溃 | intent 已持久化；create 已发送；尚未处理返回 | 重启恢复未决 intent；不能按“无返回”判定无效果 |
| 同版本不同结果 | 同 OperationIdentity，两个规范化摘要不同；冲突方随后重启 | RESULT_CONFLICT 保留；重放不能用已有 manifest 快路径抹除冲突 |
| 旧 assignment 回调 | epoch E1 被撤销，E2 接管，E1 worker 返回 | 不更新 E2 完成窗口、不提交 E1 位置 |
| 责任接收方重启 | durable handoff 已确认，接收方崩溃 | 重启可枚举并恢复责任；未确认业务处置不推进 offset |

16.17 Rust 与 C++23 的正确决策位置

两者都能实现该流水线。决策应基于：

- 现有 DBC/压缩/解析库与复用成本；
- 团队熟练度、招聘和 on-call 能力；
- 内存安全与 unsafe/FFI 边界；
- profiling、sanitizer、fuzzing、构建和供应链能力；
- CPU 性能、编译时间和部署目标；
- 长期维护与迁移成本。

推荐用同一代表性语料实现窄范围原型，比较：吞吐、p99、RSS、CPU、崩溃注入、开发复杂度和可观测性，而不是用语言声誉替代证据。

16.18 Change Envelope 与示例裁决

首次生产暴露或重大语义变更应定义受控边界：

| |
|---|
| <pre>population: selected vehicle models / tenants / object prefixes exposure: max files, max source bytes, max duration, max concurrency preconditions: required evidence IDs, DBC snapshot, artifact/config digests success criteria: correctness, resource, latency, recovery and cost thresholds stop conditions: duplicate formal result</pre> |
|---|

字面文本 · continued

```

DBC digest mismatch
unbounded queue / RSS growth
queue age or retry amplification beyond threshold
missing provenance or evidence binding

recovery:
  stop admission, drain, quarantine, reconcile, forward repair

```

建议先使用 shadow 或非权威输出验证 framing、DBC、lineage 和容量；样本必须包含大型对象、热点车型、损坏输入和依赖抖动，而不是只选最干净流量。

以下是假设已完成对应证据与授权后的裁决写法，不表示本案例真的通过 Gate；未完成的阻断项仍应 UNKNOWN/NO_GO。Gate 6 的条件只能是扩大暴露前的条件，不能作为当前暴露尚缺关键恢复能力的借口。

```

Gate 0-5: GO
Gate 6: CONDITIONAL_GO
  scope: verified node specification and <= 2x measured peak workload
  precondition: current limited exposure has verified recovery capacity
  condition: complete N-1 catch-up benchmark before expanding beyond that scope
  expiry: 2026-10-15

Gate 7: GO
Gate 8: GO for the declared corpus and failure profile
  unknown exotic header variants: QUARANTINE

Gate 9: GO
  exposure: 5% of object bytes
  automatic abort: any hard-property violation or memory-bound breach

```

这类裁决明确区分已证明范围、剩余未知、临时条件和撤销触发，比“整体看起来可以上线”更可审计。

17. Production Readiness Checklist

17.1 问题与语义

- ☐ Problem、Goals、Non-goals 和成功标准可检验；
- ☐ 风险等级与失败后果有依据；
- ☐ Safety、Liveness、Durability、SLO 和 Assumption 已分类；
- ☐ 关键性质具有身份、作用域、时间语义、Owner 和违反后果；
- ☐ 性质冲突和允许降级已预定义；
- ☐ 所有阻断级 UNKNOWN 已解决或明确拒绝放行。

17.2 状态与边界

- ☐ 权威状态和稳定身份明确；
- ☐ Mutation Authority、Technical Owner、Operational Owner 明确；
- ☐ 缓存、派生状态和副本不会形成隐式多权威；
- ☐ Boundary Contract 覆盖成功、错误、重复、乱序、超时和容量假设；
- ☐ 上游 Guarantee 满足下游 Assumption；
- ☐ 配置、策略、模型和参考数据按语义版本化。

17.3 失败与恢复

- ☐ 部分成功、结果未知和重复执行有明确收敛；
- ☐ ACK 与业务完成语义一致；
- ☐ 幂等身份稳定，原子提交或去重机制明确；
- ☐ Retry 只用于分类后的可重试错误，并有共享预算；
- ☐ Normal、Degraded、Recovery、Quarantine、Emergency 模式明确；
- ☐ 回滚、forward-recovery、reconciliation 和灾难恢复可执行；
- ☐ RPO、RTO 和恢复积压目标经过演练或证据支持。

17.4 资源、性能与成本

- ☐ 所有可增长资源有硬上限；
- ☐ 队列容量单位、满载行为、背压和公平性明确；
- ☐ 峰值、N-1、扩容延迟和恢复容量经过计算；
- ☐ retry amplification 和控制回路相互作用经过评审；
- ☐ 端到端基准使用代表性语料和目标环境；
- ☐ 单位经济性至少达到 C1 粗估；
- ☐ 新增复杂度具有 Owner、必要性和回收条件。

17.5 安全、运维与演进

- ☐ Trust Boundary、最小权限和高风险变更权威明确；
- ☐ secret、证书、artifact 和 dependency provenance 可追踪；
- ☐ Schema / protocol 兼容与混合版本行为经过验证；
- ☐ deploy、drain、shutdown、upgrade、migration、replay 有 Runbook；
- ☐ break-glass 有范围、时限、审计与补偿复审；
- ☐ 告警对应明确行动，而不是仅通知；
- ☐ Operational Owner 已接受 on-call 和恢复责任。

17.6 验证与证据

- ☐ 每项关键性质映射到机制、验证和证据；
- ☐ 证据绑定当前制品、配置、环境、语料和故障档案；
- ☐ 证据覆盖、限制、状态和失效规则明确；
- ☐ Assurance Plane 变更本身受保护；
- ☐ 例外和风险接受有到期时间；
- ☐ Change Envelope 定义暴露、成功、停止和恢复条件；
- ☐ 运行漂移能够触发相关结论重评，保留适用反证与未决 UNKNOWN。

18. 最小 19 问设计清单

在时间有限的评审中，至少回答以下 19 个问题：

1. 系统解决什么具体问题，什么明确不做？
2. 成功如何衡量，失败会造成什么后果？
3. 真实工作负载、尾部分布、突发和增长是什么？
4. 技术、组织、经济、安全、合规和时间约束是什么？
5. 哪些关键假设尚未证明，何时会失效？
6. 哪些 Safety、Liveness、Durability 和 SLO 必须成立？
7. 关键性质冲突时，哪个优先，谁裁决？
8. 每份权威状态的身份、存储、Mutation Authority 和 Owner 是谁？
9. 关键边界的 Assumption、Guarantee 和违反行为是什么？
10. 重复、乱序、超时、部分成功和 crash 时会发生什么？
11. 业务完成点、ACK 顺序、幂等身份和冲突语义是什么？
12. 一致性选择如何由业务性质推导，而不是由组件默认决定？
13. 所有队列、缓存、任务、连接、重试和临时状态是否有界？
14. 背压能否传播到可控上游，持续过载时如何处置？
15. 峰值、故障降容、积压恢复和单位成本是否可计算？
16. 正常、降级、恢复、隔离和紧急模式分别允许什么？
17. 系统如何发布、排空、升级、迁移、回滚、重放和灾难恢复？
18. 每项关键主张用什么机制、验证和当前证据支持？
19. 假设、证据或环境失效后，谁按 ASSURANCE 重评结论、保留反证与未知并触发复审？

任何高后果问题回答为“后续再看”，都应被视为显式风险，而不是默认通过。

19. 团队成熟度与采用路线

| 等级 | 典型状态 | 下一步 |
|-------------------------|---------------------|------------------------------------|
| L0 — Component-driven | 先选技术，缺少问题和状态模型 | 强制 Decision Brief 与 Non-goals |
| L1 — Functional | 正常路径可用，失败与 Owner 模糊 | 建立 Property、State、Failure Register |
| L2 — Bounded | 资源有界，有基本恢复 | 增加容量、运行模式和边界契约 |
| L3 — Operable | SLO、Owner、发布和恢复明确 | 建立 Evidence Matrix 与生产门禁 |
| L4 — Evidence-driven | 关键主张有可追踪证据 | 关联假设、环境漂移和证据时效 |
| L5 — Adaptive Assurance | 结论随漂移自动失效，事故形成保证增量 | 持续防止 Goodhart 化并回收复杂度 |

推荐采用顺序：

先统一术语和最小模板

- > 在一个高价值项目试点
- > 用真实评审和事故校准
- > 将高确定性机械规则编码进 CI
- > 保留人工审查处理未编码属性
- > 建立成本、证据与漂移平台能力
- > 定期删除低价值门禁和长期例外

不要试图通过一次性发布完整标准让团队“直接达到 L4”。流程厚度必须与现有能力和风险匹配，但关键状态、身份和认识边界必须从第一天保持诚实。

20. 文档与标准维护

20.1 版本规则

- 原则、术语或门禁语义发生不兼容变化：提升 Major；
- 新增兼容的模板、场景或要求：提升 Minor；
- 文字澄清、排版和不改变语义的修正：提升 Patch；
- 本文与方法论文档使用统一 suite version，但允许独立 patch。

20.2 复审触发

以下任一事件触发复审：

- 重大事故或近失事件；
- 关键假设失效；
- 新合规、安全或业务硬约束；

- 工作负载、成本或故障域显著变化；
- 新的分布式边界、权威状态或特权主体；
- 现有门禁频繁被例外绕过；
- 证据与生产漂移长期无法关联；
- 至少每 12 个月一次基线复审，高风险系统应更短。

20.3 单一事实来源

本 Markdown 文件是执行参考的唯一权威源。任何表单、门户、CI 配置或导出视图必须能够追溯到本文版本；不得维护无法同步的平行“精简版标准”。实际项目可以实例化模板，但不得静默改变规范语义。

最终操作原则

评审的目的不是证明设计者足够聪明，也不是让所有检查变绿，而是在代码和流量扩大之前，尽早暴露状态、失败、资源、权限、恢复和证据链中的高代价错误。最好的门禁会阻止不可接受风险，同时让低风险、可逆决策保持快速。

20.4 方法论与操作手册在套系中的边界

ESD-METHOD-001

定义：为什么这样设计、核心术语是什么、能够证明到什么范围

ESD-REFERENCE-001

执行：按什么阶段工作、提交什么制品、如何评审、裁决和持续维护

上述两篇不独占全套规范权威；风险、Gate 与例外服从 GOV，Claim、Assumption 与 Evidence 服从 ASSURANCE，运行与恢复服从 OPS，关系由 SUITE §3.1 控制。领域 Profile 可依 GOV 授权收紧要求或裁剪执行深度，不得静默改写权威定义。

20.5 结论

高质量 Architecture Review 的目标不是批准一张架构图，而是阻止未经定义的状态、失败、权限、资源和未知进入生产。一个完成的设计应留下可追踪的推理链：

Problem

- > Constraints / Assumptions / Risk
- > Critical Properties
- > State / Identity / Ownership
- > Failure / Consistency / Recovery
- > Architecture / Resource Control
- > Capacity / Economics / Evolution
- > Verification / Evidence
- > Controlled Change
- > Runtime Learning

当团队能够在任意环节诚实回答“我们知道什么、为什么相信、覆盖到哪里、什么时候失效”，系统设计才从经验与审美上升为可治理的工程实践。

系统设计与工程保证治理及符合性规范

适用范围、风险分级、角色、门禁、例外与可审计符合性

| 文档属性 | 内容 |
|------|---------------------------------------|
| 文档编号 | ESD-GOV-001 |
| 版本 | 2.0.0-draft.1 (基于 1.0.0) |
| 状态 | DRAFT / NORMATIVE CONTROL；未获发布批准 |
| 规范角色 | 定义谁必须遵循哪些要求、谁有权裁决、如何例外及如何声明符合 |
| 语义基础 | ESD-METHOD-001 《优秀系统设计与工程保证方法论》 |
| 执行参考 | ESD-REFERENCE-001 《优秀系统设计：从约束、不变量到证据》 |
| 证据规范 | ESD-ASSURANCE-001 《工程保证论证、证据与裁决规范》 |
| 运行标准 | ESD-OPS-001 《生产就绪与运行韧性标准》 |
| 权威格式 | 本 Markdown 文件；派生格式不构成独立规范 |

规范目的

本文把“优秀系统设计”从个人能力和建议性实践提升为可执行的组织控制：明确适用对象、风险等级、最低制品、角色分工、Gate 权限、例外边界、审计记录和符合性声明。它不要求所有项目使用同等厚度的流程，但要求所有裁剪、未知和偏离都可见、可负责、可到期、可复审。

摘要

高质量工程规范若只有原则而没有治理，会产生两类失败：

1. 团队把标准当作自愿阅读材料，高风险变更仍通过口头共识进入生产；
2. 组织把标准变成表单和审批数量，以形式完成代替状态、失败和证据闭合。

本规范采用风险匹配的控制模型：

- Change Classification
- > Risk Tier
 - > Applicable Requirements
 - > Required Roles and Artifacts
 - > Gate Decision
 - > Exception / Residual Risk
 - > Conformance Statement
 - > Review and Expiry

治理的目标不是把所有决定集中到委员会，而是确保：

- 高后果决策有适当独立性和证据；
- 低风险、可逆决策保持快速；
- 谁拥有事实、风险、运行和最终批准始终明确；

- UNKNOWN 不会因为排期压力自动转化为 GO；
- 例外不会成为永久架构；
- 符合性结论可以追溯到具体要求、制品和裁决主体。

1. 适用范围

1.1 适用对象

本规范适用于以下组织活动：

- 新建生产系统、平台、服务、数据管道和控制面；
- 对既有系统进行可能改变关键语义、状态权威或故障域的重大变更；
- 引入新的持久状态、外部副作用、跨团队契约或高权限主体；
- 改变 SLO、RPO、RTO、容量边界、租户隔离或合规范围；
- 进行不可逆 Schema、数据格式、身份、权限或迁移变更；
- 发布高风险 AI/Agent、自动控制或自动决策能力；
- 对核心系统执行生产准入、重大恢复演练或灾难恢复变更；
- 变更工程保证规则、门禁、证据绑定或例外逻辑本身。

1.2 默认不适用

以下活动通常不要求完整治理流程，但仍受本规范中的信息安全、退出条件和不可逆决策约束：

- 不接触真实敏感数据、无生产依赖、可随时删除的一次性实验；
- 纯排版、注释、无语义文档修正；
- 已被低风险变更策略明确覆盖的局部实现调整；
- 不改变外部契约、状态、权限、资源边界或运行行为的机械重构。

“不适用”必须是经过范围判断的结论，不得由变更提交者单方面使用标签绕过评审。

1.3 非目标

本规范不：

- 指定具体技术栈；
- 要求所有变更经过中央架构委员会；
- 允许审批替代 Owner 责任；
- 把文档篇幅作为符合性指标；
- 以全部风险归零为生产前提；
- 承诺未建模风险不存在。

2. 核心治理原则

GOV-REQ-001：风险决定控制深度

每个受管对象必须先进行风险分类，再确定流程和证据厚度。项目规模、团队级别、QPS 或技术新颖度都不能单独替代风险判断。

风险至少考虑：

- consequence
- blast radius
- uncertainty
- irreversibility
- exposure duration
- recoverability
- privilege
- regulatory / safety impact

GOV-REQ-002：裁剪必须显式

任何对默认要求的省略必须记录：

- 被省略的要求；
- 省略理由；
- 适用风险等级；
- 补偿控制；
- 风险 Owner；
- 到期时间或复审触发。

无记录的省略视为未完成，而不是合理裁剪。

GOV-REQ-003：所有关键责任必须具名

任何关键状态、性质、运行能力、例外和 Gate 决策必须有具名角色或稳定团队承担责任。Platform Team、Architecture、业务方 等泛化主体不足以构成可审计 Owner，除非能映射到明确 on-call 或责任名册。

GOV-REQ-004：职责分离与风险匹配

高风险变更中，方案提出者、证据评价者和最终批准者不得由同一非受控主体独立兼任。独立性要求随风险上升，而不是机械要求所有评审都由陌生团队完成。

GOV-REQ-005：未知必须保留

证据不足、适用范围不清、关键假设未验证或控制未执行时，状态必须为 UNKNOWN。高后果 UNKNOWN 默认阻断生产，不得通过措辞降级为“后续观察”。

GOV-REQ-006：例外是有期限的受控状态

Exception、Waiver 和 break-glass 必须拥有范围、期限、Owner、补偿控制、监测、退出路径和复审。任何无期限例外都应被视为事实上的规范变更或架构债务，不得长期停留在临时状态。

GOV-REQ-007：符合性必须可追溯

“符合本标准”的结论必须映射到：

```
suite/document versions
applicable requirements
project artifacts
assurance evidence
exceptions
blocking unknowns
decision authority
review trigger
```

GOV-REQ-008：治理系统自身受治理

Gate 规则、审批权限、例外逻辑、风险分类器和自动化符合性检查的变更必须经过不低于其所影响对象的控制强度。

3. 受管对象与变更分类

3.1 受管对象

本规范中的受管对象可以是：

- 完整系统；
- 服务、库、数据管道、控制器或平台能力；
- 外部 API、事件、Schema 或持久格式；
- 基础设施和运行环境；
- 权限、策略、密钥、Admission 或自动化控制；
- 迁移、回放、恢复或灾难恢复方案；
- Assurance Plane 工具、策略和证据流水线；
- 单次重大变更或发布批次。

每次裁决必须明确对象身份，避免把局部证据用于未声明的系统整体。

3.2 变更类型

| 类型 | 描述 | 典型控制 |
|--------------------------|-----------------------------------|-------------------------|
| NEW_SYSTEM | 新建生产能力或新权威状态 | 完整风险分类和设计流程 |
| MAJOR_SEMANTIC_CHANGE | 改变业务完成、身份、一致性、权限或恢复语义 | 重新评审相关性质与证据 |
| MAJOR_OPERATIONAL_CHANGE | 改变部署、故障域、容量、SLO、RPO/RTO 或 on-call | Production Readiness 复审 |
| DATA_MIGRATION | 改变持久数据、Schema、编码或权威位置 | 迁移、验证、回退/前向修复计划 |
| DEPENDENCY_CHANGE | 关键依赖升级、替换或语义变化 | 边界契约和组合保证复审 |
| SECURITY_CHANGE | 权限、身份、凭据、信任边界变化 | 安全 Review 与最小权限验证 |
| ASSURANCE_CHANGE | 门禁、测试、证据、策略或例外逻辑变化 | 独立评审，防止弱化控制 |
| ROUTINE_CHANGE | 已在批准变更类别和已验证范围内 | 自动或轻量门禁 |
| EMERGENCY_CHANGE | 为控制当前事故必须立即执行 | break-glass + 事后补偿复审 |
| EXPERIMENT | 在限定暴露内验证假设 | Change Envelope 与退出条件 |

3.3 重大变更触发器

出现以下任一情况，默认不得作为普通例行变更处理：

- 新增或移动业务事实的权威来源；
- 改变稳定身份、幂等键或提交点；
- 新增跨区域、跨租户或跨团队写路径；
- 引入无法原子协调的双写；
- 改变 ACK、可见性或失败恢复语义；
- 放宽权限、例外、门禁或审计；
- 改变持久格式且旧消费者可能共存；
- 引入新的自动扩缩容、重试或反馈控制回路；
- 资源上界、峰值或爆炸半径显著扩大；
- 证据不再能代表新环境或工作负载。

4. 风险分级

4.1 风险等级

| 等级 | 定义 | 典型示例 |
|-------------------------|--------------------------|-----------------------|
| R0 – Exploration | 无生产依赖、可删除、无不可逆高后果副作用 | 本地原型、脱敏数据实验 |
| R1 – Bounded Production | 影响有限且可隔离，恢复成本中等 | 内部工具、单团队辅助服务 |
| R2 – Core Production | 影响核心业务、关键数据或跨团队依赖 | 数据平台、交易链路、关键控制面 |
| R3 – High Assurance | 涉及资金、安全、合规、人身风险或大规模不可逆损害 | 支付结算、安全策略、车辆控制、关键凭据系统 |

4.2 风险评分维度

风险评估必须至少覆盖：

| 维度 | 问题 |
|------|--------------------------|
| 后果 | 错误会导致什么业务、数据、安全或法律影响？ |
| 爆炸半径 | 单请求、单租户、单区域还是全局？ |
| 可逆性 | 能回滚、重算、补偿还是不可恢复？ |
| 不确定性 | 输入、依赖、模型、团队经验和运行环境有多未知？ |
| 暴露 | 流量比例、持续时间、权限范围和数据范围多大？ |
| 检测 | 错误能否在造成损害前被发现？ |
| 恢复 | 恢复需要多久，是否有权威来源和足够容量？ |
| 特权 | 是否可修改身份、策略、资金、生产状态或安全边界？ |

GOV-REQ-009：风险等级必须有依据

风险等级不能只记录一个标签。必须包含简要理由、最高后果场景、关键未知和批准者。

GOV-REQ-010：等级下调必须审查

从更高风险等级下调必须由非原提交者的授权 Reviewer 接受理由。不能因为流程成本高而下调风险。

GOV-REQ-011：风险可随生命周期变化

实验一旦接入真实数据、生产依赖或持续用户流量，必须重新分类。历史 R0 身份不能永久豁免后续生产使用。

5. 角色与责任

5.1 核心角色

| 角色 | 核心责任 |
|-------------------------|------------------------------|
| System Owner | 对系统整体业务结果、生命周期和风险接受负责 |
| Design Author | 维护设计推理、制品和开放问题 |
| Semantic Owner | 定义业务事实、身份、契约、一致性和兼容语义 |
| Technical Owner | 对实现、技术边界、质量和演进负责 |
| Operational Owner | 对 SLO、容量、发布、恢复、on-call 和事故负责 |
| Security Owner/Reviewer | 对信任边界、权限、秘密、审计和安全风险负责 |
| Assurance Reviewer | 评价论证、证据质量、覆盖、失效和 UNKNOWN |
| Architecture Reviewer | 挑战状态、边界、失败、资源、成本和替代方案 |
| Decision Authority | 对设计 Gate 作出正式裁决 |
| Release Authority | 基于已批准设计与当前证据作生产放行裁决 |
| Exception Authority | 接受特定偏离和剩余风险，并承担期限内责任 |
| Standard Owner | 维护规范语义、版本和解释记录 |

一个人可以承担多个角色，但必须根据风险审查职责冲突。

5.2 RACI 最低要求

每个 R1+ 对象至少应建立：

| 决策/能力 | Responsible | Accountable | Consulted | Informed |
|---------|--------------------------|--------------------|------------------------|--------------------|
| 状态语义与身份 | Semantic Owner | System Owner | Reviewer | Consumers |
| 实现与质量 | Technical Owner | System Owner | Assurance/Security | Operational Owner |
| SLO 与运行 | Operational Owner | System Owner | SRE/Platform | Stakeholders |
| 生产放行 | Release Authority | System Owner 或授权主体 | Assurance/Ops/Security | Stakeholders |
| 例外接受 | Risk/Exception Authority | 明确高层责任主体 | Relevant Owners | Audit/Stakeholders |
| 事故响应 | Incident Commander | Operational Owner | System/Security | Stakeholders |

GOV-REQ-012：技术 Owner 与运营 Owner 不得缺失

任何生产关键组件必须同时有：

- 对语义、接口和实现负责的技术 Owner；
- 对 SLO、告警、恢复和 on-call 负责的运营 Owner。

两者可以是同一团队，但责任必须分别明确。

GOV-REQ-013：最终批准者必须具备风险授权

最终批准权不能仅由职级、代码所有权或会议主持权推导。必须明确授权范围，例如：

| |
|-------------------|
| 可批准 R1 |
| 可批准 R2 的非安全变更 |
| 不可批准涉及资金或全局身份的 R3 |

GOV-REQ-014：Reviewer 必须可挑战而非仅确认

评审者应有权限要求补充信息、将结论置为 UNKNOWN、阻断 Gate 或升级风险。只负责“参加会议”的 Reviewer 不构成有效控制。

6. 职责分离

6.1 最低模型

| | |
|--------------------|---|
| Proposer | proposes design/change |
| Evaluator | challenges semantics and evidence |
| Decision Authority | accepts or rejects residual risk |
| Operator | executes and observes production change |

6.2 风险分级要求

| 风险 | 最低独立性 |
|----|---|
| R0 | 提交者自评；明确退出条件 |
| R1 | 至少一名非作者 Reviewer；Owner 接受运行责任 |
| R2 | 设计、运行和证据至少由两个责任主体交叉评审；最终批准不由单一作者完成 |
| R3 | Proposer、Assurance Evaluator、Final Approver 必须有实质独立性；安全/合规按需要独立签署 |

GOV-REQ-015：保证面变更需要独立性

任何可能让失败被错误解释为通过的变更，例如：

- 将 fail-closed 改为 fail-open；
- 跳过某类验证；
- 放宽证据时效；
- 改变 Exception 逻辑；
- 降低权限审计；

必须至少按其影响的最高风险对象进行独立评审。

7. 最低工程制品

7.1 按风险等级

| 制品 | R0 | R1 | R2 | R3 |
|-------------------------------------|------|------|------|-------------|
| Problem / Non-goals / Exit Criteria | 必须 | 必须 | 必须 | 必须 |
| 风险分类与理由 | 必须 | 必须 | 必须 | 必须 |
| Constraint / Assumption Register | 简化 | 必须 | 必须 | 必须 |
| Critical Property Register | 关键项 | 必须 | 必须 | 必须 |
| State / Identity / Ownership | 不可逆项 | 必须 | 必须 | 必须 |
| Failure / Recovery Matrix | 可选 | 核心路径 | 完整 | 完整 + 独立挑战 |
| Boundary Contract | 可选 | 关键边界 | 完整 | 完整 + 组合保证 |
| Resource / Queue Contract | 硬上限 | 关键资源 | 完整 | 完整+ 故障降容 |
| Capacity / Economic Model | 粗估 | 粗估 | 定量 | 定量 + 敏感性分析 |
| Threat Model | 数据边界 | 基础 | 完整 | 完整 + 独立安全审查 |
| Verification & Evidence Matrix | 最小 | 基础 | 完整 | 完整 + 独立证据评价 |
| Change Envelope | 首次暴露 | 重大变更 | 必须 | 必须 + 自动停止 |
| Production Readiness Decision | 不适用 | 简化 | 必须 | 必须 + 多方签署 |
| Exception Register | 如有 | 如有 | 必须受控 | 必须严格受控 |
| Runbook / Recovery Drill | 不适用 | 核心故障 | 必须 | 必须+ 定期演练 |

GOV-REQ-016：制品必须与对象和版本绑定

项目制品必须关联：

- 系统或变更 ID；
- 当前代码/制品版本；
- 配置与关键依赖版本；
- 风险等级；
- Owner；
- 最近评审时间；
- 复审触发。

“架构文档存在”不足以证明它描述当前系统。

GOV-REQ-017：模板可裁剪，语义不可丢失

团队可以采用更适合工具链的格式，但必须保留规范要求的语义字段和稳定 ID。

8. Gate 治理

8.1 Gate 的目的

Gate 是风险决策点，不是会议日历。每个 Gate 必须明确：

| |
|---------------------|
| scope |
| entry criteria |
| required artifacts |
| blocking conditions |
| decision states |
| decision authority |
| evidence retained |
| review trigger |

8.2 标准 Gate

默认采用 ESD-REFERENCE-001 的 Gate 0-9：

| Gate | 核心裁决 |
|------|-----------------------------------|
| 0 | 问题、价值和风险是否成立 |
| 1 | 工作负载、约束、SLO 和假设是否可用于决策 |
| 2 | 关键性质和优先级是否完整 |
| 3 | 状态、身份、权威和边界是否闭合 |
| 4 | 失败、一致性、提交和恢复语义是否闭合 |
| 5 | 架构是否维持上述语义 |
| 6 | 系统是否存在稳定、有界、经济可承受的运行区 |
| 7 | 契约、安全、演进与 Day-2 操作是否就绪 |
| 8 | 证据是否达到当前决策所需质量 |
| 9 | 当前制品是否可在声明的 Change Envelope 内进入生产 |

团队可以合并会议，但不得合并掉裁决语义。

8.3 决策状态

| 状态 | 治理含义 |
|----------------|--------------------------|
| GO | 强制要求满足，证据当前有效，剩余风险在授权范围内 |
| CONDITIONAL_GO | 可在明确范围、期限、监测、停止和补偿条件下放行 |
| NO_GO | 已知违反关键要求或风险超出授权阈值 |
| UNKNOWN | 证据不足、过期、错绑或关键假设未验证 |
| NOT_APPLICABLE | 经论证和 Reviewer 接受后不适用 |

GOV-REQ-018：CONDITIONAL_GO 必须可执行

条件放行必须包含：

| |
|------------------------|
| scope |
| conditions |
| owner |
| deadline |
| observation signals |
| abort criteria |
| recovery path |
| final closure decision |

“上线后观察”“尽快补测试”不是有效条件。

GOV-REQ-019：高后果 UNKNOWN 默认阻断

对 Safety、权限、数据完整性、不可逆迁移、资金或灾难恢复等高后果性质，UNKNOWN 默认等同于不具备放行资格，但不应被改写为 FAIL。只有授权的 Exception 可以暂时改变阻断行为。

GOV-REQ-020：Gate 决策必须保存反对意见

如果 Reviewer 对关键结论提出未解决异议，最终记录必须保留：

- 异议内容；
- 支持证据；
- 决策者为何接受或拒绝；
- 对应剩余风险；
- 复审触发。

9. 例外、豁免与风险接受

9.1 三类偏离

| 类型 | 适用场景 |
|-------------|-------------------------|
| DEVIATION | 采用不同机制但达到同等或更强性质，不是降低要求 |
| EXCEPTION | 临时无法满足要求，以补偿控制和期限接受风险 |
| BREAK_GLASS | 当前事故中为阻止更大损害而紧急绕过正常控制 |

WAIVER 若使用，必须明确属于上述哪一类，避免成为语义模糊的永久豁免。

9.2 Exception 最低字段

```
exception_id: "EX-..."
requirement_id: "GOV/ASSUR/OPS-REQ-..."
system_or_change: "..."
reason: "..."
risk_tier: "R..."
scope: "..."
risk_consequence: "..."
compensating_controls: []
monitoring: []
owner: "..."
exception_authority: "..."
start_at: "..."
expires_at: "..."
exit_plan: "..."
abort_conditions: []
review_triggers: []
status: "OPEN|CLOSED|EXPIRED|REVOKED"
```

GOV-REQ-021：Exception 必须绑定具体要求

不得批准“整体架构例外”而不说明偏离哪些要求。每项偏离必须可追溯到 Requirement ID 或明确规范段落。

GOV-REQ-022：Exception 必须到期

所有 Exception 必须有确定到期时间。若到期前未关闭或续期，状态自动变为 EXPIRED，相关生产许可必须重新裁决。

GOV-REQ-023：补偿控制必须降低同类风险

“加强监控”只有在能够及时检测、明确响应并限制损害时才是补偿控制。与风险无直接关系的流程或文档不能作为补偿。

GOV-REQ-024：例外不能静默继承

系统复制、区域扩展、团队迁移或版本升级时，既有 Exception 不自动适用。必须重新确认范围和风险。

9.3 Break-glass

break-glass 至少要求：

- 明确触发者和授权者；
- 最小必要权限；
- 时间限制；
- 全量审计；
- 实时或事后通知；
- 可撤销性；
- 事故结束后的补偿审查；
- 对被绕过控制的恢复和证据补齐。

GOV-REQ-025：break-glass 不得成为常规路径

重复使用同一 break-glass 表明正常系统或流程存在结构性缺陷，必须触发标准、架构或权限修复。

10. 符合性模型

10.1 要求级状态

每项要求分别记录满足事实 satisfaction_status 与授权处置 governance_disposition。前者只按事实与 Evidence 判断：

| 状态 | 含义 |
|----------------|------------------------|
| CONFORMANT | 当前对象满足要求，且证据足以支持结论 |
| NONCONFORMANT | 已知不满足要求；即使有有效例外也不改变该事实 |
| UNKNOWN | 无法判断是否满足 |
| NOT_APPLICABLE | 有论证并经接受后不适用 |

授权处置另记 STANDARD 或 AUTHORIZED_EXCEPTION，后者须引用 §9 中具备有效授权、处于 OPEN 且未到期/撤销的 Exception。申请中、到期或不在本对象范围内的例外不能按有效授权处理。Exception 可改变指定范围内的阻断决策，但不能改变 Claim 支持状态或要求满足事实。

旧的要求级 **CONDITIONALLY_CONFORMANT** 不得直接迁移为“满足要求”；必须回看原证据，分别填写满足事实与授权处置，事实不明时为 **UNKNOWN**。该标签仅保留在 §10.2 的对象级治理结论中。

10.2 对象级状态

对象级符合性按当前评估范围内的适用义务汇总，不按某个 **Gate** 的阻断级别筛选义务。以下规则依次判定，以首次命中的结论为准；强制义务指 **MUST/MUST NOT**，有效不适用项按 §10.1 排除。

| 优先顺序 | 条件 | 对象级结论 |
|------|---|---------------------------------|
| 1 | 存在未获有效且适用的 Exception 授权的、适用强制义务的已知不满足 | NONCONFORMANT |
| 2 | 不存在上述已知不满足，但存在未获有效且适用的 Exception 授权的强制义务未知，或适用范围无法确定 | UNKNOWN |
| 3 | 存在缺口，但所有剩余缺口均由有效且适用的 Exception 覆盖，且相关条件可执行 | CONDITIONALLY_CONFORMANT |
| 4 | 没有上述缺口，所有适用义务及必要的偏离审查均已满足 | CONFORMANT |

SHOULD 的允许偏离本身不必判为不符合；但规范要求的理由、风险分析或审查记录应作为独立义务，按其实际强度评估。若这些记录被规定为强制义务，其不满足或未知同样进入上述汇总，不能随建议项一并略过。**MAY** 本身不产生强制缺口。

Exception 的有效性与适用性按 §9 和 §10.1 判断；未确认授权范围、已过期或条件不可执行的例外，不能消除未获有效覆盖的缺口。不可豁免的法律、安全硬边界不得通过 **Exception** 越权接受。底层不满足与未知不得被 **Exception** 改写；已知违反与未知并存时都须列出，不能用单一总状态隐藏任一项。

某项缺口是否阻断特定 **Gate**，另按 §8 与 §10.4.1 的 **Gate** 规则、风险和授权判断。**blocking: false** 不会把未满足的强制义务变为满足；对象级符合性结论也不自动等于 **Gate** 放行。

以下为说明性汇总检查，假定适用范围明确、其他义务均已满足；不代表真实项目评估或生产放行：

| 输入 | 对象级结论 | 必须保留的事实 |
|-----------------------------------|---------------------------------|-----------------------|
| 非阻断 MUST 已知不满足，无有效例外 | NONCONFORMANT | 该义务仍不满足，不能因非阻断而忽略 |
| 非阻断 MUST 未知，无有效例外 | UNKNOWN | 该义务仍未知，不能默认为满足 |
| 强制义务存在不满足或未知，所有缺口均有有效且适用的例外，条件可执行 | CONDITIONALLY_CONFORMANT | 各义务原有不满足/未知及例外授权分别保留 |
| 强制义务的已知不满足与未知并存，均无有效例外 | NONCONFORMANT | 已知不满足优先决定总状态，未知项仍完整列出 |

GOV-REQ-026：符合性必须按适用剖面判断

符合性声明必须说明：

- 风险等级；
- 使用的 Profile；
- 适用的文档和版本；
- 被裁剪或不适用的要求；
- 例外；
- 证据包。

GOV-REQ-027：符合性不是产品质量总评

符合本标准只表示该对象按声明范围满足当前工程治理要求，不表示：

- 没有缺陷；
- 满足所有业务目标；
- 对未建模环境绝对安全；
- 未来版本自动符合；
- 第三方依赖全部正确。

10.3 符合性矩阵

以下是说明性填充值，不是已执行评估或真实 Evidence；项目须替换证据、Owner 与审查记录。

表 15 · 字段记录视图（列名与原单元格逐项对应）

| 表 15 · 记录 1 · GOV-REQ-012 | |
|---------------------------|--------------|
| Requirement ID | GOV-REQ-012 |
| Applicable | Yes |
| Satisfaction Status | CONFORMANT |
| Governance Disposition | STANDARD |
| Evidence / Exception | OWN-REG-001 |
| Owner | Team A |
| Review Trigger | Owner change |

| | |
|-----------------------------|------------------|
| 表 15 · 记录 2 · ASSUR-REQ-014 | |
| Requirement ID | ASSUR-REQ-014 |
| Applicable | Yes |
| Satisfaction Status | UNKNOWN |
| Governance Disposition | STANDARD |
| Evidence / Exception | Evidence missing |
| Owner | Team A |
| Review Trigger | Before Gate 9 |

| | |
|---------------------------|--|
| 表 15 · 记录 3 · OPS-REQ-023 | |
| Requirement ID | OPS-REQ-023 |
| Applicable | Yes |
| Satisfaction Status | CONFORMANT |
| Governance Disposition | STANDARD |
| Evidence / Exception | Shared retry-budget config + exhaustion/fault tests + runtime amplification evidence |
| Owner | Team A |
| Review Trigger | Retry policy / dependency change |

OPS-REQ-023 控制共享重试预算；无状态服务仍可能重试，不能以“无状态”判定不适用。只有证明作用域内不存在重试行为（包括 SDK、代理与上游重试层），并获 **Reviewer** 确认，才可声明该前置条件不存在。引用校验必须核对条款含义、前置条件与证据关联，不能只检查 ID 存在。

10.4 统一控制索引与适用性

10.4.1 单条评估记录

同一份控制索引覆盖编号要求、未编号规范正文、强制模板字段和 **Gate** 阻断条件。每个可独立裁决的子义务一条记录，至少包含：

```
control_id: "Requirement ID 或文档 ID + 稳定段落/字段定位"
source:
  document_id: "..."
  version: "..."
  section_or_requirement: "..."
  field_or_subclause: "..."
  source_commit: "..."
modality: "MUST|MUST_NOT|SHOULD|MAY"
applicability:
  precondition: "..."
  risk_tiers: []
```

| |
|---|
| 字面文本 · continued |
| <pre>lifecycle_or_change_type: "..."
decision: "APPLICABLE NOT_APPLICABLE UNKNOWN"
rationale: "..."
reviewer: "..."
evidence_expectation: "需要什么事实、验证方法、覆盖与 Oracle"
evidence_refs: []
owner: "..."
blocking_rule:
 gate: "..."
 failure_or_unknown: "..."
 rationale: "..."
satisfaction_status: "CONFORMANT NONCONFORMANT UNKNOWN NOT_APPLICABLE"
governance_disposition: "STANDARD AUTHORIZED_EXCEPTION"
exception_id: null
exception_authority: null
review_trigger: "..."</pre> |

强度与阻断级别不是同一维度。MUST/MUST NOT 是义务；SHOULD 允许经审查的有理由偏离，缺少偏离记录不能直接算通过；MAY 本身不构成强制门禁。适用 MUST/MUST NOT 未满足或未知时，默认不得作出无条件符合/放行声明；是否允许受限暴露须按 §8-9 单独授权。高后果未知按 GOV-REQ-019 默认阻断，不可豁免边界不得被授权覆盖。

同一条款可含强制结果与建议实现方式，必须拆分子义务，不得因标题含“必须”就把正文所有建议提升为 MUST，也不得因存在一个“应”就把整条降为 SHOULD。例如 ASSUR-REQ-020 的证据多样性为 SHOULD、无法满足时记录理由为 MUST；OPS-REQ-047 的受控自动化建议与“不得要求事故中手工拼接复杂命令”分别评估。若标题和正文对同一义务确有冲突，登记解释请求并保持 UNKNOWN，不自行选择宽松版本。

10.4.2 编号要求覆盖索引

下表按连续 ID 区间压缩展示完整来源范围，不新增独立要求；实例化时必须展开为逐 ID、逐子义务记录。每项强度与具体前提从链接正文读取；“生产”“阻断级 Claim”“R2/R3”“发生重放/迁移”等条件不能被区间默认值覆盖。Evidence 列说明材料类别，不代替该条款的实际 Oracle。Owner 为责任角色，项目必须映射到具名主体。

表 16 · 字段记录视图（列名与原单元格逐项对应）

| | |
|-------------------------|------------------------------------|
| 表 16 · 记录 1 · 本文 §2 – 7 | |
| 来源 | 本文 §2-7 |
| 要求范围 | GOV-REQ-001-017 |
| 适用主题/前提 | 受管对象的风险、裁剪、责任及制品 |
| Evidence 类别 | 风险/Owner/制品登记、授权与职责分离记录 |
| 责任角色 | Technical Owner / Governance Owner |

表 16 · 记录 2 · 本文 §8 – 9

| | |
|-------------|--------------------------------|
| 来源 | 本文 §8-9 |
| 要求范围 | GOV-REQ-018-025 |
| 适用主题/前提 | Gate、条件放行、偏离或应急 |
| Evidence 类别 | 决策、Exception、补偿控制与期限 |
| 责任角色 | Decision / Exception Authority |

表 16 · 记录 3 · 本文 §10 – 17

| | |
|-------------|-----------------------------|
| 来源 | 本文 §10-17 |
| 要求范围 | GOV-REQ-026-035 |
| 适用主题/前提 | 符合性、评审、发布、复审及审计 |
| Evidence 类别 | 逐项评估、发布绑定、记录与复审 |
| 责任角色 | Assessor / Governance Owner |

表 16 · 记录 4 · ASSURANCE §3 – 6

| | |
|-------------|-----------------------------------|
| 来源 | ASSURANCE §3-6 |
| 要求范围 | ASSUR-REQ-001-015 |
| 适用主题/前提 | 受评 Claim、对象身份与边界假设 |
| Evidence 类别 | Claim/Assumption 契约、身份及保证映射 |
| 责任角色 | Assurance Author / Semantic Owner |

表 16 · 记录 5 · ASSURANCE §7 – 12

| | |
|-------------|----------------------------------|
| 来源 | ASSURANCE §7-12 |
| 要求范围 | ASSUR-REQ-016-026 |
| 适用主题/前提 | 论证、反证、证据质量及状态 |
| Evidence 类别 | Argument、Defeater、Envelope 与结果记录 |
| 责任角色 | Assurance Author / Reviewer |

表 16 · 记录 6 · ASSURANCE §13 – 18

| | |
|-------------|---------------------------|
| 来源 | ASSURANCE §13-18 |
| 要求范围 | ASSUR-REQ-027-039 |
| 适用主题/前提 | 证据失效、Oracle、工具、组合与充分性 |
| Evidence 类别 | 失效传播、测试/证明、覆盖与组合论证 |
| 责任角色 | Evidence Owner / Reviewer |

表 16 · 记录 7 · ASSURANCE §19 – 24

| | |
|-------------|---|
| 来源 | ASSURANCE §19–24 |
| 要求范围 | ASSUR-REQ-040–051 |
| 适用主题/前提 | Claim 聚合、Gate、暴露、漂移和事故 |
| Evidence 类别 | 状态依据、Bundle、暴露边界、Assurance Delta |
| 责任角色 | Reviewer / Decision / Operational Owner |

表 16 · 记录 8 · ASSURANCE §25 – 26

| | |
|-------------|-------------------------------------|
| 来源 | ASSURANCE §25–26 |
| 要求范围 | ASSUR-REQ-052–060 |
| 适用主题/前提 | Assurance Plane 完整性及职责 |
| Evidence 类别 | 错绑/缺失负向检查、审计与独立性记录 |
| 责任角色 | Assurance Platform Owner / Reviewer |

表 16 · 记录 9 · OPS §2 – 4

| | |
|-------------|-------------------------------|
| 来源 | OPS §2–4 |
| 要求范围 | OPS-REQ-001–014 |
| 适用主题/前提 | 生产服务、资源、责任与资格 |
| Evidence 类别 | 服务契约、硬上限、状态及身份记录 |
| 责任角色 | Technical / Operational Owner |

表 16 · 记录 10 · OPS §5 – 7

| | |
|-------------|-----------------------|
| 来源 | OPS §5–7 |
| 要求范围 | OPS-REQ-015–023 |
| 适用主题/前提 | SLO、运行模式、依赖与重试 |
| Evidence 类别 | SLI/SLO、模式转换、故障域与重试验证 |
| 责任角色 | Operational Owner |

表 16 · 记录 11 · OPS §8 – 10

| | |
|-------------|--------------------------|
| 来源 | OPS §8–10 |
| 要求范围 | OPS-REQ-024–033 |
| 适用主题/前提 | 容量、过载与受控发布 |
| Evidence 类别 | 工作量模型、恢复基准、满载测试、canary |
| 责任角色 | Capacity / Release Owner |

表 16 · 记录 12 · OPS §11 – 13

| | |
|-------------|-----------------------|
| 来源 | OPS §11-13 |
| 要求范围 | OPS-REQ-034-042 |
| 适用主题/前提 | 迁移、恢复、重放或 DR 能力 |
| Evidence 类别 | 兼容性、业务 Oracle、恢复历时及演练 |
| 责任角色 | Data / Recovery Owner |

表 16 · 记录 13 · OPS §14 – 16

| | |
|-------------|------------------------|
| 来源 | OPS §14-16 |
| 要求范围 | OPS-REQ-043-050 |
| 适用主题/前提 | 可观测性、Runbook、关闭和 Day-2 |
| Evidence 类别 | 告警到行动映射、演练及责任名册 |
| 责任角色 | Operational Owner |

表 16 · 记录 14 · OPS §17 – 18

| | |
|-------------|-----------------------------|
| 来源 | OPS §17-18 |
| 要求范围 | OPS-REQ-051-055 |
| 适用主题/前提 | 权限、凭据、审计与外部依赖 |
| Evidence 类别 | 权限评审、轮换、依赖契约与退出证据 |
| 责任角色 | Security / Dependency Owner |

表 16 · 记录 15 · OPS §19 – 20

| | |
|-------------|---------------------------|
| 来源 | OPS §19-20 |
| 要求范围 | OPS-REQ-056-060 |
| 适用主题/前提 | 事故或受控演练 |
| Evidence 类别 | 指挥、止损、演练边界与发现闭环 |
| 责任角色 | Incident / Exercise Owner |

表 16 · 记录 16 · OPS §21 – 24

| | |
|-------------|-----------------------------|
| 来源 | OPS §21-24 |
| 要求范围 | OPS-REQ-061-065 |
| 适用主题/前提 | 准入、扩大暴露、运行债务与退役 |
| Evidence 类别 | Readiness、扩大判据、风险登记及退出验证 |
| 责任角色 | Release / Operational Owner |

10.4.3 未编号正文、模板与 Gate 的归属

表 17 · 字段记录视图（列名与原单元格逐项对应）

| | |
|-------------------------------------|--|
| 表 17 · 记录 1 · SUITE §3 – 12 的规范控制语句 | |
| 来源 | SUITE §3-12 的规范控制语句 |
| 处理规则 | 按文档版本和段落定位登记；只在规范维护、引用、发布或声明行为发生时适用 |
| 对应控制入口 | GOV-REQ-007/008/016/026；必要时保留独立段落控制 ID |

| | |
|------------------------------------|---|
| 表 17 · 记录 2 · METHOD §4 – 14 的通用义务 | |
| 来源 | METHOD §4-14 的通用义务 |
| 处理规则 | 根据性质、状态、资源或证据主题关联领域要求；无等价领域条款时保留独立段落记录，不遗漏原义务 |
| 对应控制入口 | GOV 的责任、ASSURANCE 的主张证据、OPS 的运行控制 |

| | |
|--|---|
| 表 17 · 记录 3 · REFERENCE §3 – 12 的阶段和 Gate 条件 | |
| 来源 | REFERENCE §3-12 的阶段和 Gate 条件 |
| 处理规则 | 是默认执行剖面；采用时逐条件映射其权威条款，重复项复用同一评估，不要求重复材料 |
| 对应控制入口 | GOV-REQ-017-020、ASSUR-REQ-039、OPS-REQ-061/062 |

| | |
|--------------------------------|------------------------------------|
| 表 17 · 记录 4 · REFERENCE §13 模板 | |
| 来源 | REFERENCE §13 模板 |
| 处理规则 | 必需语义由权威规范决定，格式可裁剪；示例填充值不是 Evidence |
| 对应控制入口 | GOV-REQ-017 |

| | |
|---------------------------------|--|
| 表 17 · 记录 5 · GOV §9.2、§19 – 21 | |
| 来源 | GOV §9.2、§19-21 |
| 处理规则 | Exception、符合性与 Gate 字段分别映射身份、授权、范围、期限与事实分类 |
| 对应控制入口 | GOV-REQ-018/021/022/026/029 |

| | |
|--|--|
| 表 17 · 记录 6 · ASSURANCE §5.1、§6.2、§11.1、§28 – 31 | |
| 来源 | ASSURANCE §5.1、§6.2、§11.1、§28-31 |
| 处理规则 | Claim、Assumption、Evidence 字段绑定各自唯一状态域；缺失关键字段不得默认成功 |
| 对应控制入口 | ASSUR-REQ-001/003/012-014/023-026/043 |

表 17 · 记录 7 · OPS 服务、SLO、资源、备份、恢复与发布契约模板

| | |
|--------|---|
| 来源 | OPS 服务、SLO、资源、备份、恢复与发布契约模板 |
| 处理规则 | 字段是否强制按引入正文的“必须/至少”与实际前提判定，不把所有占位符一律强制 |
| 对应控制入口 | OPS-REQ-011/016/023-026/032/037-040/061 |

表 17 · 记录 8 · 说明、反例、参考资料、案例数值和建议路线

| | |
|--------|-------------------------------------|
| 来源 | 说明、反例、参考资料、案例数值和建议路线 |
| 处理规则 | 标注 INFORMATIVE，不生成独立强制项；明确采用的项目合同除外 |
| 对应控制入口 | 保留来源，不伪造符合性结论 |

不适用必须证明前提不存在并获 Reviewer 接受；“暂无证据”“无状态”“系统很小”均不是通用不适用理由。阻断规则必须说明对应 Gate、失败/未知行为和授权边界，不能从编号或所在章节自动推断。

10.4.4 投入自动化前的检查

本索引是人工评审入口，不宣称已经完成生产规则实现。转为 Schema/CI 前必须逐项完成：所有编号 ID 展开；未编号 MUST 与字段覆盖检查；实际前提/强度/阻断级别审查；示例与规范区分；枚举与引用语义校验；正反例验证及授权批准。未完成的映射保持 UNKNOWN 并列出差异，不以默认值代填。

至少检查以下反例：例外获批不得改变反证；应急发布不得把缺失 Evidence 变成 PASS；ID 存在但理由不相关不得形成有效符合性；迁移旧状态无法无损映射时不得丢弃维度。本轮仅修正文档，未实施生产自动化，也未执行项目试点。

11. 符合性评估流程

11.1 输入

- 对象或变更身份；
- 风险等级与理由；
- 当前标准版本；
- 适用 Profile；
- 项目制品；
- Assurance Case 与 Evidence Bundle；
- Exception Register；
- 当前运行和事故信息。

11.2 步骤

1. Determine scope and risk tier
2. Resolve applicable requirements
3. Verify artifacts and ownership
4. Evaluate evidence and unknowns
5. Check exceptions and expiry
6. Determine requirement-level status
7. Aggregate without masking blockers
8. Record decision, authority and review trigger

11.3 输出

符合性报告至少包含：

```
assessment_id: "CONF-..."
subject: "..."
subject_identity: "artifact/config/environment digests"
risk_tier: "R..."
profile: "..."
standard_versions: {}
requirement_summary:
  conformant: 0
  nonconformant: 0
  unknown: 0
  not_applicable: 0
authorized_exception_count: 0 # 与事实分类交叉计数，不相互替代
blocking_items: []
exceptions: []
residual_risks: []
result: "..."
assessor: "..."
decision_authority: "..."
decided_at: "..."
expires_or_review_at: "..."
```

GOV-REQ-028：评估结论必须有有效期或复审触发

任何符合性结论都不能无限期有效。至少在以下变化后重新评估：

- 风险等级变化；
- 重大语义或运行变更；
- 关键证据失效；
- Exception 到期；
- 重大事故；
- Owner 或信任边界变化；

- 标准 Major 不兼容更新。

12. 设计评审治理

12.1 会前准入

高质量评审必须在会前完成材料可读性检查。以下情况不应进入正式裁决会议：

- 对象、风险或问题范围不清；
- 缺少关键 Owner；
- 没有列出主要未知；
- 方案只包含组件图；
- Reviewer 无足够时间阅读；
- 决策权和评审目标未声明。

此时应返回 NOT_READY_FOR_REVIEW，它不是 Gate 结论。

12.2 会议目标

评审应按以下顺序挑战：

```
Problem and consequence
-> Critical properties
-> State and authority
-> Failure and recovery
-> Resource and capacity
-> Security and operations
-> Evidence and residual risk
-> Decision
```

禁止让选型争论过早占据评审。

12.3 决策记录

每次正式评审必须记录：

- 日期、对象与版本；
- 参与角色；
- Gate；
- 结论；
- 关键依据；
- 阻断项；
- 反对意见；
- Conditions / Exceptions；

- Owner 与期限；
- 下一次触发。

GOV-REQ-029：口头批准不构成 Gate 决策

任何生产关键裁决必须形成可追溯记录。聊天、会议印象或“大家没反对”不能替代正式结论。

13. 生产放行治理

13.1 设计批准与发布批准分离

Design Approval

表示方案在声明约束下可接受

Release Approval

表示当前制品、配置、环境和 Change Envelope 具备生产资格

设计已批准不代表数月后的任意制品自动可发布。

GOV-REQ-030：发布必须绑定当前对象

Release Decision 必须绑定：

- artifact digest；
- configuration digest；
- deployment target；
- dependency/schema/model versions；
- Evidence Bundle；
- Change Envelope；
- 当前 Exception；
- Release Authority。

GOV-REQ-031：发布权限不得绕过设计阻断

Release Authority 不能以“业务紧急”直接覆盖未解决的高后果 NO_GO 或 UNKNOWN。若确需行动，必须进入明确的 Exception 或 emergency governance。

14. 紧急变更治理

14.1 紧急变更成立条件

只有当等待正常流程会显著扩大当前损害时，才能使用 emergency path。排期、发布日期或普通商业压力不构成紧急情况。

14.2 最低控制

即使紧急，也必须保留：

- 变更者身份；
- 事故或风险上下文；
- 预期效果；
- 最大范围；
- 观察信号；
- 停止/撤销路径；
- 临时批准者；
- 完整审计。

GOV-REQ-032：紧急变更必须补偿复审

事故稳定后必须在规定期限内完成：

- 正式设计/安全/运行复审；
- 证据补齐；
- 临时权限撤销；
- 风险和例外更新；
- Assurance Delta；
- 判断是否需要修改正常路径。

15. 标准偏离与领域 Profile

15.1 Profile

领域 Profile 可以：

- 新增领域特有的 Critical Property；
- 收紧风险等级和独立性；
- 增加专用 Evidence；
- 定义特定 Gate 条件；

- 提供领域模板和反例。

Profile 不得：

- 将核心 MUST 静默降为 SHOULD；
- 改变 UNKNOWN 或 Evidence 语义；
- 允许多权威而不定义冲突；
- 省略高后果状态身份和恢复来源。

15.2 本地标准

团队可制定本地标准，但必须声明：

- 与本套系的映射；
- 额外要求；
- 任何偏离；
- 冲突裁决；
- Owner 与版本。

GOV-REQ-033：重复例外必须触发规范复审

如果多个项目对同一要求反复申请例外，应评估：

- 要求是否不可执行；
- 平台能力是否缺失；
- 风险分级是否错误；
- 是否需要标准修订或领域 Profile；
- 是否存在组织性资源不足。

不能让长期重复例外成为隐式新规范。

16. 审计与记录保留

16.1 必须保留的记录

对 R2/R3 对象至少保留：

- 设计和 ADR；
- 风险分类；
- Owner 记录；
- Gate 决策；
- Assurance Case 与 Evidence Bundle；
- Exception 与 break-glass；

- 发布和回滚记录；
- 重大事故与 Assurance Delta；
- 标准版本和符合性声明。

16.2 完整性

用于裁决的记录不得被静默修改。修正应产生新版本并保留历史、变更者、时间和理由。

GOV-REQ-034：记录保留期与风险匹配

保留期应覆盖：

- 业务和法律要求；
- 最大数据生命周期；
- 事故调查和追溯窗口；
- 证据有效期；
- 迁移和回放周期。

16.3 隐私与最小化

审计不意味着无限收集。记录应避免保存无必要的敏感值，优先保存 digest、引用和受控证据位置。

17. 治理指标与反 Goodhart

17.1 有价值的指标

可以观测：

- UNKNOWN 的关闭时长和类型；
- Exception 数量、年龄和重复主题；
- 评审前发现的高后果缺陷；
- 证据错绑、过期和不可复现率；
- 事故后 Assurance Delta 的关闭情况；
- 恢复演练通过率和实际恢复时间；
- 门禁误报、漏报和绕过原因；
- 低价值要求的维护成本。

17.2 禁止的替代目标

以下指标不能单独作为治理成功标准：

- 文档页数；
- Gate 通过率；

- 模板完成率；
- 测试覆盖率；
- 评审会议次数；
- Exception 为零；
- 发布速度单一指标。

高通过率可能意味着风险低，也可能意味着 Gate 已失去挑战能力。

GOV-REQ-035：指标必须与风险学习关联

治理指标应推动删除无价值控制、加强薄弱机制或建设平台能力，而不是只用于个人绩效排名。

18. 渐进式采用

18.1 推荐顺序

1. 统一术语、风险等级和决策状态
2. 对一个高价值项目试点
3. 建立关键 Owner 与最小制品
4. 引入 Evidence Matrix 和 Production Readiness
5. 提取高确定性规则进入 CI
6. 建立 Exception、证据时效与漂移治理
7. 用事故和运行数据校准标准
8. 删除低价值流程并建设平台自助能力

18.2 采用原则

- 先治理高后果、难逆转决策；
- 不以一次性大规模推广制造合规表演；
- 保留人工评审处理未编码属性；
- 将重复机械检查自动化；
- 对团队能力不足建立平台支持，而不是只增加审批；
- 每次扩展都明确新增价值和复杂度预算。

19. 符合性声明模板

```
conformance_declaration:
  declaration_id: "CONF-YYYY-NNN"
  system_or_change: "..."
  subject_identity:
    repository: "..."
    commit: "..."
    artifact_digest: "..."
    config_digest: "..."
    environment: "..."
  risk_tier: "R0|R1|R2|R3"
  profile: "..."
  suite_version: "<adopted-suite-version>"
  documents:
    ESD-GOV-001: "<adopted-document-version>"
    ESD-ASSURANCE-001: "<adopted-document-version>"
    ESD-OPS-001: "<adopted-document-version>"
  requirement_results: # 按满足事实分类，不能因例外改写
    conformant: []
    nonconformant: []
    unknown: []
    not_applicable: []
  governance_dispositions: [] # 每条关联 requirement/control ID、STANDARD 或
    └─ AUTHORIZED_EXCEPTION
  exceptions: []
  blocking_unknowns: []
  residual_risks: []
  evidence_bundle: "..."
  result: "CONFORMANT|CONDITIONALLY_CONFORMANT|NONCONFORMANT|UNKNOWN"
  assessor: "..."
  decision_authority: "..."
  decided_at: "..."
  valid_until_or_review_trigger: "..."
```

20. Gate 决策模板

```
# Gate Decision

- Decision ID:
- System / Change:
- Subject identity:
- Risk tier:
- Gate:
- Decision: GO / CONDITIONAL_GO / NO_GO / UNKNOWN / NOT_APPLICABLE
```

```
字面文本 · continued

- Decision authority:
- Date:

## Scope

## Inputs reviewed

## Requirement status

## Blocking findings

## Conditions

| Condition | Owner | Deadline | Evidence | Abort / Escalation |
|---|---|---|---|---|

## Exceptions

## Residual risks

## Dissenting views

## Review triggers

## Final rationale
```

21. Exception 模板

```
# Exception EX-XXX

- Requirement ID:
- Subject:
- Risk tier:
- Status:
- Owner:
- Exception authority:
- Effective date:
- Expiry date:

## Requirement and deviation

## Reason the requirement cannot currently be met

## Risk and maximum consequence
```

| |
|----------------------------|
| 字面文本 · continued |
| ## Scope |
| ## Compensating controls |
| ## Monitoring and alerting |
| ## Abort conditions |
| ## Exit plan |
| ## Evidence retained |
| ## Review triggers |
| ## Closure record |

22. 最终符合性检查清单

范围与风险

- ☐ 对象身份和变更范围明确；
- ☐ 风险等级有后果、爆炸半径、不确定性和可逆性依据；
- ☐ 普通变更、重大变更、实验和 emergency path 未混淆；
- ☐ Profile 与裁剪有记录。

责任

- ☐ System、Semantic、Technical 和 Operational Owner 明确；
- ☐ Reviewer 与 Decision Authority 具备授权；
- ☐ R2/R3 职责分离满足要求；
- ☐ on-call 和恢复责任已接受。

制品与 Gate

- ☐ 最低制品与风险匹配；
- ☐ Gate 输入、阻断和结论可追溯；
- ☐ 高后果 UNKNOWN 未被隐藏；
- ☐ 设计批准和发布批准已区分。

例外

- ☐ 所有偏离绑定 Requirement ID;
- ☐ Exception 有范围、期限、Owner 和补偿控制;
- ☐ 过期例外未继续支撑生产许可;
- ☐ break-glass 已完成补偿复审。

符合性

- ☐ 按适用要求逐项判断;
- ☐ NOT_APPLICABLE 有论证;
- ☐ 当前证据与对象版本绑定;
- ☐ 结论有有效期或复审触发;
- ☐ 最终声明未夸大为绝对正确性。

23. 结论

优秀的治理不是把工程师置于更多审批之下，而是建立一套与风险匹配的责任和决策结构：低风险变更可以沿已验证路径快速流动，高风险变更必须暴露状态、失败、未知、证据和责任；例外可以存在，但必须可见、可到期、可撤销；最终裁决必须由有权承担后果的人基于当前对象和当前证据作出。

本规范的最终目标是形成以下闭环：

- 明确适用范围
 - > 风险匹配的要求
 - > 具名责任和独立挑战
 - > 可执行 Gate
 - > 受控例外
 - > 可追溯符合性
 - > 周期复审和事故学习

当这条链路成立时，系统设计标准才不再只是优秀文章，而成为能够约束真实交付和真实风险的工程控制面。

工程保证论证、证据与裁决规范

从设计主张到可审计 Assurance Case、Evidence Bundle 与发布决策

| 文档属性 | 内容 |
|------|---------------------------------------|
| 文档编号 | ESD-ASSURANCE-001 |
| 版本 | 2.0.0-draft.1 (基于 1.0.0) |
| 状态 | DRAFT / NORMATIVE DOMAIN; 未获发布批准 |
| 规范角色 | 定义如何提出可裁决主张、组织论证、评价证据、处理反证并支持 Gate 决策 |
| 语义基础 | ESD-METHOD-001 《优秀系统设计与工程保证方法论》 |
| 治理基础 | ESD-GOV-001 《系统设计与工程保证治理及符合性规范》 |
| 执行参考 | ESD-REFERENCE-001 《优秀系统设计：从约束、不变量到证据》 |
| 权威格式 | 本 Markdown 文件；PDF 与其他导出仅为阅读视图 |

规范目的

本文回答一个工程系统必须持续面对的问题：我们为什么有资格相信某项关键性质在当前制品、配置、环境、工作负载和故障范围内成立？它要求工程团队不仅提供测试结果，还要说明主张、假设、论证策略、证据质量、反证、覆盖边界、证据时效和最终裁决之间的关系。

摘要

工程保证不是“多写测试”的同义词，也不是对系统绝对正确性的承诺。它是一条受控推理链：

Claim

- > Context and Assumptions
- > Argument
- > Mechanisms
- > Verification Activities
- > Evidence
- > Defeaters and Limitations
- > Decision
- > Runtime Revalidation

一份测试报告可能真实，却与当前二进制无关；一项形式证明可能严密，却只覆盖了抽象模型；一个生产指标可能代表真实流量，却无法证明未观测到的错误不存在。高质量 Assurance Case 的任务，是说明不同证据分别支持什么、不支持什么，它们如何组合，以及在什么变化后必须失效。

本规范明确区分四类经常被混淆的状态：

| 层次 | 示例状态 |
|---------|---|
| 验证执行 | PASS / FAIL / ERROR / SKIPPED |
| 证据结论 | SUPPORTS / CONTRADICTS / INCONCLUSIVE / NOT_APPLICABLE |
| 证据生命周期 | CURRENT / STALE / INVALIDATED / SUPERSEDED / NOT_REPRODUCIBLE |
| Gate 裁决 | GO / CONDITIONAL_GO / NO_GO / UNKNOWN / NOT_APPLICABLE |

这四层不得互相替代。测试命令退出码为零，不自动推出关键性质成立；证据当前有效，也不自动授权生产发布；Gate GO 只对声明的对象和 Change Envelope 有效。

1. 适用范围与非目标

1.1 适用范围

本规范适用于：

- R1–R3 系统和重大变更的关键性质论证；
- Architecture Review 和 Production Readiness 的证据评价；
- 发布、迁移、恢复、重放和安全变更的 Evidence Bundle；
- CI/CD 中产生的测试、分析、策略、构建和供应链证据；
- 生产指标、审计日志、演练结果和事故材料；
- AI/Agent、控制器和自动化系统的评估与运行证据；
- Assurance Plane 自身的规则、工具、例外和完整性证明。

1.2 非目标

本文不：

- 要求所有项目采用特定图形化 Assurance Case 语言；
- 宣称有限测试可以证明开放世界中的绝对无缺陷；
- 用单一分数取代对阻断性风险的判断；
- 规定具体测试框架、形式化工具或监控平台；
- 让证据 Reviewer 替代 System Owner 和 Decision Authority 的风险责任；
- 将自动验证扩展到它没有编码的属性。

2. 核心术语

| 术语 | 定义 |
|-----------------------|---|
| Claim | 关于特定对象、作用域和条件下应成立性质、具有明确验证义务的陈述 |
| Top-level Claim | Assurance Case 最终试图支持的系统级结论 |
| Subclaim | 为支持上层 Claim 而分解出的较小主张 |
| Context | 解释 Claim、Argument 或 Evidence 含义所需的背景事实 |
| Assumption | 论证成立所依赖、可被证伪并可能失效的外部条件 |
| Argument | 说明 Evidence 与子主张为何足以支持 Claim 的推理结构 |
| Strategy | 对 Claim 的分解方式，例如按故障类别、边界、性质或运行模式分解 |
| Evidence | 能支持、反驳或限制 Claim 的可追溯事实或制品 |
| Counterevidence | 与 Claim 相冲突或削弱现有论证的证据 |
| Defeater | 若成立会使 Argument 失效或显著减弱的条件、反例或替代解释 |
| Mechanism | 系统中用于维持某项性质的控制、协议、约束或实现 |
| Verification Activity | 用于生成或评价 Evidence 的分析、测试、检查、证明或演练 |
| Evidence Envelope | Evidence 所绑定的主体、环境、范围、版本、覆盖、限制和时效 |
| Residual Risk | 在当前控制和 Evidence 后仍被接受或保持未知的风险 |
| Assurance Case | Claim、Argument、Evidence、Assumption、Defeater 和 Decision 的版本化结构 |
| Evidence Bundle | 为特定 Gate 或发布收集的不可混淆证据集合及清单 |
| Decision-grade | Evidence 和 Argument 达到当前风险与裁决所需的质量，而非“证据很多” |

3. 工程保证基本原则

ASSUR-REQ-001：主张必须具有类型匹配的验证义务

关键主张必须明确适用范围、成立前提、违反语义及与性质类型匹配的验证义务。Safety、有界活性、无界活性和统计 SLO 不得被强行套用同一种有限观测证伪规则，具体见 §5.2。以下表达不合格：

- 系统足够可靠。

性能很好。

架构支持 `exactly-once`。

系统是安全的。

合格表达至少包含：

```
subject
property
scope
identity
temporal semantics
operating mode
assumptions
violation consequence
```

例如：

对同一 `source_object_version` 与 `pipeline_semantic_version`，在 NORMAL 和 RECOVERY 模式下，最多存在一个状态为 COMMITTED 的正式 completion manifest；并发重试、进程崩溃和 Kafka 重投递不得产生第二个不同结果。

ASSUR-REQ-002：主张、机制和证据必须分离

“系统使用事务”是机制，不是性质；“测试通过”是验证结果，不是系统主张；“线上三个月无事故”是观察，不是绝对证明。Assurance Case 必须分别表达这些层次。

ASSUR-REQ-003：Evidence 必须绑定具体对象

每份用于裁决的 Evidence 必须能够关联当前：

- 源代码或模型；
- 构建制品；
- 配置；
- Schema、策略、模型、DBC 或参考数据；
- 环境；
- 工作负载语料；
- 故障档案；
- 验证工具版本。

无法完成主体绑定的 Evidence 不能支持当前发布，只能作为背景信息。

ASSUR-REQ-004：负面证据不得被删除或隐藏

失败测试、事故、反例、已知缺陷、Reviewer 异议和生产异常都必须进入 Assurance Case。只收集支持性材料会形成确认偏差。

ASSUR-REQ-005：没有证据时保持未知

未执行、工具错误、证据过期、错绑、不可复现或覆盖不足必须产生 UNKNOWN / INCONCLUSIVE，不得转化为 PASS。

ASSUR-REQ-006：局部证据不能自动推出端到端性质

组件 A 和 B 各自通过不意味着 A→B 的边界、顺序、超时、容量和混合版本组合已被证明。组合保证必须显式论证。

ASSUR-REQ-007：Evidence 具有生命周期

证据必须有当前状态、失效条件和复审触发。证据不是一经产生永久有效的勋章。

ASSUR-REQ-008：裁决必须保留覆盖边界

任何 Gate 或发布结论必须说明：

- 支持到什么范围；
- 哪些情况未覆盖；
- 哪些 Assumption 尚未验证；
- 在什么变化后结论失效；
- 哪些 Residual Risk 被谁接受。

ASSUR-REQ-009：保证面自身必须 fail-closed

对 R2/R3 对象，Evidence 缺失、策略引擎不可用、验证执行异常或身份无法关联时，Assurance Plane 不得默认放行。

4. Assurance Object 与身份

4.1 Assurance Object

任何 Assurance Case 必须声明其对象，可能是：

- 系统或服务；
- 某个版本的组件；
- 架构决策；
- 数据迁移；
- 发布候选；
- 恢复能力；
- 运行模式；
- 权限和策略集合；
- 自动化门禁本身。

4.2 主体身份

最低身份结构：

```
subject_identity:
  system_id: "..."
  repository: "..."
  source_commit: "..."
  build_id: "..."
  artifact_digest: "sha256:..."
  config_digest: "sha256:..."
  schema_or_policy_versions: []
  environment_id: "..."
  deployment_target: "..."
```

ASSUR-REQ-010：不能只用可变标签标识对象

latest、分支名、环境昵称或镜像 tag 不能作为唯一 Evidence 绑定依据。必须存在不可变 digest 或等价身份。

ASSUR-REQ-011：设计 Assurance 与发布 Assurance 必须区分

Design Assurance

支持方案在声明假设下能够维护性质

Release Assurance

支持当前制品和配置在当前目标环境中具备放行资格

Operational Assurance

支持运行系统仍处于已验证边界内

三者可以共享 Evidence，但不能互相替代。

5. Claim Contract

5.1 最低字段

每项阻断级 Claim 至少包含：

```
claim_id: "CLM-..."
property_id: "CP-..."
title: "..."
subject: "..."
claim_type:
  └ "SAFETY|LIVENESS|DURABILITY|SECURITY|RESOURCE|SLO|RECOVERY|ASSURANCE_META"
scope:
```

字面文本 · continued

```
identities: []
operations: []
tenants_or_regions: []
operating_modes: []
versions: []
temporal_semantics: "always|eventually|within-window|statistical"
statement: "..."
assumptions: []
allowed_degradation: "..."
violation: "..."
verification_obligation: "..."
observation_limits: "..."
consequence: "..."
criticality: "BLOCKING|MAJOR|MINOR"
owner: "..."
review_triggers: []
```

5.2 Claim 类型

Safety Claim

说明“不允许发生什么”：

- 同一业务身份不得出现两个互相冲突的正式提交。
- 未授权主体不得修改策略。
- `Partial result` 不得被标记为 `completed`。

Liveness Claim

说明在明确 Assumption 下“最终会发生什么”：

- 已接纳且依赖可用的任务最终进入 `COMMITTED`、`QUARANTINED` 或 `FAILED_FINAL`。

Liveness 必须声明公平性、依赖可用性、资源和时间假设，并区分：

| 性质 | 违反/支持义务 | 有限观测的边界 |
|--------|--------------------------------|---|
| Safety | 明确有限坏前缀或违反事件；在模型内可检查可达性 | 单个反例可反驳，未发现反例不等于全域证明 |
| 有界活性 | 在规定前提下定义起点、截止时间、完成 Oracle | 超时可反驳对应有界 Claim |
| 无界活性 | 形式化进展性质；论证公平调度、持续资源、依赖可用性与恢复进展 | 有限时间未完成不直接反驳“最终完成”；模型中的无限反例或进展证明须保留模型前提 |
| SLO | 明确统计总体、窗口、样本、估计方法及不确定性 | 不能从有限样本外推未覆盖流量与未来窗口 |

以上 Safety/Liveness 与公平性区分属于理论基础，参见 Lamport, *Specifying Systems*, 第 8–9 章；本套系的风险等级与裁决流程则是组织治理选择。

无界活性的运行告警只能表明进展风险或有界运营目标违反，不能自动冒充该无界命题的反证。需要可执行门禁时，应另立有界恢复/完成 Claim，而不是静默改写原命题。

验证方法的结论边界遵循 **METHOD §3.1**：模型检查与定理证明可在明确形式化范围内证明性质，不只是“没发现问题”；但需单独论证模型到实现的符合关系。

Durability / Recovery Claim

说明在指定故障集合内哪些事实保持可恢复、多久恢复以及从哪个权威源恢复。

Resource Claim

说明资源硬上界和过载行为，而不是只给平均利用率。

SLO Claim

说明统计窗口、流量范围、排除项和测量方法。SLO 不能替代硬正确性 Claim。

Assurance Meta-claim

说明证据、门禁或身份链自身不会被静默绕过，例如：

缺失或过期证据不得被解释为 PASS。

ASSUR-REQ-012：Claim 必须有作用域

没有版本、身份、模式、租户、区域或时间范围的全局 Claim 默认不具备可裁决性。

ASSUR-REQ-013：Claim 必须有违反定义

必须按 §5.2 说明 Claim 的违反语义、可用反证、支持性论证和观测限制。无界活性需进展与假设论证，不能要求有限“尚未完成”作为反证；SLO 需统计判据。运行时不可直接检测的部分必须登记补充验证方法、Owner 与剩余未知。

6. Context、Assumption 与 Guarantee

6.1 Context

Context 是定义域事实，例如：

- Kafka 为 at-least-once delivery；
- 单对象硬上限为 40 MiB；

- DBC Registry 版本不可变；
- 生产节点规格为某 CPU/内存组合。

Context 不应被用于掩盖未验证 Assumption。

6.2 Assumption

每个关键 Assumption 至少包含：

```
assumption_id: "ASM-..."
statement: "..."
owner: "..."
validation_method: "..."
runtime_signal: "..."
valid_range: "..."
expiry_or_review_trigger: "..."
violation_response: "..."
validation_status: "UNVALIDATED|VALIDATED|VIOLATED|UNKNOWN"
lifecycle_status: "PROPOSED|ACTIVE|EXPIRED|RETIRED"
monitoring_status: "MONITORED|UNMONITORED|UNAVAILABLE|NOT_APPLICABLE"
risk_flag: "NONE|AT_RISK"
evidence_refs: []
last_validated_at: "..."
supersedes: []
```

以上为 Assumption 字段的唯一受控定义；METHOD 与 REFERENCE 只摘要引用，不另建状态机。四个维度独立：

- `validation_status` 表示未验证、证据支持、已知违反或无法判断；必须保留证据和作用域。
- `lifecycle_status` 表示提出、使用中、有效期结束或退役；到期不是事实违反。
- `monitoring_status` 表示当前有监测、未配置、监测不可用或经论证无需运行监测；后者仍需复审机制。
- `risk_flag` 是预警标记，AT_RISK 不自动改变验证结论。

迁移旧单字段时，VALIDATED/UNVALIDATED/VIOLATED/UNKNOWN 进入验证维度，PROPOSED/EXPIRED/RETIRED 进入生命周期，MONITORED 进入监测维度，AT_RISK 进入风险维度。其他维度须根据原始记录补齐，不能默认“已验证”。旧 INVALIDATED 必须人工区分事实违反、支持证据失效或假设被替换；旧 REPLACED 以旧记录 RETIRED 加新 ID / `supersedes` 表达。

证据不足或过期时重评验证状态；仍有适用反证则保留 VIOLATED，不得借过期、风险接受或退役把已知违反改写为未知。依赖 Claim 按 §19 重评。

ASSUR-REQ-014：关键 Assumption 必须可监测或可复审

若性质依赖某个外部条件，而系统既不验证、也不监测、也无复审触发，则该 Assumption 形成 Epistemic Debt。对阻断级 Claim，这通常导致 UNKNOWN。

6.3 Assume–Guarantee

边界契约应表达：

If Assumptions(A) hold,
component A Guarantees G(A).

Consumer B may rely on G(A)
only if G(A) satisfies Assumptions(B).

边界必须覆盖：

- 成功结果；
- 错误和超时；
- 重复和乱序；
- 容量和背压；
- 版本兼容；
- 降级与恢复模式。

ASSUR-REQ-015：上游 Guarantee 必须满足下游 Assumption

仅仅“接口字段相同”不足以证明组合成立。时序、容量、重试、可见性和错误语义也必须匹配。

7. Argument 结构

7.1 标准结构

Top-level Claim
supported by Strategy
 -> Subclaim 1
 -> Evidence A, B
 -> Subclaim 2
 -> Evidence C
 -> Subclaim 3
 -> Evidence D, E
constrained by Context
dependent on Assumptions
challenged by Defeaters / Counterevidence

7.2 常用分解策略

| Strategy | 适用场景 |
|----------|--|
| 按关键性质 | Safety、Liveness、Durability、Security 分解 |
| 按状态转换 | CREATE、PROCESS、COMMIT、ACK、RECOVER 分解 |
| 按故障类别 | crash、timeout、partition、resource exhaustion、operator error |
| 按边界 | producer、broker、worker、store、consumer |
| 按运行模式 | NORMAL、DEGRADED、DRAINING、RECOVERY、EMERGENCY |
| 按租户/区域 | 多租户隔离、多区域故障域 |
| 按生命周期 | build、deploy、operate、migrate、retire |

ASSUR-REQ-016：分解必须保持完整性

分解后必须说明为什么所有子 Claim 共同足以支持上层 Claim。不能只列出一组测试名称而没有覆盖论证。

7.3 Argument Sufficiency

论证充分性至少检查：

- Claim 是否定义清楚；
- Assumption 是否可接受；
- Strategy 是否遗漏关键状态或故障；
- Evidence 是否与 Claim 相关；
- 证据是否代表目标环境；
- Counterevidence 是否已处理；
- 组合边界是否成立；
- 剩余未知是否在授权范围内。

ASSUR-REQ-017：不得用数量替代充分性

更多测试、更多页面或更高覆盖率不能自动弥补一个未覆盖的阻断级提交窗口、权限边界或恢复路径。

8. Defeater 与 Counterevidence

8.1 Defeater 类型

| 类型 | 示例 |
|--------------|--------------------------------|
| 适用性 Defeater | 测试环境不是生产架构 |
| 身份 Defeater | Evidence 来自旧 commit |
| 覆盖 Defeater | 未覆盖写成功但超时的结果未知窗口 |
| 代表性 Defeater | 基准只使用小对象和干净数据 |
| 工具 Defeater | 静态分析器对相关语言特性不支持 |
| 组合 Defeater | A/B 单独通过，但 mixed-version 边界未测试 |
| 运行 Defeater | 输入分布已经超出验证范围 |
| 组织 Defeater | 手工步骤无稳定 Owner，无法重复执行 |

8.2 Defeater Register

```
defeater_id: "DFT-..."
related_claims: []
statement: "..."
source: "review|incident|test|runtime|assumption"
severity: "BLOCKING|MAJOR|MINOR"
status: "OPEN|MITIGATED|ACCEPTED|REFUTED|CLOSED"
owner: "..."
resolution_evidence: []
review_trigger: "..."
```

ASSUR-REQ-018：阻断级 Defeater 未解决时不得给出支持性结论

对阻断级 Claim，如果存在尚未反驳或缓解的阻断级 Defeater，Evidence Conclusion 必须为 INCONCLUSIVE 或 CONTRADICTS。

ASSUR-REQ-019：Counterevidence 与支持性 Evidence 同等保留

不得删除失败报告、异常样本或 Reviewer 异议以“清理”发布包。关闭它们必须保留修复和重新验证链路。

9. Evidence 分类

9.1 Evidence 类型

| 类型 | 示例 | 主要优势 | 常见边界 |
|-------------|------------------------------------|----------|-------------------------|
| DESIGN | ADR、协议说明、状态机 | 解释意图与语义 | 不能证明实现符合 |
| STATIC | 类型检查、lint、静态分析、
依赖扫描 | 可重复、覆盖广 | 仅检查已建模规则 |
| FORMAL | 模型检查、定理证明、协议
验证 | 对模型范围内强 | 抽象与实现映射可能不足 |
| UNIT | 单元测试、单元级
mutation 测试 | 快、可定位 | 环境与组合覆盖有限 |
| PROPERTY | 属性生成、性质驱动测试 | 探索输入与不变量 | 受生成器、Oracle 与采样范围
限制 |
| INTEGRATION | 跨组件契约和故障测试 | 验证边界 | 语料和环境仍可能不代表生产 |
| SYSTEM | 端到端、性能、恢复、soak | 代表真实组合 | 成本高、覆盖仍有限 |
| SECURITY | threat model、SAST/DAST、
渗透、权限审计 | 关注攻击面 | 时效和环境敏感 |
| OPERATIONAL | SLI、审计、生产 invariant、
事件 | 真实运行事实 | 缺少反事实，未观测不等于不
存在 |
| RECOVERY | restore、replay、DR、
game day | 验证恢复能力 | 需与生产规模和依赖匹配 |
| REVIEW | 专家评审、独立挑战 | 发现未编码风险 | 主观、需记录依据 |
| PROVENANCE | 构建证明、签名、digest、
SBOM | 建立对象身份链 | 不证明业务语义正确 |

本表是 `evidence_type` 的唯一受控枚举，§11.1 必须与之一致。每份 Evidence 选一个主要类型；其他方法用 `tags` 或独立 Evidence 记录。旧值 `UNIT/PROPERTY` 按实际活动明确迁移为 `UNIT` 或 `PROPERTY`，无法区分时要求人工复审；未知值属于记录验证错误，不得静默映射为任意已知类型。

ASSUR-REQ-020：阻断级 Claim 应使用证据多样性

对 R2/R3 阻断级 Claim，原则上不应只依赖单一类型 Evidence。至少应组合：

- 设计/语义 Evidence；
- 实现或动态 Verification；
- 对目标环境具有代表性的系统/运行 Evidence；
- 必要时的独立 Review。

无法满足时必须记录理由和剩余风险。

9.2 Evidence 不是装饰

每份 Evidence 必须回答：

- 1. 它支持或反驳哪个 Claim?
- 2. 它检查了什么机制或场景?
- 3. 它绑定哪个对象?
- 4. 它在什么环境和输入下产生?
- 5. 结果是什么?
- 6. 哪些情况未覆盖?
- 7. 何时失效?

10. Evidence Quality Model

10.1 质量维度

每份关键 Evidence 至少按以下维度评价：

| 维度 | 关键问题 |
|--------------------|---------------------------|
| Relevance | 与 Claim 的逻辑关系是否直接? |
| Validity | 方法是否真正测量了声称测量的性质? |
| Provenance | 来源、生成过程和责任主体是否清楚? |
| Subject Binding | 是否绑定当前制品、配置和环境? |
| Reproducibility | 能否在受控条件下重新产生? |
| Representativeness | 语料、负载和环境是否代表目标使用? |
| Coverage | 覆盖了哪些状态、故障、版本和边界? |
| Freshness | 当前系统变化后是否仍有效? |
| Integrity | Evidence 是否防篡改、可验证、未静默修改? |
| Independence | 生成和评价是否具备风险匹配的独立性? |
| Limitations | 未覆盖和方法弱点是否显式? |
| Actionability | 失败时是否能定位、阻断或触发响应? |

10.2 质量结论

建议使用：

- STRONG
- ADEQUATE
- LIMITED
- INSUFFICIENT
- INVALID

不得把这些质量词聚合成一个可以掩盖阻断项的平均分。

ASSUR-REQ-021: Evidence 质量必须与 Claim 关键性匹配

MINOR Claim 可以接受较轻 Evidence；BLOCKING Claim 必须达到足以支持生产裁决的质量。关键性越高，对代表性、独立性、覆盖和完整性的要求越高。

ASSUR-REQ-022: 不能只评价执行是否成功

测试框架运行成功、无崩溃或生成报告不等于 Evidence 有效。必须评价测试是否正确、是否命中目标属性、是否使用代表性场景。

11. Evidence Envelope

11.1 标准结构

```
evidence:
  evidence_id: "EVD-..."
  title: "..."
  evidence_type: "DESIGN|STATIC|FORMAL|UNIT|PROPERTY|INTEGRATION|SYSTEM|SECURITY|
    ↳ OPERATIONAL|RECOVERY|REVIEW|PROVENANCE"
  related_claims: []
  subject_identity:
    source_commit: "..."
    artifact_digest: "..."
    config_digest: "..."
    schema_policy_model_versions: []
  environment:
    environment_id: "..."
    platform: "..."
    node_or_runtime_spec: "..."
    dependencies: []
  verification:
    activity_id: "..."
    tool: "..."
    tool_version: "..."
    invocation_or_protocol: "..."
    workload_corpus_digest: "..."
    fault_profile: "..."
    started_at: "..."
    completed_at: "..."
  result:
    execution_status: "PASS|FAIL|ERROR|SKIPPED"
    evidence_conclusion: "SUPPORTS|CONTRADICTS|INCONCLUSIVE|NOT_APPLICABLE"
    summary: "..."
  coverage:
```

字面文本 · continued

```
    included: []
    excluded: []
    known_limitations: []
  quality:
    relevance: "...
    representativeness: "...
    reproducibility: "...
    independence: "...
  provenance:
    producer: "...
    artifact_uri: "...
    artifact_digest: "...
    signature_or_attestation: "...
  lifecycle:
    status: "CURRENT|STALE|INVALIDATED|SUPERSEDED|NOT_REPRODUCIBLE"
    valid_until: "...
    invalidation_rules: []
    supersedes: []
  owner: "...
  reviewer: "...
```

ASSUR-REQ-023: Evidence ID 必须稳定且唯一

同一 Evidence ID 不得用于不同执行结果。重复验证应生成新的 Evidence ID，并通过 `supersedes` 或关联字段形成历史链。

ASSUR-REQ-024: 原始制品和摘要必须区分

仪表盘截图、PR 评论或人工摘要不能替代原始日志、报告、数据和 `digest`。摘要可以用于阅读，但必须引用原始 Evidence Artifact。

ASSUR-REQ-025: Evidence 不得只依赖易失链接

用于长期裁决的 Evidence 必须存放在受控、可保留、可验证完整性的系统中。临时 CI URL 到期后，裁决链不能失去依据。

12. 结果状态的严格区分

12.1 Verification Execution Status

| 状态 | 含义 |
|---------|------------------------|
| PASS | 验证活动按定义执行，断言未被违反 |
| FAIL | 验证活动执行且发现违反 |
| ERROR | 工具、环境或基础设施错误，未得到有效性质结果 |
| SKIPPED | 验证未执行或被有意跳过 |

ERROR 和 SKIPPED 不是 PASS。

12.2 Evidence Conclusion

| 状态 | 含义 |
|----------------|----------------------|
| SUPPORTS | 在声明覆盖内为 Claim 提供支持 |
| CONTRADICTS | 为 Claim 的违反或关键反例提供证据 |
| INCONCLUSIVE | 不能合理支持或反驳 Claim |
| NOT_APPLICABLE | 经论证与该 Claim 无关 |

一个验证 PASS 仍可能产生 INCONCLUSIVE，例如它只验证了不相关配置。

12.3 Evidence Lifecycle Status

| 状态 | 含义 |
|------------------|------------------------|
| CURRENT | 当前仍在声明有效范围内 |
| STALE | 时间或环境变化使代表性下降，需要复审 |
| INVALIDATED | 已知变化使 Evidence 不再适用 |
| SUPERSEDED | 被更新的 Evidence 替代，仍保留历史 |
| NOT_REPRODUCIBLE | 无法按声明方法重新产生 |

12.4 Gate Decision

Gate 状态由 ESD-GOV-001 定义。Evidence 只能支持裁决，不能自行批准生产。

ASSUR-REQ-026：状态不得跨层偷换

以下映射均不成立：

```
Verification PASS -> Gate GO
No alert -> Property PASS
Evidence CURRENT -> System conformant
Test ERROR -> Test PASS
Old benchmark -> Current capacity proven
```

13. Evidence 生命周期与失效

13.1 生命周期

下图是证据 workflow 阶段，不是 `lifecycle.status` 的另一个枚举；计划、生成、评审和归档由活动记录表达，状态字段只使用 §12.3 的受控值。

```
PLANNED
-> PRODUCED
-> REVIEWED
-> CURRENT
-> STALE / INVALIDATED / SUPERSEDED / NOT_REPRODUCIBLE
-> ARCHIVED
```

13.2 失效触发

至少以下变化应触发相关 Evidence 复审或失效：

- 源代码、构建器、编译器或依赖变化；
- 配置、Feature Flag、策略或权限变化；
- Schema、协议、模型、Prompt、DBC 或参考数据变化；
- 目标环境、节点规格、内核、区域或存储后端变化；
- 工作负载分布、对象大小、热点或突发超出范围；
- 新故障模式、事故或安全漏洞；
- Evidence 到达 `valid_until`；
- 生成工具被发现存在缺陷；
- 原始 Evidence 无法读取或校验 digest。

ASSUR-REQ-027：失效必须传播到 Claim 与 Gate

Evidence 失效后，依赖它的 Claim Conclusion 和未过期 Gate Decision 必须被重新计算。若既无足够支持也无适用反证，Claim 为 UNKNOWN；存在适用反证时保留 CONTRADICTED，Gate 另行重评。

ASSUR-REQ-028：失效规则必须机器可判定或具名负责

能够自动判断的变化应自动失效；无法自动判断的部分必须有 Owner 和复审触发，不能依赖“大家记得”。

13.3 证据时效

时效不只由时间决定。以下二者都可能成立：

- 一年前针对不可变数学模型的证明仍有效；
- 昨天在错误配置上跑出的性能测试已经无效。

因此必须结合对象、环境和变化判断，而不是统一设置固定天数。

14. Verification Plan

14.1 从 Claim 反推验证

Verification Plan 应按以下顺序设计：

```
Claim
-> Violation
-> Mechanism
-> Failure / Input Space
-> Verification Method
-> Oracle
-> Evidence
-> Runtime Revalidation
```

14.2 验证方法选择

| 目标 | 适合方法 |
|---------|---|
| 类型和结构约束 | 编译器、类型系统、Schema、静态分析 |
| 广泛输入性质 | Property-based、fuzz、mutation |
| 并发和协议 | 模型检查、线性化测试、故障注入 |
| 边界契约 | Contract、integration、mixed-version test |
| 资源上界 | 压力、soak、限流、故障降容测试 |
| 恢复 | crash-window、restore、replay、DR 演练 |
| 安全 | Threat Model、权限负向测试、扫描、渗透 |
| 生产适用性 | canary、shadow、runtime invariant、SLI |

ASSUR-REQ-029: Oracle 必须明确

每项动态验证必须说明如何判断对错。只运行负载而没有正确性 Oracle，只能生成性能观察，不能证明业务性质。

ASSUR-REQ-030: 关键失败窗口必须枚举

对不可逆副作用、提交和 ACK，至少在每个持久化或外部副作用边界前后注入失败，验证恢复和重复语义。

ASSUR-REQ-031: 性能 Evidence 必须使用代表性分布

平均大小和理想输入不足以支撑容量 Claim。应覆盖：

- p50/p95/p99 与硬上限；
- 热点、偏斜和 burst；
- 依赖变慢；
- N-1 或降容；
- 积压恢复；
- retry amplification；
- 目标运行时和硬件。

14.3 负向验证

验证必须尝试推翻 Claim，而不是只证明 happy path。可使用：

- fault injection；
- fuzzing；
- mutation testing；
- invalid/unknown input；
- mixed versions；
- dependency slowdown；
- permission denial；
- clock skew；
- capacity exhaustion；
- operator error；
- stale configuration。

15. 工具与验证器可信度

15.1 工具是保证链的一部分

编译器、测试框架、策略引擎、静态分析器、模型检查器、CI runner 和报告聚合器都可能失败。必须评估：

- 工具适用范围；
- 版本和配置；
- 已知缺陷；
- 失败语义；
- 输出完整性；
- 是否可能被项目绕过；
- 是否需要资格确认或交叉验证。

ASSUR-REQ-032：工具错误必须显式

工具不可用、超时、解析失败或结果不完整必须产生 ERROR / UNKNOWN，不得因“基础设施问题”被转换为通过。

ASSUR-REQ-033：高风险验证规则变更必须受保护

改变测试 Oracle、静态规则、门禁策略、允许列表或报告聚合逻辑可能改变事实裁决，必须经过独立 Review 和回归证据。

15.2 工具资格确认

R3 或高后果性质应考虑：

- 用已知正反样例验证工具；
- 使用两个独立机制交叉检查；
- 固定工具版本和配置；
- 记录工具已知不支持特性；
- 对自研验证器进行单独测试和变更控制；
- 防止项目代码修改验证逻辑后自行获益。

ASSUR-REQ-034：验证器与被验证对象不得共享未经审查的同一错误假设

例如生成器和验证器使用同一错误解析函数，会产生伪独立 Evidence。应尽量使用独立 Oracle、参考实现或差分验证。

16. 覆盖、代表性与采样

16.1 覆盖维度

Coverage 至少可以包括：

```
requirements
states and transitions
inputs and distributions
fault windows
versions and combinations
tenants / regions
operating modes
resource boundaries
security actors
recovery paths
```

“代码覆盖率 90%” 只覆盖其中一个弱代理维度。

ASSUR-REQ-035：阻断级 Claim 必须有 Coverage Argument

Coverage Argument 必须说明为何已选择的场景足以支撑 Claim，以及哪些仍未覆盖。

16.2 代表性

代表性应基于真实或有依据的目标分布，包括：

- 生产 telemetry；
- 历史事故；
- 数据样本分析；
- 明确容量预测；
- 合同或业务上限；
- 攻击模型；
- 故障域。

只挑选最干净样本会高估系统能力。

16.3 采样边界

生产采样 Evidence 必须说明：

- 采样率；
- 漏失偏差；
- 高基数控制；
- 时钟和聚合误差；
- 隐私处理；

- 异常数据是否被过滤；
- 结果能否代表全量。

ASSUR-REQ-036：未观测到违反不等于违反不存在

运行指标没有出现错误，只能支持“在当前观测能力和样本中未发现”，不能证明全局 Safety。需要结合 invariant、审计、负向测试和检测覆盖。

17. 组合保证

17.1 组合问题

局部组件 Evidence 组合时常见断裂：

- 版本不同；
- 配置不同；
- 上下游 Assumption 不匹配；
- 边界错误语义未测试；
- 单组件容量足够但端到端排队失稳；
- mixed-version deployment 未覆盖；
- 共享依赖导致故障相关性；
- 局部幂等不等于端到端 exactly-once effect。

17.2 组合检查

```
For each boundary A -> B:  
  Guarantee(A) satisfies Assumption(B)  
  Error semantics are compatible  
  Capacity and backpressure are compatible  
  Identity and version are preserved  
  Failure correlation is modeled  
  Combined evidence refers to the same release composition
```

ASSUR-REQ-037：Release Composition 必须唯一标识

Evidence Bundle 必须说明服务、配置、Schema、模型、依赖和环境的完整组合。不能将不同时间、不同候选版本的最佳结果拼成一份虚构发布证据。

ASSUR-REQ-038：混合版本窗口必须进入论证

滚动升级、回滚和灰度期间的版本组合是生产真实状态，必须明确兼容保证和 Evidence。

18. Assurance Case 充分性

18.1 充分性不是绝对证明

“充分”表示：在当前风险等级、Claim 关键性、对象范围和决策类型下，Argument 和 Evidence 足以支持授权主体作出有边界的决策。

18.2 最低判据

一个 Decision-grade Assurance Case 至少满足：

- Top-level Claim 具有类型匹配的验证义务与违反语义；
- Claim 分解覆盖主要状态、边界和故障；
- 关键 Assumption 分别登记验证、生命周期和监测状态；监测不替代验证，证据不足显式保持未知；
- Evidence 与当前对象绑定；
- 证据质量和代表性与风险匹配；
- Counterevidence 和 Defeater 已处理；
- 组合边界已论证；
- Evidence 当前有效；
- 剩余风险、未知和例外明确；
- Decision Authority 在授权范围内。

18.3 按风险等级

| 风险 | Assurance 最低深度 |
|----|---|
| R0 | 可逆实验假设、数据边界、硬上限和退出证据 |
| R1 | 关键 Claim、基础动态 Evidence、Owner Review、生产观察 |
| R2 | 完整 Assurance Matrix、多样 Evidence、代表性环境、故障与恢复验证 |
| R3 | 独立评价、Defeater Register、工具资格、组合保证、持续漂移与严格证据完整性 |

ASSUR-REQ-039：不得用总分掩盖阻断项

Assurance Case 不得通过加权平均、总体通过率或“风险分数低”覆盖单个未解决的阻断级 Safety、Security 或 Recovery Claim。

19. Evidence Conclusion 与 Claim Status

19.1 Claim Status

建议使用：

| 状态 | 含义 |
|-------------------------|--|
| SUPPORTED | 当前 Argument 与 Evidence 足以支持声明范围内的 Claim |
| CONDITIONALLY_SUPPORTED | 仅在明确限定的前提、对象或范围内有充分 Argument 与 Evidence 支持；该限定须作为 Claim 的可审计组成部分 |
| CONTRADICTED | 存在足以反驳 Claim 的 Evidence |
| UNKNOWN | Evidence 或 Argument 不足 |
| NOT_APPLICABLE | 经论证与当前对象无关 |

Exception 只授权接受偏离/风险，不增加事实支持；Change Envelope 只限制暴露，也不自动补齐证据。缩小范围或增加前提以挽救主张时，必须生成明确的 Claim 修订版本或新 ID，重新绑定 Evidence，并保留原 Claim 的状态与历史反证。

三个维度必须分别记录：Claim 支持状态（本章）、要求满足事实与 Exception 授权（GOV §10）、发布裁决（GOV §8）。它们不存在自动升格映射。

反例校准（说明性，非真实演练记录）：原 Claim 声明“恢复 ≤30 分钟”，适用演练结果为 45 分钟。即使有效 Exception 接受了风险，原 Claim 仍为 CONTRADICTED；要求满足事实仍为 NONCONFORMANT。授权范围允许时 Gate 可另判 CONDITIONAL_GO，不可豁免或超出授权时为 NO_GO。若改为“恢复 ≤60 分钟”，须显式修订 Claim，并审查 45 分钟 Evidence 是否足以支持其完整作用域，而非仅比较两个数字。

19.2 状态传播

| |
|---|
| Blocking subclaim CONTRADICTED -> parent claim cannot be SUPPORTED |
| Blocking subclaim UNKNOWN -> parent claim usually UNKNOWN |
| Evidence INVALIDATED -> dependent claim recomputed |
| Assumption validation_status VIOLATED -> reassess dependent claim and its scope |

假设失效可能使原论证不再适用，但并非自动构成所有依赖 Claim 的反例。已有适用反证时保留 CONTRADICTED；只有不能支持且不能反驳时才记 UNKNOWN。阻断级 Defeater 被风险接受不等于已被反驳或消除，不能因此提高 Claim 支持状态。

ASSUR-REQ-040：父 Claim 状态必须可解释

任何聚合结论必须能够追溯到子 Claim、Evidence 和传播规则。不得只显示绿色总览而隐藏阻断来源。

20. Gate 与发布裁决

20.1 Evidence Review 输出

Assurance Reviewer 应输出：

```
assurance_review:
  review_id: "AR-..."
  subject_identity: "..."
  claims_reviewed: []
  supported: []
  conditional: []
  contradicted: []
  unknown: []
  stale_or_invalid_evidence: []
  open_defeaters: []
  residual_risks: []
  conclusion: "DECISION_GRADE|NOT_DECISION_GRADE|PARTIALLY_DECISION_GRADE"
  scope: "..."
  reviewer: "..."
  reviewed_at: "..."
```

20.2 Gate 决策

Gate 决策必须引用 Assurance Review，而不是复制测试列表。Decision Authority 负责接受剩余风险，Assurance Reviewer 负责评价证据充分性，两者职责不同。

ASSUR-REQ-041：Assurance Reviewer 不自动拥有风险接受权

证据评价结论 DECISION_GRADE 只表示可以作出决策，不表示必须 GO。业务、运营、安全和残余风险仍由授权 Decision Authority 裁决。

ASSUR-REQ-042：Decision Authority 不得改写证据事实

Decision Authority 可以在授权范围接受风险或批准 Exception，但不能把 FAIL、CONTRADICTS 或失效 Evidence 修改为 PASS，也不能据此把 CONTRADICTED/UNKNOWN Claim 改为 SUPPORTED/CONDITIONALLY_SUPPORTED。证据事实、Claim 支持状态、要求满足事实、Exception 授权与 Gate 决策必须分开记录并关联。

21. Release Evidence Bundle

21.1 目标

Release Evidence Bundle 是对一次具体生产候选的证据封装。它必须可独立回答：

- 发布的究竟是什么；

- 依赖哪些配置和外部版本；
- 哪些 Claim 得到支持；
- 哪些未知和例外仍存在；
- 证据从哪里产生；
- 允许多大生产暴露；
- 什么情况立即停止。

21.2 最低内容

```
release-manifest.yaml
subject-identity.json
claim-status.json
assurance-case.md or structured representation
evidence-index.json
verification reports
capacity and recovery evidence
security evidence
exceptions.yaml
residual-risks.md
change-envelope.yaml
approvals.yaml
sha256sums.txt
```

ASSUR-REQ-043: Bundle 必须防止证据混淆

Bundle 必须通过 manifest 和 digest 关联所有关键文件。不能从多个候选中挑选最优结果后组成发布包。

ASSUR-REQ-044: Bundle 必须包含失败和限制

失败测试、被接受的 Defeater、UNKNOWN、Exception 和未覆盖范围必须进入 Bundle，而不是只包含通过材料。

ASSUR-REQ-045: Bundle 必须可长期验证

至少应保留：

- 生成时间；
- 工具和环境身份；
- 原始 Artifact digest；
- Evidence digest；
- 签名或不可篡改证明；
- 决策主体；
- 保留策略。

22. Change Envelope

22.1 Assurance 视角

Change Envelope 将 Evidence 覆盖转化为生产暴露边界：

```
change_envelope:
  subject_identity: "...
  population: "...
  traffic_or_data_scope: "...
  max_duration: "...
  max_concurrency_or_bytes: "...
  preconditions: []
  success_criteria: []
  observation_signals: []
  abort_conditions: []
  rollback_or_forward_recovery: "...
  owner: "...
  authority: "...
```

ASSUR-REQ-046：暴露不得超出 Evidence 范围

如果容量只在某节点规格和 $2\times$ 峰值下验证，Change Envelope 不得直接授权更大规模。扩大范围须补足相应 Evidence；若按 GOV 授权程序临时接受未覆盖风险，应另记 Exception、未知范围和条件裁决，不得声称扩大范围内的 Claim 已获支持。

ASSUR-REQ-047：停止条件必须对应关键性质

Canary 的自动停止条件应包括关键正确性、资源和安全违反，而不只是错误率或延迟。

23. 运行时持续保证

23.1 Runtime Evidence

运行时可以提供：

- invariant violation;
- queue depth/age;
- resource bound;
- duplicate/conflict;
- provenance mismatch;
- assumption drift;
- error budget;
- recovery progress;

- permission/audit event;
- mixed-version state。

ASSUR-REQ-048: 运行监测必须映射到 Claim 或 Assumption

“监控很多”不是持续保证。每个关键运行信号应说明它验证、挑战或失效哪个 Claim/Assumption, 以及触发什么动作。

23.2 Drift

Drift 可以来自:

```
artifact drift
configuration drift
environment drift
dependency drift
workload drift
schema/model/reference-data drift
organizational ownership drift
```

ASSUR-REQ-049: Drift 必须触发结论降级

一旦运行对象超出 Evidence Envelope, 相关 Claim 不能仅凭旧 Evidence 继续显示 SUPPORTED。系统应自动或由 Owner 按 §19 重评: 不足以支持且无适用反证时为 UNKNOWN, 已有适用反证时为 CONTRADICTED, 并触发再验证或限制暴露。

23.3 Continuous Assurance

持续保证不是每次提交都执行所有昂贵验证, 而是建立分层策略:

```
per-change deterministic checks
per-merge integration checks
scheduled system / fault / recovery checks
pre-release decision-grade evidence
runtime drift and invariant monitoring
incident-triggered revalidation
```

24. Incident → Assurance Delta

24.1 事故作为反证

事故和近失事件必须被视为 Counterevidence, 而不仅是运维事件。它可能表明:

- Claim 定义错误;
- Assumption 未显式;
- Failure Model 不完整;

- Mechanism 无效；
- Verification 漏掉关键窗口；
- Evidence 代表性不足；
- Gate 或 Exception 失效；
- 运行监测无法及时检测。

ASSUR-REQ-050：重大事故必须产生 Assurance Delta

至少改变以下一项：

```
Claim / Property
Assumption
Argument
Mechanism
Verification Activity
Evidence Requirement
Runtime Signal
Exception / Authority
Runbook / Recovery
Standard / Profile
```

24.2 Delta 结构

```
assurance_delta:
  incident_id: "...
  affected_claims: []
  counterevidence: []
  invalidated_evidence: []
  newly_discovered_assumptions: []
  argument_gaps: []
  required_changes: []
  owners: []
  closure_evidence: []
  standard_impact: "...
```

ASSUR-REQ-051：事故关闭不得只依赖文字提醒

如果没有改变控制、验证、证据、权限、容量或恢复能力，系统通常没有获得可验证的抵抗力提升。

25. Assurance Plane 元不变量

ASSUR-REQ-052：缺失或错绑 Evidence 不得为通过

这是保证面的最高元不变量。

ASSUR-REQ-053：用于裁决的 Evidence 不得静默修改

修改必须产生新 Evidence ID 和完整历史。

ASSUR-REQ-054：门禁规则和 Exception 逻辑必须版本化

每次裁决必须能够说明使用了哪一版规则。

ASSUR-REQ-055：高风险裁决必须保留职责分离

Proposer、Evaluator 和 Final Approver 的独立性必须符合 ESD-GOV-001。

ASSUR-REQ-056：Assurance Plane 不可用不得默认放行

对 R2/R3，验证服务、策略引擎、Evidence Store 或身份链不可用时，默认结果为 UNKNOWN / NO RELEASE QUALIFICATION。

ASSUR-REQ-057：例外必须进入 Evidence Bundle

不得在 Bundle 之外通过口头、聊天或临时配置隐藏偏离。

ASSUR-REQ-058：运行漂移必须使证据结论失效

不能让监控发现对象已变化，而发布面仍显示历史绿色状态。

26. 角色与独立性

26.1 角色

| 角色 | 责任 |
|--------------------|--------------------------------|
| Claim Owner | 定义并维护 Claim 及违反语义 |
| Evidence Producer | 执行验证并生成原始 Evidence |
| Evidence Custodian | 保证存储、完整性、保留和访问 |
| Assurance Author | 构建 Argument 和 Evidence 映射 |
| Assurance Reviewer | 挑战充分性、Defeater、覆盖和时效 |
| Decision Authority | 接受剩余风险并作 Gate 决策 |
| Tool Owner | 维护验证器能力、版本和已知限制 |
| Operational Owner | 维护 Runtime Evidence 和 drift 响应 |

ASSUR-REQ-059：Evidence Producer 与 Reviewer 的独立性风险匹配

R3 阻断级 Claim 不应仅由实现作者自产生并自评价全部 Evidence。可以共享技术知识，但必须存在独立挑战和最终批准。

ASSUR-REQ-060：Assurance Author 不得隐藏不利材料

Assurance Case 必须包含 open Defeater、Counterevidence、已知缺陷和 Evidence 限制。

27. 常见反模式

27.1 Test List as Assurance Case

症状：列出数百个测试，却没有 Claim、作用域和覆盖论证。

校正：从阻断级 Claim 反推验证与 Evidence。

27.2 Green Dashboard Fallacy

症状：所有 CI 绿色，因此认为系统整体安全。

校正：明确哪些性质未编码、哪些 Evidence 已过期或局部化。

27.3 Evidence Shopping

症状：从多个候选中选最优性能报告，忽略失败候选。

校正：Bundle 绑定唯一 Release Composition，并保留失败历史。

27.4 Old Evidence, New System

症状：用旧环境或旧版本测试支持当前发布。

校正：Subject Binding 和自动失效。

27.5 Local Pass, Global Claim

症状：组件测试通过，被提升为端到端 exactly-once。

校正：Assume–Guarantee 与组合 Evidence。

27.6 No Incident Means Safe

症状：线上无事故被解释为性质已证明。

校正：评估检测覆盖、样本量、暴露和未观测风险。

27.7 Exception Hidden Outside the Case

症状：正式报告全绿，实际风险只存在聊天记录。

校正：Exception、Unknown 和 Residual Risk 必须进入 Bundle。

27.8 Tool Authority Inflation

症状：把静态分析器、LLM Reviewer 或策略引擎当作绝对仲裁者。

校正：声明工具覆盖、已知限制、失败语义和人工挑战边界。

27.9 Confidence Score Theater

症状：用 92% “可信度” 掩盖阻断级未知。

校正：逐 Claim 状态和 Defeater；不以总分覆盖硬阻断。

27.10 Evidence Never Expires

症状：一次压测长期支撑任意规模和环境。

校正：Evidence Envelope、valid range 和 drift invalidation。

28. Assurance Case 模板

```
# Assurance Case AC-XXX
```

```
## 1. Subject
```

- System / Change:
- Risk tier:
- Source commit:

字面文本 · continued

- Artifact digest:

- Config digest:

- Environment:

- Change Envelope:

2. Top-level Claim

CLM-001

- Statement:

- Type:

- Scope:

- Temporal semantics:

- Assumptions:

- Violation:

- Criticality:

- Owner:

3. Argument Strategy

Explain why the following subclaims are jointly sufficient.

4. Subclaims

| | | | | |
|----------|-----------|--------|----------|-----------|
| Claim ID | Statement | Status | Evidence | Defeaters |
| --- | --- | --- | --- | --- |

5. Assumptions

| | | | | | |
|---------------|----------------|--------------------|-----------|------------|-----------|
| Assumption ID | Statement | Validation | Lifecycle | Monitoring | Risk Flag |
| └ Evidence | Runtime Signal | Violation Response | | | |
| --- | --- | --- | --- | --- | --- |

6. Mechanisms

7. Verification Activities

8. Evidence Index

| | | | | | |
|-------------|------|------------|-----------|-----------------|-------------|
| Evidence ID | Type | Conclusion | Lifecycle | Subject Binding | Limitations |
| --- | --- | --- | --- | --- | --- |

9. Counterevidence and Defeaters

10. Coverage Argument

11. Composition Argument

| |
|---|
| 字面文本 · continued |
| ## 12. Residual Risks and Unknowns |
| ## 13. Assurance Review |
| ## 14. Decision and Scope |
| ## 15. Invalidation and Review Triggers |

29. Verification & Evidence Matrix 模板

下列行仅示范字段关联，不代表存在已执行测试或当前有效证据；项目须以真实记录替换。

表 16 · 字段记录视图（列名与原单元格逐项对应）

| | |
|----------------------|------------------------------|
| 表 16 · 记录 1 · CP-001 | |
| Property / Claim | CP-001 |
| Mechanism | Conditional commit |
| Verification | Crash-window test |
| Oracle | Single manifest per identity |
| Environment / Corpus | Target-like |
| Evidence ID | EVD-001 |
| Conclusion | SUPPORTS |
| Lifecycle | CURRENT |
| Limitations | No region partition |
| Owner | Team A |

30. Evidence Review Checklist

Claim

- ☐ Claim 的验证义务与 Safety、活性或统计性质匹配；
- ☐ 主体、身份、版本、模式和时间语义明确；
- ☐ 违反条件和后果明确；
- ☐ Criticality 与风险匹配；
- ☐ Assumption 和允许降级已列出。

Argument

- ☐ 分解策略合理；
- ☐ 子 Claim 共同足以支持父 Claim；
- ☐ 状态、边界、故障和混合版本未遗漏；
- ☐ Defeater 和替代解释已挑战；
- ☐ 组合保证成立。

Evidence

- ☐ 与当前对象不可混淆地绑定；
- ☐ 方法与 Claim 相关；
- ☐ Oracle 明确；
- ☐ 代表性和覆盖可接受；
- ☐ 原始 Artifact 可访问并校验；
- ☐ 工具版本和限制明确；
- ☐ 生命周期为 CURRENT；
- ☐ Counterevidence 未被隐藏。

Decision

- ☐ 阻断级未知未被平均分掩盖；
- ☐ Assurance Reviewer 与 Decision Authority 职责分开；
- ☐ Residual Risk 有授权 Owner；
- ☐ Change Envelope 不超出 Evidence 范围；
- ☐ 结论有失效规则和复审触发。

31. Release Evidence Bundle Manifest 示例

```
release_evidence_bundle:
  bundle_id: "REB-2026-001"
  suite_version: "<adopted-suite-version>"
  subject_identity:
    source_commit: "...
    artifact_digest: "...
    config_digest: "...
    environment: "...
  assurance_case: "AC-001"
  claims:
    supported: []
    conditional: []
```

字面文本 · continued

```
contradicted: []
unknown: []
evidence_index: "evidence-index.json"
exceptions: []
residual_risks: []
change_envelope: "change-envelope.yaml"
reviewers: []
decision_authority: "..."/>

```

32. 最终判定标准

一个 Assurance Case 只有在能够回答以下问题时，才具备支持高质量裁决的基础：

1. 主张针对哪个不可混淆对象？
2. 主张到底声称什么，需要哪类验证、反驳或统计评价，观测边界在哪里？
3. 主张依赖哪些 Assumption，它们当前是否成立？
4. 论证如何把系统级主张分解为可验证子主张？
5. 机制为何能够维护这些性质？
6. 验证是否覆盖关键状态、边界、故障和混合版本？
7. Evidence 是否代表目标环境和真实工作负载？
8. Evidence 是否绑定当前制品、配置和依赖？
9. 工具、Oracle 和验证器是否可信且失败语义明确？
10. 有哪些 Counterevidence、Defeater 和已知限制？
11. 局部 Evidence 如何组合成端到端保证？
12. 哪些 Claim 仍为 UNKNOWN，是否阻断？
13. 谁评价 Evidence，谁接受 Residual Risk？
14. Change Envelope 是否超出已验证范围？
15. 什么变化会使结论失效，系统如何检测该变化？
16. 事故发生后 Assurance Case 如何增长？

结论

工程保证的成熟标志不是“我们拥有更多绿色报告”，而是团队能够对每项关键结论给出一条诚实、可挑战、可失效、可复现的推理链。真正可信的 Assurance Case 会同时展示支持、限制、反证和未知，并把最终决定严格绑定到当前对象和授权范围。

生产就绪与运行韧性标准

从可部署到可运行、可退化、可恢复、可演进的生产准入基线

| 文档属性 | 内容 |
|-------|------------------------------------|
| 文档编号 | ESD-OPS-001 |
| 版本 | 2.0.0-draft.1 (基于 1.0.0) |
| 状态 | DRAFT / NORMATIVE DOMAIN; 未获发布批准 |
| 规范角色 | 定义生产准入、SLO、运行模式、容量、发布、恢复、值班和持续韧性要求 |
| 方法论基础 | ESD-METHOD-001 《优秀系统设计与工程保证方法论》 |
| 治理基础 | ESD-GOV-001 《系统设计与工程保证治理及符合性规范》 |
| 证据基础 | ESD-ASSURANCE-001 《工程保证论证、证据与裁决规范》 |
| 权威格式 | 本 Markdown 文件; 派生 PDF 或门户页面不构成独立规范 |

规范目的

本文用于判断一个系统是否不仅“能够上线”，而且能够在真实生产中被观察、控制、退化、恢复、迁移和持续运营。生产就绪不是部署流水线成功，而是当前系统、团队和证据共同具备在声明风险边界内承受正常负载、故障、过载、变更和事故的能力。

摘要

大多数系统在正常路径上都能运行。真正区分生产工程质量的，是系统在以下场景中的行为：

- 依赖变慢
- 流量突增
- 队列积压
- 节点丢失
- 发布失败
- 数据迁移半完成
- 写入成功但响应超时
- 凭据轮换
- 误操作
- 区域故障
- 恢复期间再次故障

本标准将生产就绪定义为一个多维条件，而不是单一 Gate：

Correctness and Safety

- + Operability
- + Bounded Resources
- + Capacity and Recovery Headroom
- + Controlled Change
- + Data Recovery
- + Observability and On-call
- + Evidence-backed Readiness

系统必须有显式的运行模式、服务目标、故障域、依赖契约、资源上界、发布与停止条件、恢复权威、Runbook、Owner 和当前 Evidence。任何关键能力无法证明时，应保持 UNKNOWN；任何无法接受的高后果违反应为 NO_GO。

1. 适用范围与非目标

1.1 适用范围

本标准适用于：

- 新系统首次进入生产；
- 核心系统重大版本、架构或运行环境变更；
- 新区域、新租户、新数据类型或显著流量扩展；
- 数据迁移、Schema 变更、权威状态移动；
- SLO、RPO、RTO、容量或故障域变化；
- 新增外部依赖、特权控制或自动化决策；
- 恢复、重放、灾难恢复和备份体系变更；
- 改变 on-call、Owner、告警或 break-glass 路径；
- 事故后对生产资格的重新评估。

1.2 非目标

本文不：

- 规定所有服务必须达到同样 SLO；
- 要求零故障、零风险或全自动恢复；
- 以监控项数量替代可诊断性；
- 以“有备份”替代恢复演练；
- 以“支持回滚”替代数据和协议层面的可逆性；
- 让 SRE 或平台团队替代系统 Owner 的业务责任；
- 把所有运行复杂度都转移给值班人员。

2. 核心生产原则

OPS-REQ-001：生产完成语义必须明确

系统必须定义什么状态才算业务完成，以及该状态由哪个权威事实表示。进程返回成功、消息 ACK、HTTP 200、对象写入或监控绿色都不能在未定义时自动等同于业务完成。

OPS-REQ-002：运行模式必须显式

生产系统至少应识别适用的运行模式，例如：

```
NORMAL
DEGRADED
READ_ONLY
DRAINING
RECOVERY
QUARANTINE
MAINTENANCE
EMERGENCY
```

每种模式必须定义允许操作、禁止操作、进入/退出条件、Owner 和可观察信号。

OPS-REQ-003：所有资源与积压必须有界

线程、任务、连接、队列、缓存、inflight bytes、重试、临时文件、staging、待补偿工作和隔离积压必须具有可执行的硬上限，以及满载时的准入、背压、拒绝或安全转储行为。仅声明 retention 或“定期清理”不能证明空间有界。

若使用保留窗口推导空间预算，必须同时给出最大入站速率 $\lambda_{\max}$ 、单项最大尺寸 $s_{\max}$ 、保留时间 T 与清理最大延迟 Δ ，再计入元数据、副本、重试副本、在途写入及清理失败积压：

$$B_{\text{required}} \geq \lambda_{\max} \times s_{\max} \times (T + \Delta) + B_{\text{overhead}} + B_{\text{backlog}} + B_{\text{inflight}}$$

这是给定前提下的预算估计，不是自动成立的容量上界。若清理延迟或入站率无法硬约束，必须由容量配额与 admission 在达到硬上限前阻断增长；不得为了清理而删除仍承担提交、对账或恢复责任的数据。

OPS-REQ-004：过载必须显式处置

系统必须选择并实现可观察的准入、背压、降级、负载卸载、拒绝或排队策略。持续过载不得只依赖“自动扩容最终会跟上”。

OPS-REQ-005：恢复能力必须经过验证

备份、Runbook、回滚脚本或多副本只有在代表性故障和目标规模下完成演练，才能作为恢复 Evidence。

OPS-REQ-006：发布是受控实验

重大生产变更必须定义 Change Envelope、前置条件、观察窗口、成功标准、自动停止条件和恢复路径。

OPS-REQ-007：可观测性必须支持行动

每个阻断级告警应能够回答：发生了什么、影响什么、谁负责、下一步做什么。无法指导行动的通知不构成有效运行控制。

OPS-REQ-008：Owner 必须接受 Day-2 责任

在生产准入前，Operational Owner 必须明确接受 SLO、容量、告警、Runbook、on-call、恢复、依赖协调和事故改进责任。

OPS-REQ-009：生产资格必须绑定当前对象

Production Readiness Decision 必须绑定当前 artifact、config、environment、dependency composition 和 Evidence Bundle，不能永久授权未来任意版本。

OPS-REQ-010：运行漂移必须降低确定性

当配置、环境、工作负载、依赖、Owner 或证据超出已验证范围时，生产资格和相关 Claim 必须重新评估；不能继续显示历史 GO 而忽略现实变化。

3. 生产服务定义

3.1 Service Definition

每个生产系统必须拥有最小服务定义：

```
service:
  service_id: "...
  business_capability: "...
  users_or_consumers: []
  business_completion: "...
  authoritative_states: []
  critical_dependencies: []
  critical_properties: []
  supported_operating_modes: []
  risk_tier: "R1|R2|R3"
  technical_owner: "...
  operational_owner: "...
  on_call: "...
  data_classification: "...
  regions_or_failure_domains: []
  lifecycle_status: "...
```

OPS-REQ-011：服务边界必须以能力和状态定义

服务定义不得只是一组部署名称。必须说明它对谁提供什么业务能力、拥有或依赖哪些权威状态，以及失败如何表现。

3.2 用户旅程和关键路径

SLO、告警和事故优先级应映射到用户或业务关键路径，例如：

```
request accepted
  -> durable command recorded
  -> processing completed
  -> result committed
  -> result visible to consumer
```

基础设施指标必须能关联到该路径，而不是独立存在。

3.3 生产依赖清单

每个关键依赖至少记录：

| 字段 | 内容 |
|--------------|--------------------------|
| Dependency | 服务、存储、网络、身份、Schema、人员流程等 |
| Owner | 依赖责任主体 |
| Contract | 成功、错误、超时、容量、版本和支持边界 |
| Criticality | 阻断、可降级、可缓存、可绕行 |
| Failure mode | 慢、不可用、错误、旧数据、权限拒绝等 |
| Detection | 如何识别依赖异常 |
| Fallback | 降级、隔离、重试、缓存或停止 |
| Recovery | 谁恢复、多久、如何验证 |

OPS-REQ-012：依赖必须包含非软件主体

DNS、证书、密钥、配置、人员审批、供应商、配额、网络 and 运行手册都可能是关键依赖，不得只列服务调用图。

4. 生产就绪状态模型

4.1 生命周期状态

建议使用：

```
DESIGN
-> PRE_PRODUCTION
-> LIMITED_PRODUCTION
-> GENERAL_PRODUCTION
-> DEPRECATED
-> RETIRED
```

另有横向状态：

```
READINESS_UNKNOWN
READINESS_SUSPENDED
RECOVERY_ONLY
```

4.2 状态含义

| 状态 | 含义 |
|---------------------|----------------------------------|
| PRE_PRODUCTION | 尚未承载正式业务结果 |
| LIMITED_PRODUCTION | 仅在 Change Envelope 内承载有限范围 |
| GENERAL_PRODUCTION | 具备声明范围内的常规生产资格 |
| READINESS_SUSPENDED | 因事故、漂移、证据失效或 Owner 缺失暂时失去扩展/发布资格 |
| RECOVERY_ONLY | 只允许恢复、对账、排空或必要维修操作 |
| DEPRECATED | 禁止新依赖接入，维持退出期运行 |
| RETIRED | 生产流量、状态和权限已安全移除 |

OPS-REQ-013：生产资格可以撤销

GENERAL_PRODUCTION 不是永久属性。重大事故、证据失效、长期 SLO 违约、恢复能力丧失或 Owner 缺失可以将系统降为 READINESS_SUSPENDED。

OPS-REQ-014：扩大暴露必须重新检查资格

从单租户到多租户、单区域到多区域、低流量到大规模、只读到写入或 shadow 到 authoritative output 都属于新的暴露边界，不应被历史资格自动覆盖。

5. SLI、SLO 与错误预算

5.1 SLI 设计

SLI 必须基于用户或业务结果，至少说明：

```
what is measured
numerator / denominator
source of truth
aggregation window
scope
exclusions
late / missing data behavior
measurement delay
owner
```

OPS-REQ-015：SLI 必须测量结果而不只是资源

CPU、内存和 pod availability 是诊断信号，不应单独作为服务可用性 SLI。应优先测量成功完成、正确可见、延迟、完整性或新鲜度。

5.2 SLO Contract

```
slo:
  slo_id: "SLO-..."
  user_journey: "..."
  sli: "..."
  objective: "..."
  window: "..."
  scope: "..."
  exclusions: []
  measurement_source: "..."
  owner: "..."
  error_budget_policy: "..."
  review_trigger: "..."
```

OPS-REQ-016: SLO 与正确性性质不得混写

99.9% 成功率不能允许 0.1% 的资金重复扣款或权限越界。硬 Safety 性质必须独立于统计 SLO。

5.3 错误预算

错误预算策略至少定义：

- 消耗速率；
- 多窗口 burn-rate；
- 何时限制发布；
- 何时转入稳定性工作；
- 哪些错误不允许预算化；
- 异常测量或数据延迟如何处理。

OPS-REQ-017: 错误预算不得覆盖硬属性违反

重复正式结果、数据损坏、越权写入、审计丢失等硬属性违反，即使数量很少，也应按性质处理，而不是纳入普通错误预算。

5.4 新鲜度与积压 SLO

数据和异步系统还应定义：

- end-to-end lag；
- oldest queue age；
- completion freshness；
- replay catch-up time；
- backlog clearance time；

- stale data exposure。

6. 运行模式

6.1 Operating Mode Contract

表 4 · 字段记录视图（列名与原单元格逐项对应）

| | |
|---------------------|---------------|
| 表 4 · 记录 1 · NORMAL | |
| Mode | NORMAL |
| Allowed | 正常读写和发布 |
| Forbidden | 绕过控制 |
| Entry | 所有前置满足 |
| Exit | 依赖/资源异常 |
| Signals | SLO、invariant |
| Authority | Operator |

| | |
|-----------------------|------------------|
| 表 4 · 记录 2 · DEGRADED | |
| Mode | DEGRADED |
| Allowed | 有界降级 |
| Forbidden | 未声明语义变化 |
| Entry | 依赖或容量下降 |
| Exit | 恢复验证 |
| Signals | degradation flag |
| Authority | Owner |

| | |
|------------------------|------------|
| 表 4 · 记录 3 · READ_ONLY | |
| Mode | READ_ONLY |
| Allowed | 查询和恢复 |
| Forbidden | 新写入 |
| Entry | 写路径不安全 |
| Exit | 写路径验证 |
| Signals | write gate |
| Authority | Authority |

| 表 4 · 记录 4 · DRAINING | |
|-----------------------|----------------|
| Mode | DRAINING |
| Allowed | 完成已接纳工作 |
| Forbidden | 新准入 |
| Entry | 发布/维护 |
| Exit | inflight=0 |
| Signals | queue/inflight |
| Authority | Operator |

| 表 4 · 记录 5 · RECOVERY | |
|-----------------------|-------------------------|
| Mode | RECOVERY |
| Allowed | replay/reconcile/repair |
| Forbidden | 非必要功能 |
| Entry | 故障后 |
| Exit | 恢复 Gate |
| Signals | recovery metrics |
| Authority | Incident/Owner |

| 表 4 · 记录 6 · QUARANTINE | |
|-------------------------|----------------|
| Mode | QUARANTINE |
| Allowed | 保存异常对象 |
| Forbidden | 猜测处理 |
| Entry | 输入未知/损坏 |
| Exit | 人工或新规则 |
| Signals | quarantine age |
| Authority | Owner |

| 表 4 · 记录 7 · EMERGENCY | |
|------------------------|-----------------------|
| Mode | EMERGENCY |
| Allowed | 最小保命操作 |
| Forbidden | 常规变更 |
| Entry | 严重事故 |
| Exit | 补偿复审 |
| Signals | audit trail |
| Authority | Break-glass Authority |

OPS-REQ-018：模式转换必须有状态机语义

模式切换不能只依赖口头通知。至少应有：

- 可观察状态；
- 进入条件；
- 幂等操作；
- 权限；
- 失败恢复；
- 审计；
- 退出验证。

OPS-REQ-019：降级不得静默改变业务事实

如果降级会返回旧数据、跳过校验、丢弃非关键字段、改变顺序或延迟完成，必须在契约和可观测性中显式表达。

OPS-REQ-020：恢复模式容量必须单独建模

恢复、replay 和 reconciliation 的负载通常不同于正常流量，不能直接使用正常模式容量结论。

7. 故障域与依赖韧性

7.1 Failure Domain

必须识别：

- 进程/容器；
- 节点；
- 机架/可用区；
- 区域；
- 存储分区；
- 网络/路由；
- 身份/密钥；
- 配置/控制面；
- 供应商；
- 人员和权限；
- 共用部署流水线。

OPS-REQ-021：副本必须跨真实故障域

同一节点、同一存储后端、同一凭据、同一配置错误或同一控制面下的多个副本不等于独立容灾。

7.2 Dependency Contract

关键依赖必须定义：

```
timeout
retryability
idempotency
capacity quota
rate limit
error semantics
stale / partial behavior
version compatibility
maintenance communication
recovery escalation
```

OPS-REQ-022：调用超时必须小于上游 deadline

下游 timeout、重试和排队时间必须共享端到端 deadline budget。层层独立超时会导致无意义的尾部延迟和工作放大。

OPS-REQ-023：重试必须有共享预算

重试策略至少包含：

- 可重试错误分类；
- 最大次数或总时间；
- jitter；
- retry budget；
- 幂等条件；
- 过载时抑制；
- 可观察性；
- 失败终态。

无限重试和所有错误统一重试均不符合本标准。

7.3 Circuit Breaker 与隔离

熔断器必须定义：

- 触发指标；
- 窗口；
- 半开探测；

- 失败语义；
- 与重试/扩容的相互作用；
- 恢复和人工覆盖。

隔离策略应考虑 bulkhead、租户配额、优先级和公平性，防止单一热点耗尽全局资源。

8. 资源边界、容量与恢复余量

8.1 Resource Budget

每个关键阶段至少记录：

表 5 · 字段记录视图（列名与原单元格逐项对应）

排版说明：横线表示原表空字段，不表示已有值或已完成填写。

| 表 5 · 记录 1 · inflight requests/files | |
|--------------------------------------|-------------------------|
| 资源 | inflight requests/files |
| 单位 | count |
| 正常上限 | |
| 故障上限 | |
| 满载行为 | reject/backpressure |
| Owner | |
| Signal | |

| 表 5 · 记录 2 · inflight bytes | |
|-----------------------------|----------------------|
| 资源 | inflight bytes |
| 单位 | bytes |
| 正常上限 | |
| 故障上限 | |
| 满载行为 | block/admission stop |
| Owner | |
| Signal | |

| 表 5 · 记录 3 · queue | |
|--------------------|-------------|
| 资源 | queue |
| 单位 | items/bytes |
| 正常上限 | |
| 故障上限 | |
| 满载行为 | shed/defer |
| Owner | |
| Signal | |

| 表 5 · 记录 4 · memory working set | |
|---------------------------------|--------------------|
| 资源 | memory working set |
| 单位 | bytes |
| 正常上限 | |
| 故障上限 | |
| 满载行为 | stop admission |
| Owner | |
| Signal | |

| 表 5 · 记录 5 · connections | |
|--------------------------|--------------|
| 资源 | connections |
| 单位 | count |
| 正常上限 | |
| 故障上限 | |
| 满载行为 | queue/reject |
| Owner | |
| Signal | |

| 表 5 · 记录 6 · retries | |
|----------------------|---------------|
| 资源 | retries |
| 单位 | attempts/time |
| 正常上限 | |
| 故障上限 | |
| 满载行为 | fail final |
| Owner | |
| Signal | |

| 表 5 · 记录 7 · staging | |
|----------------------|--------------------|
| 资源 | staging |
| 单位 | bytes/objects |
| 正常上限 | |
| 故障上限 | |
| 满载行为 | cleanup/quarantine |
| Owner | |
| Signal | |

OPS-REQ-024：容量必须以工作量而不只是请求数建模

如果 item 大小、fan-out、压缩率、模型复杂度或依赖调用差异显著，容量和准入必须使用 bytes、CPU work、cost units 或其他代表工作量的单位。

8.2 Capacity Model

最低模型：

| |
|---------------------------|
| arrival rate |
| size / work distribution |
| per-stage service demand |
| parallelism |
| stage capacity |
| queueing delay |
| headroom |
| failure-degraded capacity |
| recovery rate |
| cost per unit |

OPS-REQ-025：峰值和恢复必须分别计算

系统必须回答：

- 正常峰值是否稳定；
- N-1 或依赖降容时是否稳定；
- 故障后积压增长多快；
- 恢复后多久清空；
- 恢复流量是否压垮正常流量；
- 是否需要优先级或限速。

8.3 Headroom

Headroom 用于：

- 突发；
- 自动扩容延迟；
- 节点/区域丢失；
- 发布混合版本；
- replay/recovery；
- 依赖抖动；
- 估算误差。

平均利用率低不等于有恢复余量。

OPS-REQ-026：容量结论必须绑定环境和语料

压测结果必须声明节点规格、运行时、配置、数据分布、依赖行为和版本。环境变化后需要重新确认。

8.4 单位经济性

至少对 R2/R3 或高成本系统计算：

```
$/request  
$/GB processed  
$/tenant  
$/vehicle-hour  
$/replay  
$/recovery-hour
```

OPS-REQ-027：成本控制不得破坏硬性质

成本优化可以调整副本、保留、压缩和批量，但不能静默放宽完整性、安全、恢复和资源边界。

9. 准入、背压与过载

9.1 Admission Control

准入应考虑：

- 当前资源而不是只看实例数；
- 已接纳工作；
- 每租户公平性；
- 请求成本；
- deadline；

- 恢复和高优先级流量；
- 依赖健康；
- Change Envelope。

OPS-REQ-028：先限制入口，再限制内部损害

如果系统在内部队列增长后才发现过载，资源边界通常已经失效。准入应尽可能靠近可控入口，并与端到端 byte/task permits 联动。

9.2 Backpressure

背压链必须说明：

```
where pressure is detected
how it propagates
where work waits
maximum wait
what is rejected or shed
how upstream reacts
how operators observe it
```

OPS-REQ-029：队列满载行为必须确定

每个队列满载时只能执行预定义行为，例如：

- 阻塞；
- 拒绝；
- 降级；
- 丢弃允许丢弃的低优先级工作；
- 延迟准入；
- 隔离到受控存储。

不得退化为无界分配或无限重试。

9.3 Load Shedding

负载卸载必须明确：

- 哪些工作可丢弃；
- 对用户的可见语义；
- 公平性；
- 重试建议；
- 审计；
- 恢复后是否补偿。

OPS-REQ-030：不得随机丢弃关键业务事实

如果工作代表不可丢失命令、资金或审计事实，过载时应拒绝准入或持久化到有界权威队列，而不是静默丢弃。

9.4 控制回路

自动扩容、重试、限流、熔断、队列消费者和数据库连接池共同形成控制系统。必须评审：

- 采样延迟；
- 控制延迟；
- 过度响应；
- 振荡；
- 多回路相互放大；
- 冷启动；
- 配额硬限制。

10. 发布策略与 Change Envelope

10.1 发布前置

重大生产发布至少需要：

- 当前制品和配置身份；
- Production Readiness Decision；
- Release Evidence Bundle；
- Change Envelope；
- 版本兼容和迁移状态；
- 回滚或 forward-recovery 路径；
- Owner 和观察人员；
- 变更窗口与依赖协调；
- 自动停止条件。

OPS-REQ-031：发布流程必须支持停止

发布系统必须能够：

- 暂停新扩展；
- 停止准入；
- 回滚可逆制品；
- 转入安全模式；
- 保留证据；

- 触发人工升级。

只能“继续直到完成”的流水线不适合高风险变更。

10.2 Change Envelope

```
change_envelope:
  change_id: "...
  subject_identity: "...
  semantic_delta: "...
  population: "...
  max_traffic_or_data: "...
  max_duration: "...
  max_regions_or_tenants: "...
  preconditions: []
  success_criteria: []
  observation_window: "...
  abort_conditions: []
  rollback_or_forward_recovery: "...
  owner: "...
  release_authority: "...
```

OPS-REQ-032: Change Envelope 必须与 Evidence 范围一致

如果只验证了某类对象、某区域或某规模，首次暴露不得超出该范围。

10.3 Canary、Shadow 与 Dark Launch

Canary

用于有限权威流量，必须有正确性、资源、安全和 SLO 停止条件。

Shadow

可验证处理逻辑和容量，但不应产生权威副作用。必须防止影子路径意外写入或污染指标。

Dark Launch

可以提前部署依赖和资源，但未承载业务结果。不能替代真实流量下的语义验证。

OPS-REQ-033: Canary 不能只观察错误率

应至少观察：

- hard property violations;
- duplicate/conflict;

- lineage/provenance;
- queue age;
- memory/resource bounds;
- retry amplification;
- dependency impact;
- user outcome;
- unit cost。

10.4 Freeze 与高风险窗口

在依赖团队不可用、无恢复能力、重大业务高峰或证据过期时，应限制高风险发布。Freeze 应基于风险，不应成为长期替代工程能力的固定仪式。

11. 回滚、前向修复与不可逆变更

11.1 回滚类型

| 类型 | 内容 |
|------------------------|----------------|
| Binary rollback | 恢复旧二进制 |
| Configuration rollback | 恢复旧配置/策略 |
| Traffic rollback | 将流量移回旧路径 |
| Data rollback | 恢复旧数据状态，通常最危险 |
| Protocol rollback | 回到旧协议/Schema |
| Forward recovery | 保留新状态，通过修复继续前进 |

OPS-REQ-034：必须区分代码可回滚与系统可回滚

旧二进制可以部署，不代表它能读取新 Schema、理解新状态或撤销已经产生的外部副作用。

11.2 不可逆变更

以下通常属于难逆转：

- 删除或重编码权威数据；
- 改变稳定身份；
- 外部消费者不可控的契约；
- 不可补偿副作用；
- 权限和密钥泄露；
- 跨区域数据移动；
- irreversible encryption/key changes。

OPS-REQ-035：不可逆变更必须采用分阶段协议

常见模式：

```
expand
-> dual-read / compatible write
-> migrate
-> verify / reconcile
-> switch authority
-> observe
-> contract
```

不得在同一不可回退步骤中同时改变数据、代码、权限和权威边界。

11.3 Forward Recovery

当回滚不安全时，必须预先定义：

- 如何停止进一步损害；
- 如何识别已受影响对象；
- 如何补偿或修复；
- 如何验证收敛；
- 如何向用户和依赖方表达状态；
- 谁拥有最终完成判断。

12. 数据完整性、备份与恢复

12.1 数据分类

至少区分：

- 权威状态；
- 派生状态；
- 可重建缓存；
- 审计/证据；
- 临时 staging；
- 隔离数据；
- 用户可见结果。

恢复策略必须根据状态类型确定。

OPS-REQ-036：恢复必须从权威状态开始

不得将缓存、过期副本或临时文件当作恢复权威，除非系统契约明确它们拥有最终裁决权。

12.2 Backup Contract

```
backup:
  data_scope: "...
  authority_source: "...
  frequency: "...
  retention: "...
  encryption: "...
  immutability: "...
  cross_failure_domain: "...
  integrity_check: "...
  owner: "...
```

OPS-REQ-037：有备份不等于可恢复

必须验证：

- 备份可读取；
- 密钥和权限可用；
- 依赖版本可重建；
- 恢复顺序正确；
- 恢复后的数据完整性可校验；
- RTO/RPO 可达到；
- 恢复不会覆盖新的正确数据。

12.3 Restore Drill

演练至少记录：

```
scope
backup identity
restore target
time to detect
time to start
time to restore
time to validate
data loss / divergence
manual steps
failed steps
follow-up actions
```

OPS-REQ-038：恢复验证必须有业务 Oracle

“数据库启动成功”不代表业务恢复完成。必须验证关键实体、约束、lineage、计数、对账和用户可见结果。

12.4 Replay

Replay 必须定义：

- 原始输入是否不可变；
- pipeline/config/model/schema 版本；
- 重放身份；
- 副作用隔离；
- 重复和冲突；
- 容量与优先级；
- 结果对账；
- 旧结果替换/并存语义。

OPS-REQ-039：Replay 不得与实时流量争夺无界资源

必须有速率、配额、独立队列或优先级，保证恢复不会使生产再次失稳。

12.5 Reconciliation

对跨系统或非原子边界，应提供：

- 可枚举业务身份；
- 权威比较；
- 差异分类；
- 幂等修复；
- 审计；
- 完成标准。

13. RPO、RTO 与灾难恢复

13.1 定义

- RPO：允许丢失或需要重新生成的最大业务时间范围；
- RTO：本套系统一以目标业务能力实际中断时刻为起点，到通过业务 Oracle 确认恢复为终点的目标时间上限；不得按检测时刻重新起算；
- MTPD：业务可承受的最大中断时间；
- Recovery Capacity：恢复期间处理正常流量和积压的能力。

每次演练/事故至少记录以下时间点，使用一致时钟或注明时钟误差：

| 时间点 | 含义 |
|-------------|-------------------------|
| t0 | 目标业务能力开始中断 |
| t_detected | 被监测或人员发现 |
| t_decided | 决定启动恢复 |
| t_restored | 技术组件恢复可服务 |
| t_validated | 业务 Oracle 确认目标能力与数据状态恢复 |

实际恢复历时为 $t_{\text{validated}} - t_0$ ，与 RTO 比较；检测耗时、决策耗时、技术恢复耗时和验证耗时分开保留。若 t_0 不可精确确定，应报告最早/最晚可能时刻及恢复历时区间，以保守上界评估；无可靠边界时保持 UNKNOWN。可以另报“检测后恢复耗时”，但不得用该较短指标宣称满足以业务中断为起点的 RTO。

OPS-REQ-040：RPO/RTO 必须从业务后果推导

不能只使用云服务默认值。目标必须与数据语义、客户承诺、法律和恢复成本一致。

13.2 DR Scope

DR 计划必须覆盖：

- 故障场景；
- 受影响状态和依赖；
- 权威切换；
- DNS/身份/凭据；
- 数据一致性；
- 人员和沟通；
- 恢复验证；
- 回切；
- 事后对账。

OPS-REQ-041：多区域不自动等于 DR

如果区域共享控制面、密钥、身份、部署、错误配置或同步损坏，多区域可能复制故障而不是隔离故障。

13.3 演练等级

| 等级 | 内容 |
|-------------------------|----------------|
| Tabletop | 角色和决策路径演练 |
| Component Restore | 单组件恢复 |
| Partial Failover | 有限流量/租户切换 |
| Full Technical Failover | 整体技术切换，可能非权威 |
| Business DR Exercise | 业务、数据、沟通和回切全链路 |

OPS-REQ-042：R2/R3 必须周期演练

周期由风险、变化速度和业务要求决定。架构或依赖重大变化后,不应等待固定年度周期才重新验证。

14. 可观测性与诊断

14.1 四类信号

用户/业务结果

- completion success;
- correctness/conflict;
- latency/freshness;
- data completeness;
- business rejection。

系统状态

- queue depth/age;
- inflight;
- state transition;
- operating mode;
- dependency status;
- version composition。

资源

- CPU、memory、disk、network;
- connection pool;
- byte permits;

- retry budget;
- staging/quarantine。

保证与安全

- invariant violation;
- provenance mismatch;
- evidence/assumption drift;
- authz denial;
- break-glass;
- audit continuity。

OPS-REQ-043：可观测性必须从设计模型生成

指标和日志应映射到关键性质、状态机、队列、依赖、运行模式和恢复步骤。上线后随意添加资源图不能替代系统模型。

14.2 关联身份

日志、trace、结果和审计应使用稳定业务身份关联：

```
request / operation ID
source identity
result identity
pipeline/version
config/schema/model version
tenant/region
release ID
```

OPS-REQ-044：高基数必须受控

稳定身份不意味着把所有 ID 直接写入指标标签。应根据查询模式在 logs/traces/audit 中保存，指标使用聚合或受控维度。

14.3 Alert Contract

```
alert:
  alert_id: "ALERT-..."
  symptom_or_property: "..."
  threshold_or_rule: "..."
  severity: "..."
  user_impact: "..."
  owner: "..."
  first_actions: []
  runbook: "..."
  escalation: "..."
  auto_mitigation: "..."
  false_positive_review: "..."
```

OPS-REQ-045: 告警必须具备可执行响应

没有 Owner、Runbook 或明确行动的告警应被删除、降级为仪表盘信号或补齐控制，而不是长期制造噪声。

14.4 Dashboard

至少应能回答：

1. 用户是否受到影响？
2. 哪个运行模式和版本正在运行？
3. 压力积累在哪里？
4. 哪个依赖或故障域异常？
5. 恢复是否前进？
6. 是否存在 hard property violation？
7. 是否超出已验证范围？

15. Runbook 与操作安全

15.1 Runbook 最低内容

| |
|------------------------------|
| purpose and scope |
| preconditions |
| required authority |
| risk and blast radius |
| exact steps |
| verification after each step |
| abort conditions |
| rollback / forward recovery |
| evidence to retain |
| escalation |
| last exercised date |
| owner |

OPS-REQ-046：Runbook 必须可执行且经过演练

只描述原则、包含过期命令或依赖作者记忆的 Runbook 不构成生产能力。

15.2 操作分级

| 类型 | 要求 |
|---------------------------|---------------------------|
| Safe read-only diagnostic | 广泛可用，最小权限 |
| Routine mutation | 自动化、幂等、审计、受限范围 |
| High-risk mutation | 双人/审批、预检查、Change Envelope |
| Emergency mutation | break-glass、最小期限、全量审计 |

OPS-REQ-047：高风险操作应优先自动化为受控命令

不得要求值班人员在事故压力下手工拼接复杂数据库、Kubernetes 或云控制命令。自动化应提供预检查、范围限制、dry-run、审计和结果验证。

15.3 Drain 与 Shutdown

系统必须定义：

- 停止新准入；
- 等待或转移 inflight；
- checkpoint/commit；
- 超时后处理；

- 可观察完成；
- 被强制终止后的恢复。

OPS-REQ-048：优雅关闭不得依赖无限等待

必须有明确 deadline 和未完成工作的持久化/重投递/终态语义。

16. On-call 与运营所有权

16.1 On-call Readiness

生产准入前必须确认：

- 值班覆盖；
- 联系和升级路径；
- 权限；
- Runbook；
- 仪表盘；
- 依赖联系人；
- 事故角色；
- 新系统培训；
- 交接；
- 常见故障演练。

OPS-REQ-049：无 Operational Owner 不得进入核心生产

如果没有团队愿意承担告警、恢复和事故责任，系统还不是可运营产品。

16.2 值班负担

应观测：

- page 数量；
- 非行动告警比例；
- 夜间唤醒；
- 重复事件；
- 手工修复频率；
- 权限和工具缺口；
- 事故后待办关闭率。

OPS-REQ-050：长期人工补偿必须进入工程计划

持续依赖值班人员手工清理、重放、扩容或修复，说明系统存在结构性运行债务，不能被视为稳定状态。

17. 安全与运行控制

17.1 最小权限

生产身份应按：

- 角色；
- 环境；
- 租户；
- 操作；
- 时间；
- 资源范围；

进行限制。

OPS-REQ-051：读、写、管理和 break-glass 权限必须分离

拥有观测权限不应自动获得状态修改权限；自动化发布身份不应拥有无限人工控制权限。

17.2 Secret 与证书

必须定义：

- 来源；
- 轮换；
- 过期监测；
- 撤销；
- 备份/恢复；
- 使用范围；
- 审计；
- 紧急替换。

OPS-REQ-052：凭据轮换必须演练

未验证的轮换可能导致全局故障。关键证书、密钥和 token 的轮换应在到期前完成受控演练。

17.3 Audit

审计应覆盖：

- 特权操作；
- 权限变化；
- 生产发布；
- Exception 与 break-glass；
- 数据修复；
- 证据和门禁变更；
- 权威切换；
- 恢复和回放。

OPS-REQ-053：审计链中断必须可检测

审计缺失不能被视为“没有操作发生”。对高风险主体，审计不可用应限制变更或触发降级。

18. 第三方与供应商韧性

18.1 依赖评估

关键第三方必须评估：

- SLA/SLO 与实际表现；
- 速率、配额和突发；
- 数据导出和迁移；
- 版本/弃用通知；
- 安全与合规；
- 支持和升级路径；
- 区域和故障域；
- 单点依赖；
- 退出成本。

OPS-REQ-054：外部 SLA 不自动满足内部 SLO

必须考虑调用链、重试、配额、共享故障和测量口径差异。

18.2 供应商故障

应预定义：

- 缓存或降级；
- 用户可见语义；

- 暂停写入；
- 替代路径；
- 数据对账；
- 合同升级；
- 恢复后补偿。

OPS-REQ-055：关键供应商退出必须有可行路径

不一定需要随时双活，但必须知道数据如何导出、契约如何迁移、身份如何重建、最长退出时间和主要阻断。

19. 事故响应

19.1 Incident Lifecycle

Detect

- > Triage
- > Stabilize
- > Contain
- > Recover
- > Validate
- > Communicate
- > Learn
- > Assurance Delta

OPS-REQ-056：先稳定再追因

事故期间应优先限制损害、保护证据和恢复关键能力，避免在不安全状态下进行高风险根因实验。

19.2 角色

至少定义：

- Incident Commander；
- Operations Lead；
- Communications Lead；
- Subject Matter Expert；
- Scribe；
- Security/Compliance Liaison（如适用）。

OPS-REQ-057：事故指挥权必须明确

多人并发修改同一生产状态会扩大事故。Incident Commander 应控制优先级和高风险操作授权。

19.3 恢复完成

事故“恢复”必须至少验证：

- 用户结果恢复；
- 数据完整性；
- 积压收敛；
- 依赖稳定；
- 临时权限撤销；
- 运行模式回归；
- 证据保留；
- 未知和残余风险记录。

OPS-REQ-058：恢复后必须防止自动重新进入故障

重新打开流量、重试或自动扩容前，必须验证根因控制和容量边界，避免恢复动作触发第二次故障。

19.4 事故复盘

复盘应关注：

- 为什么系统允许损害发生；
- 检测为何及时或迟延；
- 哪些 Assumption 错误；
- 哪些控制、Evidence 和 Runbook 缺失；
- 组织和权限如何影响响应；
- Assurance Delta。

20. 韧性验证与演练

20.1 验证层级

| 层级 | 示例 |
|-------------|---|
| Component | crash、timeout、disk full、dependency errors |
| Integration | broker/store/identity 边界故障 |
| System | 端到端过载、混合版本、恢复 |
| Operational | Runbook、on-call、权限、沟通 |
| Disaster | 区域/控制面/凭据/数据恢复 |

OPS-REQ-059：演练必须有安全边界

每次演练应定义：

- 假设；
- 最大爆炸半径；
- 禁止动作；
- 观察指标；
- 停止条件；
- 恢复路径；
- 责任人；
- Evidence。

20.2 Game Day

Game Day 不应只验证技术故障，还应覆盖：

- 告警是否到达；
- 值班是否有权限；
- Runbook 是否准确；
- 依赖是否可协调；
- 业务是否理解降级；
- 沟通是否及时；
- 恢复是否可验证。

OPS-REQ-060：演练发现必须闭环

发现的问题应进入：

- Critical Property;
- Capacity/Failure Model;
- Runbook;
- Alert;
- Automation;
- Evidence;
- Assurance Delta;
- Owner 与期限。

21. Production Readiness Review

21.1 输入

R2/R3 Review 至少需要：

- Service Definition;
- 风险等级;
- Design/Architecture Decision;
- Critical Property 和状态模型;
- SLO/RPO/RTO;
- 依赖与故障域;
- Resource/Capacity Model;
- Operating Modes;
- Release/Recovery Plan;
- Observability/Runbook/On-call;
- Threat Model;
- Assurance Review;
- Release Evidence Bundle;
- Exception Register;
- Change Envelope。

21.2 阻断条件

以下情况默认 NO_GO 或 UNKNOWN：

- 业务完成和权威状态不清;
- 关键资源无上界;

- 高后果副作用的提交/ACK/恢复语义不清；
- 无 Operational Owner 或 on-call；
- 关键 SLO 无测量来源；
- 数据恢复未验证；
- 不可逆变更无前向修复；
- 发布无法停止或限制暴露；
- 阻断级 Evidence 失效或错绑；
- 关键 Exception 无期限或补偿控制；
- break-glass 无审计；
- 运行漂移无法检测。

OPS-REQ-061: Readiness Review 必须区分设计缺口与发布缺口

设计语义缺失通常不能通过临时运行条件补偿；发布环境、容量或证据缺口可以在严格条件下 CONDITIONAL_GO。

21.3 决策

Production Readiness Decision 使用 ESD-GOV-001 的状态，并必须说明：

```
scope
subject identity
risk tier
supported claims
blocking unknowns
exceptions
residual risks
change envelope
release authority
validity / review trigger
```

OPS-REQ-062: GO 只对声明范围有效

有限租户、单区域或某规模的 GO 不得被解释为全局通用生产资格。

22. 上线后观察与扩大

22.1 Observation Window

上线后观察至少覆盖：

- 正常流量；
- 一个有意义的峰值或业务周期；
- 队列和资源稳定；

- 错误预算；
- hard property；
- 依赖影响；
- 单位成本；
- 支持和 on-call 负担。

OPS-REQ-063：扩大必须基于成功标准而非时间自动发生

Canary 运行满固定小时不代表成功。必须满足预定义正确性、资源、恢复和用户结果条件。

22.2 Closure

Limited Production 结束时必须作出：

```
EXPAND  
HOLD  
ROLL_BACK  
FORWARD_REPAIR  
SUSPEND_READINESS
```

并保存 Evidence 和理由。

23. 持续运行评审

23.1 触发器

以下事件触发 Readiness 复审：

- SLO 长期违约；
- 重大事故或数据修复；
- 容量或工作负载越界；
- 依赖、区域、Owner 或 on-call 变化；
- 证据失效；
- 关键 Exception 到期；
- 新不可逆状态或权限；
- 长期手工补偿；
- 恢复演练失败；
- 至少每 12 个月一次。

OPS-REQ-064：运行债务必须可见

长期告警噪声、手工修复、容量不足、过期 Runbook、未关闭 Exception 和未演练恢复都应进入 Operating Risk Register，而不是被日常操作吸收。

23.2 Retirement Readiness

系统退出也必须设计：

- 停止新依赖；
- 数据保留/删除；
- 消费者迁移；
- 权限撤销；
- 证据保留；
- 域名、证书、队列、存储和账单清理；
- 最终审计；
- Owner 解除。

OPS-REQ-065：废弃不等于无人维护

在所有状态、流量、权限和依赖安全退出前，系统仍需 Operational Owner 和最低运行能力。

24. 按风险等级的最低运行要求

| 能力 | R0 | R1 | R2 | R3 |
|--------------------------|------|------|------|-----------|
| 退出/删除条件 | 必须 | 必须 | 必须 | 必须 |
| 业务完成语义 | 关键项 | 必须 | 必须 | 必须 |
| SLI/SLO | 可选 | 基础 | 完整 | 完整 + 独立审查 |
| Operational Owner | 可选 | 必须 | 必须 | 必须 |
| 运行模式 | 最小 | 核心 | 完整 | 完整 + 权限控制 |
| Resource Bounds | 硬上限 | 必须 | 完整 | 完整 + 故障降容 |
| On-call | 不适用 | 按需 | 必须 | 必须 + 演练 |
| Recovery Drill | 可选 | 核心恢复 | 必须 | 周期性完整演练 |
| Change Envelope | 首次暴露 | 重大变更 | 必须 | 必须 + 自动停止 |
| DR | 不适用 | 按业务 | 必须评估 | 必须验证 |
| Evidence Bundle | 最小 | 基础 | 完整 | 完整 + 独立评价 |
| Incident Assurance Delta | 可选 | 重大事故 | 必须 | 必须 |

25. Production Readiness Decision 模板

```
# Production Readiness Decision

- Decision ID:
- System / Change:
- Risk tier:
- Subject identity:
- Current lifecycle state:
- Requested lifecycle state:
- Review date:
- Release Authority:

## 1. Business capability and completion semantics

## 2. Critical properties and authoritative states

## 3. SLI / SLO / RPO / RTO

## 4. Dependencies and failure domains

## 5. Operating modes

## 6. Resource bounds, capacity and recovery headroom

## 7. Overload, backpressure and retry policy

## 8. Release, migration and recovery strategy

## 9. Data integrity, backup, restore and reconciliation

## 10. Observability, alerting, runbooks and on-call

## 11. Security and privileged operations

## 12. Assurance Review and Evidence Bundle

## 13. Exceptions, unknowns and residual risks

## 14. Change Envelope

## 15. Decision
```

字面文本 · continued

```
GO / CONDITIONAL_GO / NO_GO / UNKNOWN

## 16. Conditions and deadlines

## 17. Review and invalidation triggers
```

26. Operating Mode Contract 模板

空表模板 · 26. Operating Mode Contract 模板

| | |
|----------------|--|
| Mode | |
| Purpose | |
| Allowed | |
| Forbidden | |
| Entry Criteria | |
| Exit Criteria | |
| Signals | |
| Authority | |
| Runbook | |

27. Dependency Resilience Contract 模板

```
dependency:
  dependency_id: "DEP-..."
  name: "..."
  owner: "..."
  contract:
    availability: "..."
    latency: "..."
    timeout: "..."
    capacity_quota: "..."
    retryable_errors: []
    idempotency: "..."
    version_compatibility: "..."
  criticality: "BLOCKING|DEGRADABLE|OPTIONAL"
```

字面文本 · continued

```
failure_modes: []
detection: []
fallback: "...
escalation: "...
recovery_validation: "...
review_trigger: "...
```

28. Recovery Drill Record 模板

```
# Recovery Drill DRILL-XXX

- System:
- Scenario:
- Risk tier:
- Date:
- Participants:
- Source state / backup identity:
- Target environment:

## Objectives

## Preconditions and safety boundary

## Expected RPO / RTO

## Timeline

## Actions

## Business validation

## Actual RPO / RTO

## Failed or manual steps

## Evidence

## Residual risks

## Required changes

## Assurance Delta
```

29. 最终 Production Readiness Checklist

业务与语义

- ☐ 服务能力、用户和业务完成语义明确；
- ☐ 权威状态、稳定身份和关键性质明确；
- ☐ 风险等级和失败后果有依据；
- ☐ 设计批准和发布批准已区分。

目标与运行模式

- ☐ SLI 基于用户/业务结果；
- ☐ SLO、RPO、RTO 和测量来源明确；
- ☐ hard property 未被错误预算化；
- ☐ NORMAL、DEGRADED、RECOVERY 等模式有契约；
- ☐ 模式切换、权限和退出验证可执行。

故障与资源

- ☐ 关键依赖和真实故障域已建模；
- ☐ timeout、retry budget、熔断和背压闭合；
- ☐ 所有可增长资源有上界；
- ☐ 峰值、N-1、积压和恢复容量经过计算；
- ☐ 单位经济性处于可接受范围。

发布与迁移

- ☐ Change Envelope 与 Evidence 范围一致；
- ☐ 发布可以暂停和停止；
- ☐ Canary 包含正确性和资源停止条件；
- ☐ mixed-version 行为经过验证；
- ☐ 回滚、前向修复和不可逆步骤明确；
- ☐ 迁移具有 expand/migrate/verify/contract 纪律。

数据与恢复

- ☐ 权威、派生、缓存和 staging 已分类；
- ☐ 备份完整性和密钥可用性已验证；
- ☐ Restore/Replay/Reconciliation 有业务 Oracle；
- ☐ RPO/RTO 在代表性演练中得到支持；
- ☐ 恢复流量不会使系统再次失稳；
- ☐ DR 计划覆盖控制面、身份、数据和人员。

可观测性与运营

- ☐ 用户结果、状态、资源、依赖和保证信号可观察；
- ☐ stable IDs 支持端到端关联；
- ☐ 告警有 Owner、Runbook 和行动；
- ☐ Runbook 经过演练且命令仍有效；
- ☐ on-call、权限、升级和依赖联系人明确；
- ☐ 高风险操作自动化、受限、审计。

安全与事故

- ☐ 读、写、管理和 break-glass 权限分离；
- ☐ secret/证书轮换和撤销经过验证；
- ☐ 审计链完整且中断可检测；
- ☐ 事故角色、通信和证据保留明确；
- ☐ 重大事故会产生 Assurance Delta。

证据与裁决

- ☐ Assurance Review 达到 Decision-grade；
- ☐ Release Evidence Bundle 绑定当前对象；
- ☐ 阻断级 Evidence 为 CURRENT；
- ☐ Exception 有范围、期限和补偿控制；
- ☐ 高后果 UNKNOWN 未被静默放行；
- ☐ GO 的范围、有效期和失效触发明确。

30. 结论

生产就绪不是“代码已合并、镜像已构建、部署已成功”。它是一项持续的系统能力：团队知道什么是业务完成，系统知道自己处于什么模式，资源和故障不会无界扩散，发布可以被限制和停止，数据可以从权威事实恢复，值班人员有权限和 Runbook，关键主张有当前 Evidence，环境漂移会使过期结论失效。

最终闭环为：

```
Design semantics
-> Production controls
-> Readiness evidence
-> Limited exposure
-> Runtime observation
-> Recovery and incident learning
-> Revalidated readiness
```

当系统能够在正常、过载、降级、恢复和演进中保持这些能力时，它才真正从“可以部署的软件”成为“可以长期负责的生产系统”。

条款定位索引（生成导航）

| | |
|--------------------------------------|-----|
| ESD-GOV-001 · 系统设计与工程保证治理及符合性规范 | |
| GOV-REQ-001: 风险决定控制深度 | 107 |
| GOV-REQ-002: 裁剪必须显式 | 107 |
| GOV-REQ-003: 所有关键责任必须具名 | 107 |
| GOV-REQ-004: 职责分离与风险匹配 | 107 |
| GOV-REQ-005: 未知必须保留 | 108 |
| GOV-REQ-006: 例外是有期限的受控状态 | 108 |
| GOV-REQ-007: 符合性必须可追溯 | 108 |
| GOV-REQ-008: 治理系统自身受治理 | 108 |
| GOV-REQ-009: 风险等级必须有依据 | 110 |
| GOV-REQ-010: 等级下调必须审查 | 110 |
| GOV-REQ-011: 风险可随生命周期变化 | 111 |
| GOV-REQ-012: 技术 Owner 与运营 Owner 不得缺失 | 112 |
| GOV-REQ-013: 最终批准者必须具备风险授权 | 112 |
| GOV-REQ-014: Reviewer 必须可挑战而非仅确认 | 112 |
| GOV-REQ-015: 保证面变更需要独立性 | 113 |
| GOV-REQ-016: 制品必须与对象和版本绑定 | 114 |
| GOV-REQ-017: 模板可裁剪，语义不可丢失 | 115 |
| GOV-REQ-018: CONDITIONAL_GO 必须可执行 | 116 |
| GOV-REQ-019: 高后果 UNKNOWN 默认阻断 | 116 |
| GOV-REQ-020: Gate 决策必须保存反对意见 | 116 |
| GOV-REQ-021: Exception 必须绑定具体要求 | 117 |
| GOV-REQ-022: Exception 必须到期 | 117 |
| GOV-REQ-023: 补偿控制必须降低同类风险 | 118 |
| GOV-REQ-024: 例外不能静默继承 | 118 |
| GOV-REQ-025: break-glass 不得成为常规路径 | 118 |
| GOV-REQ-026: 符合性必须按适用剖面判断 | 120 |
| GOV-REQ-027: 符合性不是产品质量总评 | 120 |
| GOV-REQ-028: 评估结论必须有有效期或复审触发 | 128 |
| GOV-REQ-029: 口头批准不构成 Gate 决策 | 130 |
| GOV-REQ-030: 发布必须绑定当前对象 | 130 |

| | |
|--|-----|
| GOV-REQ-031：发布权限不得绕过设计阻断 | 130 |
| GOV-REQ-032：紧急变更必须补偿复审 | 131 |
| GOV-REQ-033：重复例外必须触发规范复审 | 132 |
| GOV-REQ-034：记录保留期与风险匹配 | 133 |
| GOV-REQ-035：指标必须与风险学习关联 | 134 |
| ESD-ASSURANCE-001 · 工程保证论证、证据与裁决规范 | |
| ASSUR-REQ-001：主张必须具有类型匹配的验证义务 | 141 |
| ASSUR-REQ-002：主张、机制和证据必须分离 | 142 |
| ASSUR-REQ-003：Evidence 必须绑定具体对象 | 142 |
| ASSUR-REQ-004：负面证据不得被删除或隐藏 | 142 |
| ASSUR-REQ-005：没有证据时保持未知 | 142 |
| ASSUR-REQ-006：局部证据不能自动推出端到端性质 | 143 |
| ASSUR-REQ-007：Evidence 具有生命周期 | 143 |
| ASSUR-REQ-008：裁决必须保留覆盖边界 | 143 |
| ASSUR-REQ-009：保证面自身必须 fail-closed | 143 |
| ASSUR-REQ-010：不能只用可变标签标识对象 | 144 |
| ASSUR-REQ-011：设计 Assurance 与发布 Assurance 必须区分 | 144 |
| ASSUR-REQ-012：Claim 必须有作用域 | 146 |
| ASSUR-REQ-013：Claim 必须有违反定义 | 146 |
| ASSUR-REQ-014：关键 Assumption 必须可监测或可复审 | 147 |
| ASSUR-REQ-015：上游 Guarantee 必须满足下游 Assumption | 148 |
| ASSUR-REQ-016：分解必须保持完整性 | 149 |
| ASSUR-REQ-017：不得用数量替代充分性 | 149 |
| ASSUR-REQ-018：阻断级 Defeater 未解决时不得给出支持性结论 | 150 |
| ASSUR-REQ-019：Counterevidence 与支持性 Evidence 同等保留 | 150 |
| ASSUR-REQ-020：阻断级 Claim 应使用证据多样性 | 151 |
| ASSUR-REQ-021：Evidence 质量必须与 Claim 关键性匹配 | 153 |
| ASSUR-REQ-022：不能只评价执行是否成功 | 153 |
| ASSUR-REQ-023：Evidence ID 必须稳定且唯一 | 154 |
| ASSUR-REQ-024：原始制品和摘要必须区分 | 154 |
| ASSUR-REQ-025：Evidence 不得只依赖易失链接 | 154 |
| ASSUR-REQ-026：状态不得跨层偷换 | 156 |
| ASSUR-REQ-027：失效必须传播到 Claim 与 Gate | 156 |
| ASSUR-REQ-028：失效规则必须机器可判定或具名负责 | 157 |

| | |
|--|-----|
| ASSUR-REQ-029: Oracle 必须明确 | 158 |
| ASSUR-REQ-030: 关键失败窗口必须枚举 | 158 |
| ASSUR-REQ-031: 性能 Evidence 必须使用代表性分布 | 158 |
| ASSUR-REQ-032: 工具错误必须显式 | 159 |
| ASSUR-REQ-033: 高风险验证规则变更必须受保护 | 159 |
| ASSUR-REQ-034: 验证器与被验证对象不得共享未经审查的同一错误假设 | 159 |
| ASSUR-REQ-035: 阻断级 Claim 必须有 Coverage Argument | 160 |
| ASSUR-REQ-036: 未观测到违反不等于违反不存在 | 161 |
| ASSUR-REQ-037: Release Composition 必须唯一标识 | 161 |
| ASSUR-REQ-038: 混合版本窗口必须进入论证 | 161 |
| ASSUR-REQ-039: 不得用总分掩盖阻断项 | 162 |
| ASSUR-REQ-040: 父 Claim 状态必须可解释 | 163 |
| ASSUR-REQ-041: Assurance Reviewer 不自动拥有风险接受权 | 164 |
| ASSUR-REQ-042: Decision Authority 不得改写证据事实 | 164 |
| ASSUR-REQ-043: Bundle 必须防止证据混淆 | 165 |
| ASSUR-REQ-044: Bundle 必须包含失败和限制 | 165 |
| ASSUR-REQ-045: Bundle 必须可长期验证 | 165 |
| ASSUR-REQ-046: 暴露不得超出 Evidence 范围 | 166 |
| ASSUR-REQ-047: 停止条件必须对应关键性质 | 166 |
| ASSUR-REQ-048: 运行监测必须映射到 Claim 或 Assumption | 167 |
| ASSUR-REQ-049: Drift 必须触发结论降级 | 167 |
| ASSUR-REQ-050: 重大事故必须产生 Assurance Delta | 168 |
| ASSUR-REQ-051: 事故关闭不得只依赖文字提醒 | 168 |
| ASSUR-REQ-052: 缺失或错绑 Evidence 不得为通过 | 169 |
| ASSUR-REQ-053: 用于裁决的 Evidence 不得静默修改 | 169 |
| ASSUR-REQ-054: 门禁规则和 Exception 逻辑必须版本化 | 169 |
| ASSUR-REQ-055: 高风险裁决必须保留职责分离 | 169 |
| ASSUR-REQ-056: Assurance Plane 不可用不得默认放行 | 169 |
| ASSUR-REQ-057: 例外必须进入 Evidence Bundle | 169 |
| ASSUR-REQ-058: 运行漂移必须使证据结论失效 | 169 |
| ASSUR-REQ-059: Evidence Producer 与 Reviewer 的独立性风险匹配 | 170 |
| ASSUR-REQ-060: Assurance Author 不得隐藏不利材料 | 170 |
| ESD-OPS-001 · 生产就绪与运行韧性标准 | |
| OPS-REQ-001: 生产完成语义必须明确 | 178 |

| | |
|--|-----|
| OPS-REQ-002: 运行模式必须显式 | 178 |
| OPS-REQ-003: 所有资源与积压必须有界 | 178 |
| OPS-REQ-004: 过载必须显式处置 | 178 |
| OPS-REQ-005: 恢复能力必须经过验证 | 178 |
| OPS-REQ-006: 发布是受控实验 | 179 |
| OPS-REQ-007: 可观测性必须支持行动 | 179 |
| OPS-REQ-008: Owner 必须接受 Day-2 责任 | 179 |
| OPS-REQ-009: 生产资格必须绑定当前对象 | 179 |
| OPS-REQ-010: 运行漂移必须降低确定性 | 179 |
| OPS-REQ-011: 服务边界必须以能力和状态定义 | 180 |
| OPS-REQ-012: 依赖必须包含非软件主体 | 181 |
| OPS-REQ-013: 生产资格可以撤销 | 182 |
| OPS-REQ-014: 扩大暴露必须重新检查资格 | 182 |
| OPS-REQ-015: SLI 必须测量结果而不只是资源 | 182 |
| OPS-REQ-016: SLO 与正确性性质不得混写 | 183 |
| OPS-REQ-017: 错误预算不得覆盖硬属性违反 | 183 |
| OPS-REQ-018: 模式转换必须有状态机语义 | 186 |
| OPS-REQ-019: 降级不得静默改变业务事实 | 186 |
| OPS-REQ-020: 恢复模式容量必须单独建模 | 186 |
| OPS-REQ-021: 副本必须跨真实故障域 | 187 |
| OPS-REQ-022: 调用超时必须小于上游 deadline | 187 |
| OPS-REQ-023: 重试必须有共享预算 | 187 |
| OPS-REQ-024: 容量必须以工作量而不只是请求数建模 | 190 |
| OPS-REQ-025: 峰值和恢复必须分别计算 | 190 |
| OPS-REQ-026: 容量结论必须绑定环境和语料 | 191 |
| OPS-REQ-027: 成本控制不得破坏硬性质 | 191 |
| OPS-REQ-028: 先限制入口，再限制内部损害 | 192 |
| OPS-REQ-029: 队列满载行为必须确定 | 192 |
| OPS-REQ-030: 不得随机丢弃关键业务事实 | 193 |
| OPS-REQ-031: 发布流程必须支持停止 | 193 |
| OPS-REQ-032: Change Envelope 必须与 Evidence 范围一致 | 194 |
| OPS-REQ-033: Canary 不能只观察错误率 | 194 |
| OPS-REQ-034: 必须区分代码可回滚与系统可回滚 | 195 |
| OPS-REQ-035: 不可逆变更必须采用分阶段协议 | 196 |

| | |
|---|-----|
| OPS-REQ-036: 恢复必须从权威状态开始 | 197 |
| OPS-REQ-037: 有备份不等于可恢复 | 197 |
| OPS-REQ-038: 恢复验证必须有业务 Oracle | 198 |
| OPS-REQ-039: Replay 不得与实时流量争夺无界资源 | 198 |
| OPS-REQ-040: RPO/RTO 必须从业务后果推导 | 199 |
| OPS-REQ-041: 多区域不自动等于 DR | 199 |
| OPS-REQ-042: R2/R3 必须周期演练 | 200 |
| OPS-REQ-043: 可观测性必须从设计模型生成 | 201 |
| OPS-REQ-044: 高基数必须受控 | 201 |
| OPS-REQ-045: 告警必须具备可执行响应 | 202 |
| OPS-REQ-046: Runbook 必须可执行且经过演练 | 203 |
| OPS-REQ-047: 高风险操作应优先自动化为受控命令 | 203 |
| OPS-REQ-048: 优雅关闭不得依赖无限等待 | 204 |
| OPS-REQ-049: 无 Operational Owner 不得进入核心生产 | 204 |
| OPS-REQ-050: 长期人工补偿必须进入工程计划 | 205 |
| OPS-REQ-051: 读、写、管理和 break-glass 权限必须分离 | 205 |
| OPS-REQ-052: 凭据轮换必须演练 | 205 |
| OPS-REQ-053: 审计链中断必须可检测 | 206 |
| OPS-REQ-054: 外部 SLA 不自动满足内部 SLO | 206 |
| OPS-REQ-055: 关键供应商退出必须有可行路径 | 207 |
| OPS-REQ-056: 先稳定再追因 | 207 |
| OPS-REQ-057: 事故指挥权必须明确 | 208 |
| OPS-REQ-058: 恢复后必须防止自动重新进入故障 | 208 |
| OPS-REQ-059: 演练必须有安全边界 | 209 |
| OPS-REQ-060: 演练发现必须闭环 | 210 |
| OPS-REQ-061: Readiness Review 必须区分设计缺口与发布缺口 | 211 |
| OPS-REQ-062: GO 只对声明范围有效 | 211 |
| OPS-REQ-063: 扩大必须基于成功标准而非时间自动发生 | 212 |
| OPS-REQ-064: 运行债务必须可见 | 213 |
| OPS-REQ-065: 废弃不等于无人维护 | 213 |